

Operating System

Process Threads



Contents

1. Program and process
2. Process state diagram
3. Operations of process
4. Interprocess Communication
5. Concurrency
6. Multithreading
7. Process scheduling

Process

Process is a **program in execution**

Process execution must progress in sequential fashion

Program is a **passive entity** stored on disk (executable file), process is **active entity**

Program becomes process when executable file loaded into memory

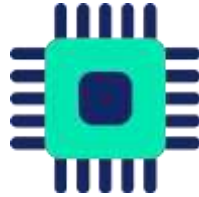
Executable program can be loaded into memory

1. By mouse click
2. By command line

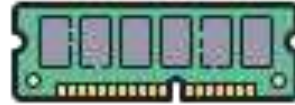
One program can be several processes

- Consider a program being executed by multiple users

Process



Processor



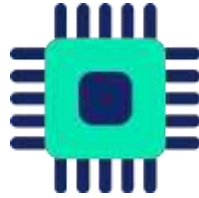
RAM



HDD

- A program is written using programming languages like Java, Python etc.
- The program is also referred to as source code
- After compilation of source code a byte code (executable) is created
- Bytecode is initially placed on disk like HDD/SSD

Process



Processor



RAM



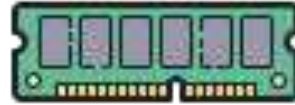
HDD

- When user run executable file using mouse or command line interface, executable and data related to program brought into RAM
- Memory gets allocated to code and data in RAM

Process



Processor



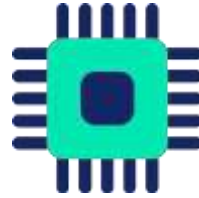
RAM



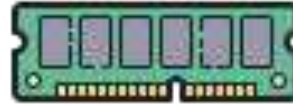
HDD

- Processor reads the data and instructions from memory and perform computations.
- Read/write happens on main memory.

Process



Processor



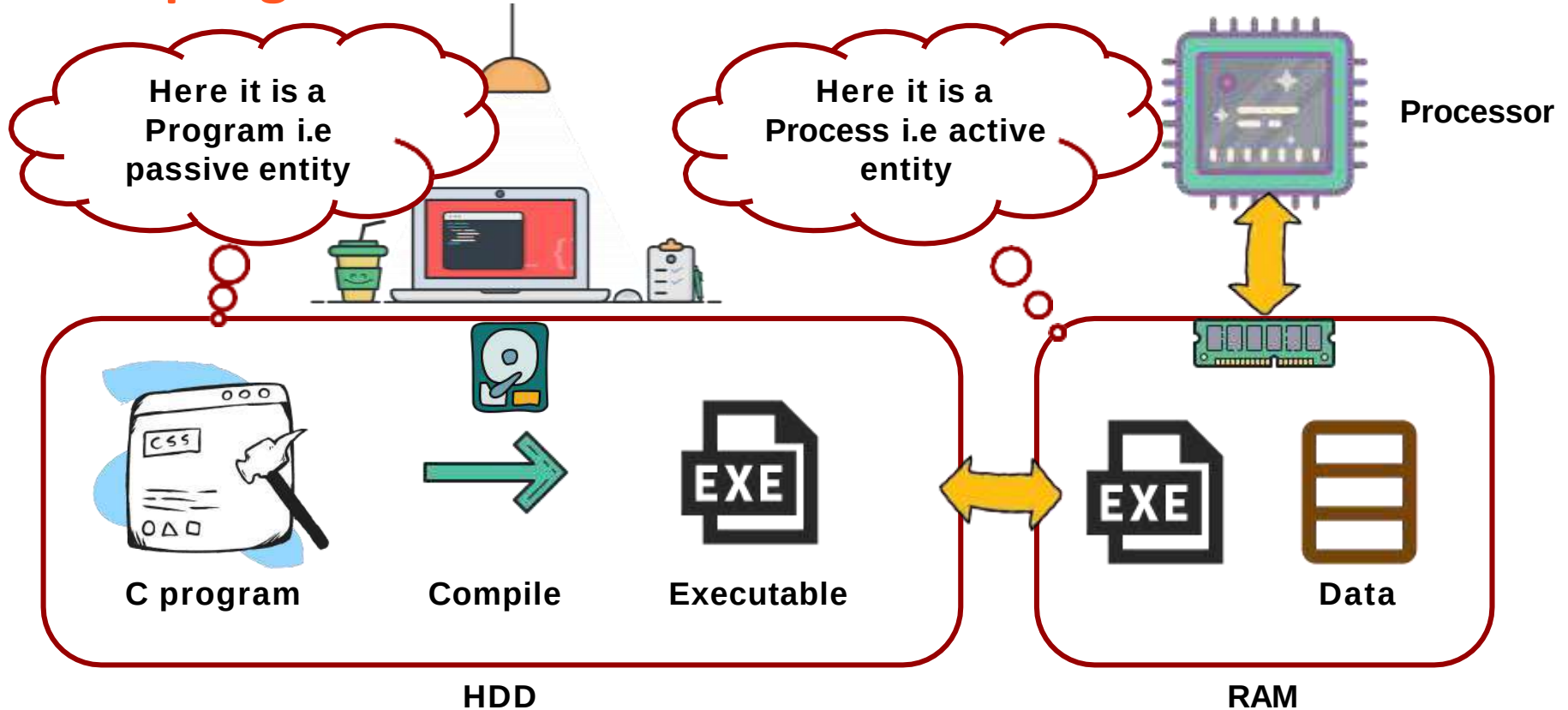
RAM



HDD

- After program execution source code and data related to code is removed from memory
- The executable remains on HDD/SSD

A C program execution



Process structure

```
1. Int a,b,c,Result
2. Int call_func(int a, int b)
3. {
4.     Int x = a*b
5.     Int y = a-b
6.     Return (y+x)
7. }
8. Int main()
9. {
10. Read a,b,c
11. Result = Call_func(a,b,c)
12. print(Results)
13. Return main
14. }
```

A process is program under execution
A process is more than program code
It also includes

- **Process ID**
- **program counter**
- **registers**
- **state**
- **open files pointers**
- **children**
- **Current directory, etc.**

Process Management

(Processes and Threads)

Process:

A process can be thought of as a program in execution.

Thread:

A thread is the unit of execution within a process. A process can have anywhere from just one thread to many threads.

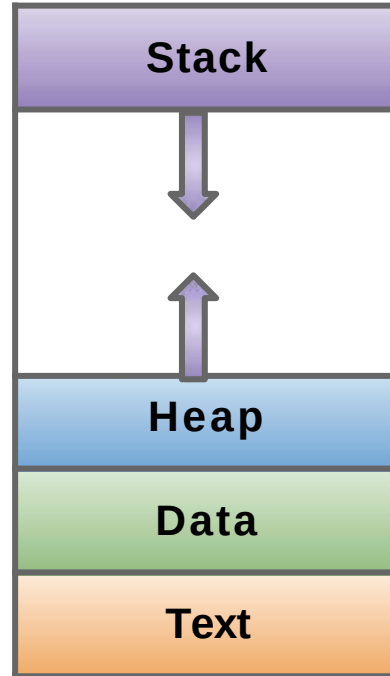


Feature	Process	Thread
Definition	A process is an independent program in execution with its own memory space.	A thread is a lightweight unit of a process that shares the same memory.
Memory	Has its own memory (code, data, stack, heap).	Shares memory with other threads of the same process (except for its own stack).
Communication	Inter-process communication (IPC) is slow and complex (e.g., pipes, sockets).	Threads can communicate easily via shared memory.
Overhead	High overhead to create and manage.	Low overhead – faster to create and switch.
Independence	Processes are independent – one crash doesn't affect others.	Threads are dependent – one thread crashing can affect the whole process.
Execution	Can execute independently.	Executes as part of a process.

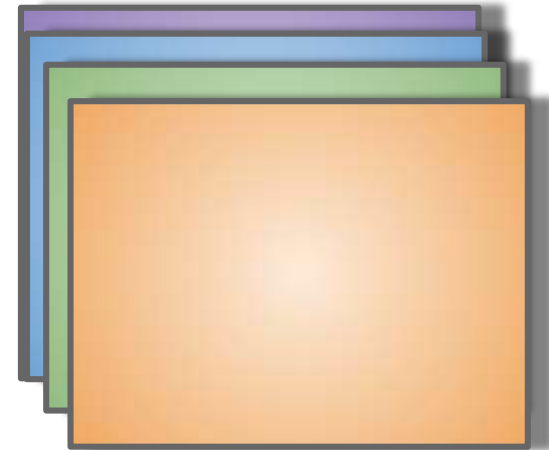
Process in Memory

```
1. Int a,b,c,Result
2. Int call_func(int a, int b)
3. {
4.     Int x = a*b
5.     Int y = a-b
6.     Return (y+x)
7. }
8. Int main()
9. {
10. Read a,b,c,Result
11. Result = Call_func(a,b)
12. print(Results)
13. Return main
14. }
```

Max

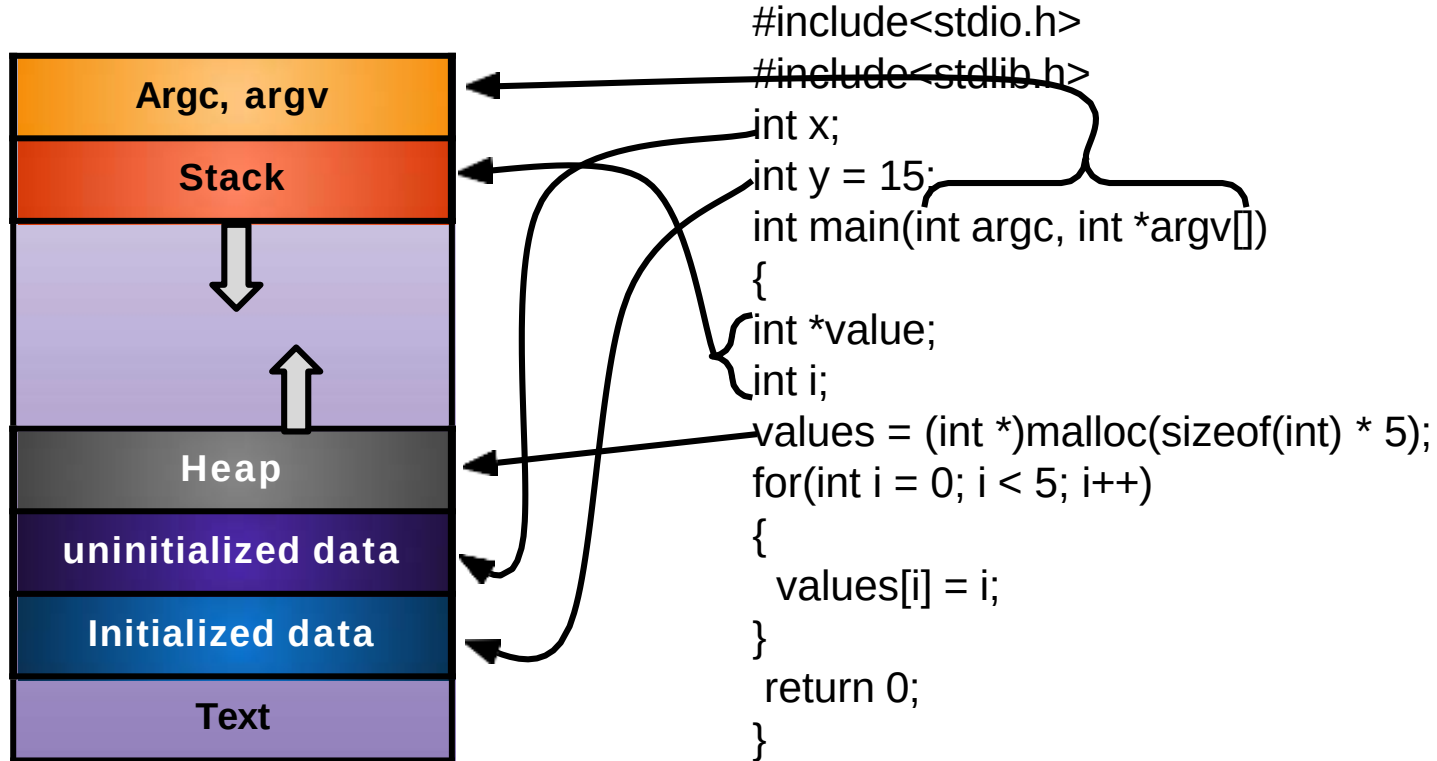


0



A program counter register keep track current instruction being executed and processor registers holds the variable values

Memory layout of a C program



Process states

New

The Process is being created

Ready

The process is waiting to be assigned to processor

Running

Instructions are being executed

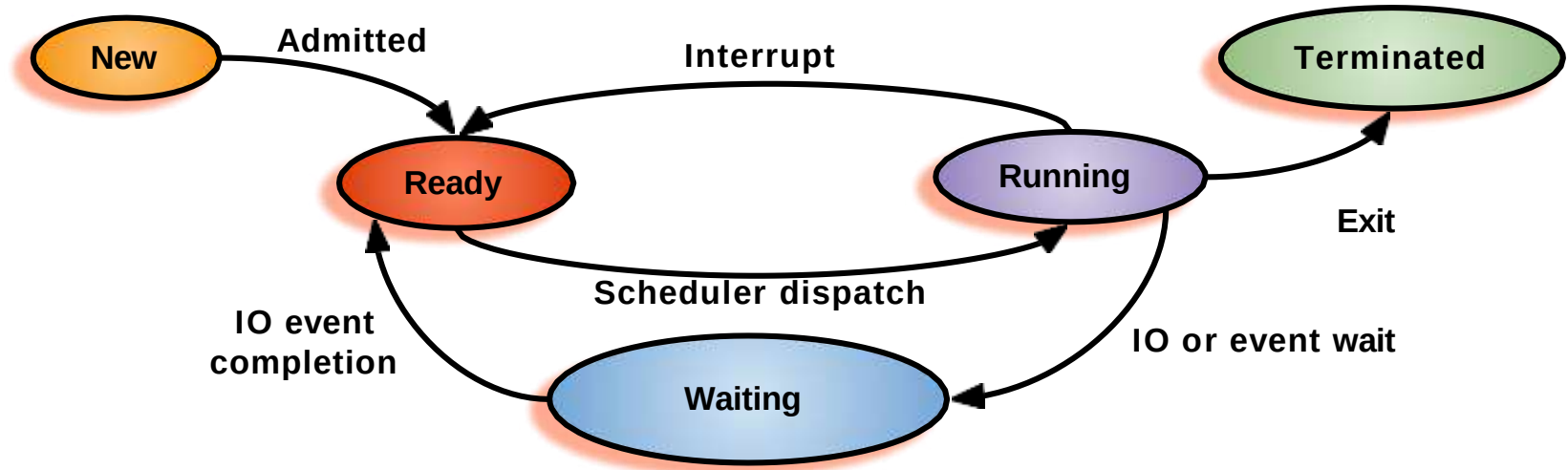
Waiting

The process is waiting for some event to occur

Terminated

The process completes its execution

Process state diagram

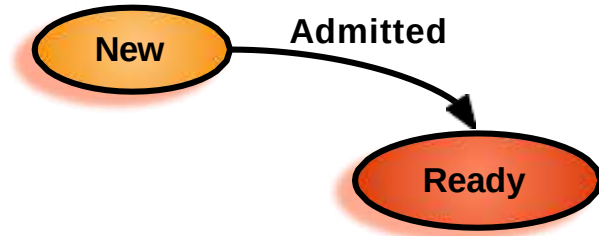


Process state diagram



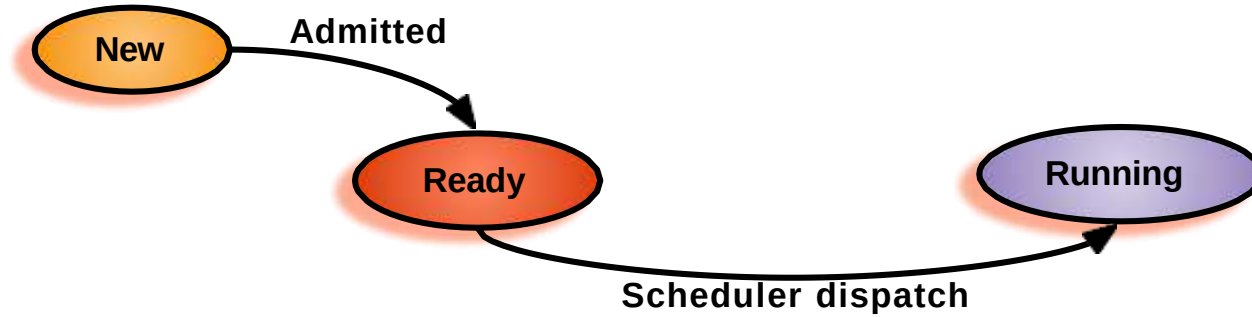
- When user clicks or run executable, data and source code of the program copied from disk into main memory after memory gets allocated.
- A process is just created and not assigned with any CPU resources

Process state diagram



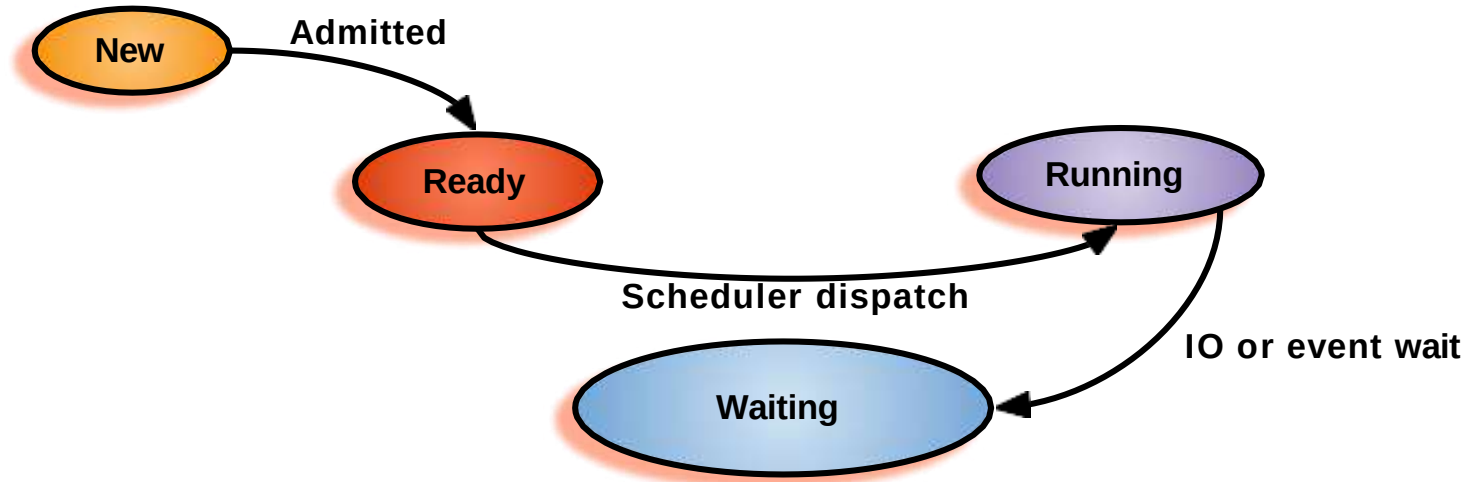
- After allocation of memory process is ready to execute
- Process is placed into ready queue by a **long term scheduler**
- A process will get chance to CPU resource based on **short term scheduler**
- Process may not get CPU for execution immediately as other process running on CPU

Process state diagram



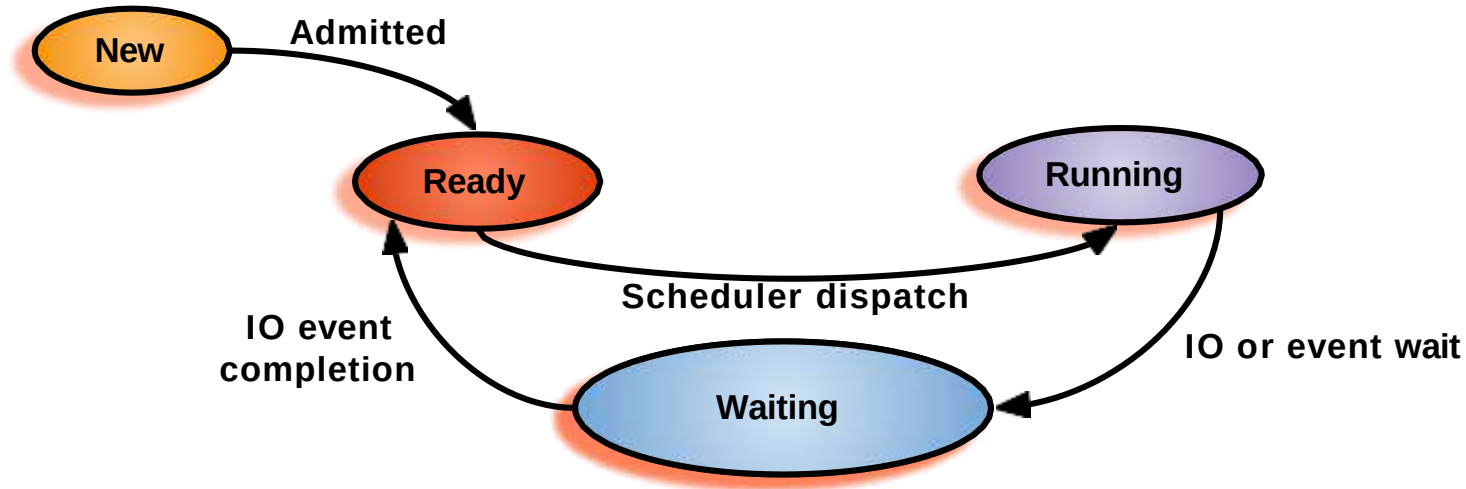
- When scheduler (short term) decides to execute the process, process moves to running state
- In running state process got the CPU and its executing

Process state diagram



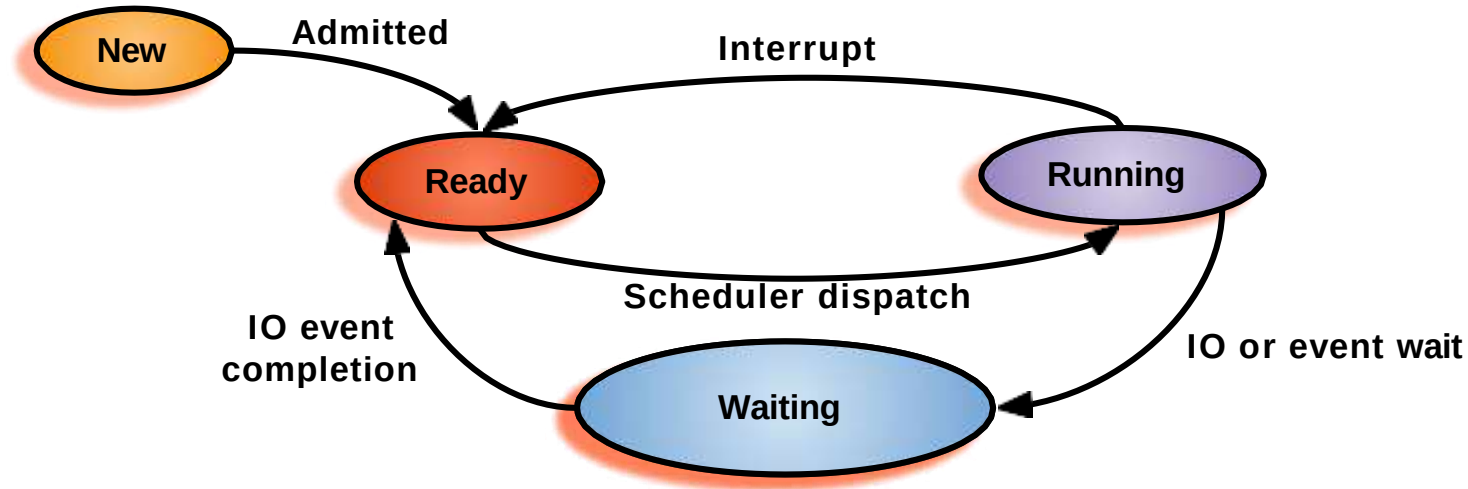
- During process execution process may expect IO operation such as read input from the user from keyboard
- Users are much slower as compared to processor
- Process should be kept in waiting state so that other processes get chance for CPU

Process state diagram



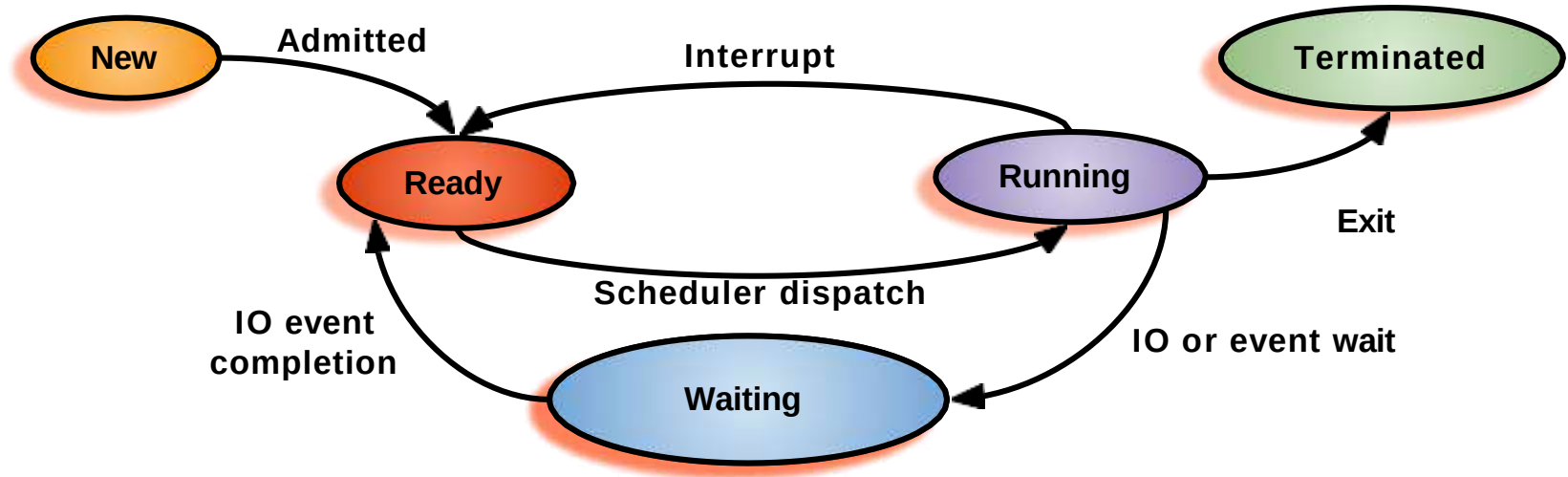
- On completion of IO, process will again kept in ready state
- Short term scheduler takes decision to keep process into running state again.

Process state diagram



- Sometimes process is given with time quanta to execute on CPU
- After completion of time quanta process gets interrupted and removed from running state to ready state

Process state diagram



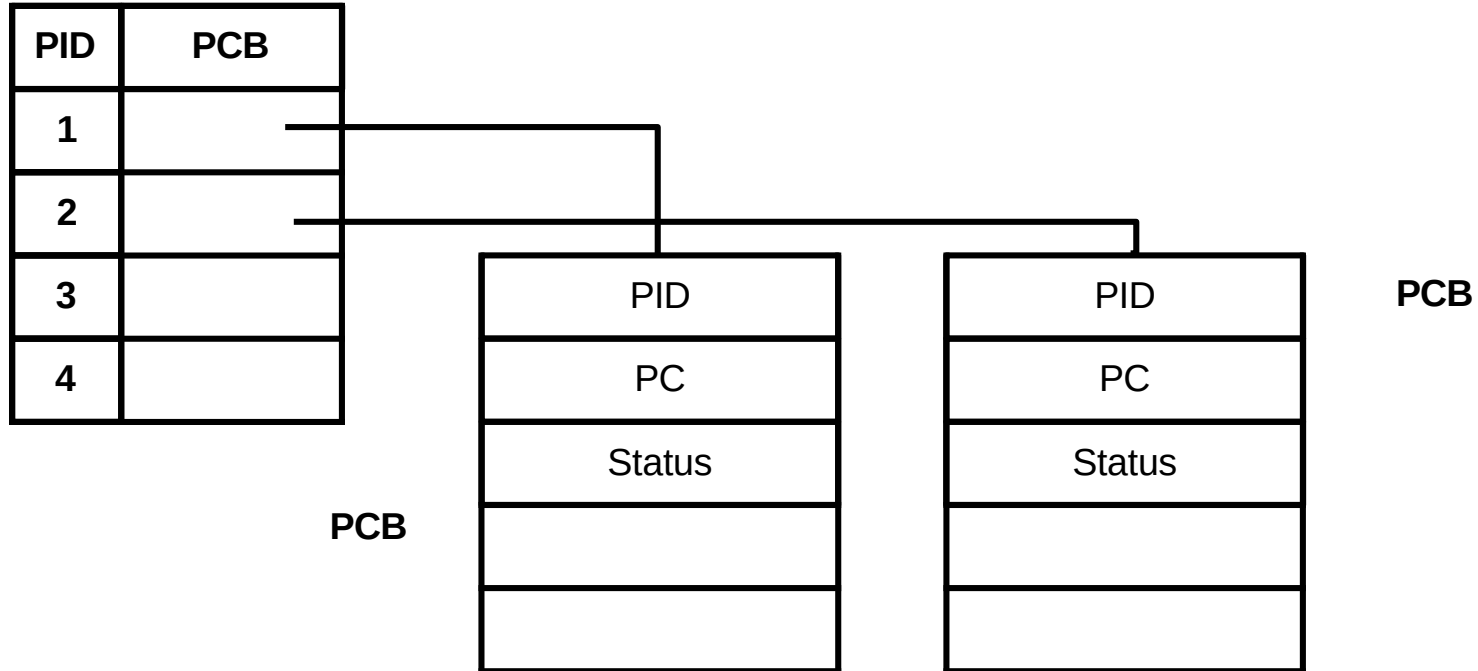
- After completion of process execution it moves to terminated state

Process Control Block (PCB)

- Every process has Process Control Block (PCB) and Information associated with a process is stored in PCB
 1. **Process state** - Ready, running, terminated, etc.
 2. **Process ID** - Unique identification number assigned when process gets created.
 3. **Program counter** - Location of the next instruction to be executed
 4. **CPU registers** - Contents of all process centric registers.
 5. **CPU scheduling information** - Priorities, scheduling queue pointers, etc.
 6. **Memory management information** - Memory allocation to process in code, data, stack and heap segment and their limits.
 7. **Accounting information** - CPU used, clock time elapsed since start, time limits.
 8. **I/O status information** - I/O devices allocated to process, list of open files, etc.

Process representation in Linux

Process Table



Process Scheduling

- At any point of time number of processes are ready for the execution
- Operating system has to decide which process to be executed next
- The main objective of process scheduling is **to maximize CPU utilization.**
- Process “gives” up the CPU under two conditions, **I/O request** and after **N units of time have elapsed**
- Once process “gives” up the CPU it is added to the ready queue
- Process scheduler selects among available processes in the ready queue for the next execution on CPU.

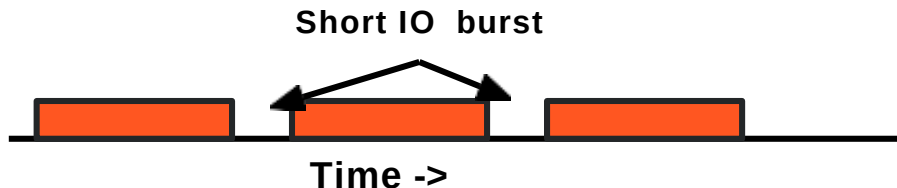
Process Scheduling Queues

- OS maintains scheduling queues of process
1. **Job queue** - Set of all processes in the system
 2. **Ready queue** - set of processes residing in the main memory, ready and waiting to execute.
 3. **Device queue** - Set of processes waiting for an IO device

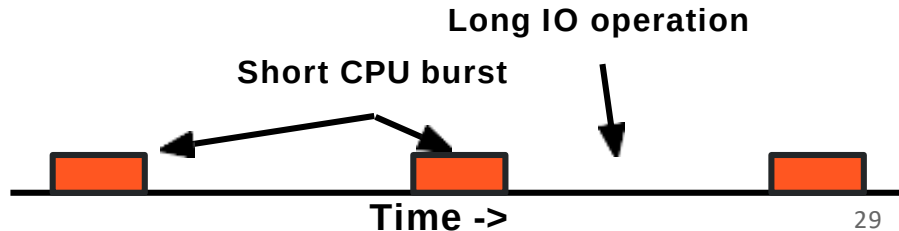
Processes migrate among various queues

CPU bound vs IO bound process

- A **CPU-bound** process requires more CPU time
- Spends more time in the running state.
- CPU bound means the program is bottlenecked by the CPU
- Spends more time doing computations
- In a CPU-bound environment, most times, the processor is the only component being used for execution.



- An **IO bound** process requires less CPU time
- An I/O-bound process spends more time in the waiting state.
- I/O bound means the program is bottlenecked by the IO operations
- Spends more time on IO
- I/O bound operations are characterized by many and fewer CPU bursts during execution

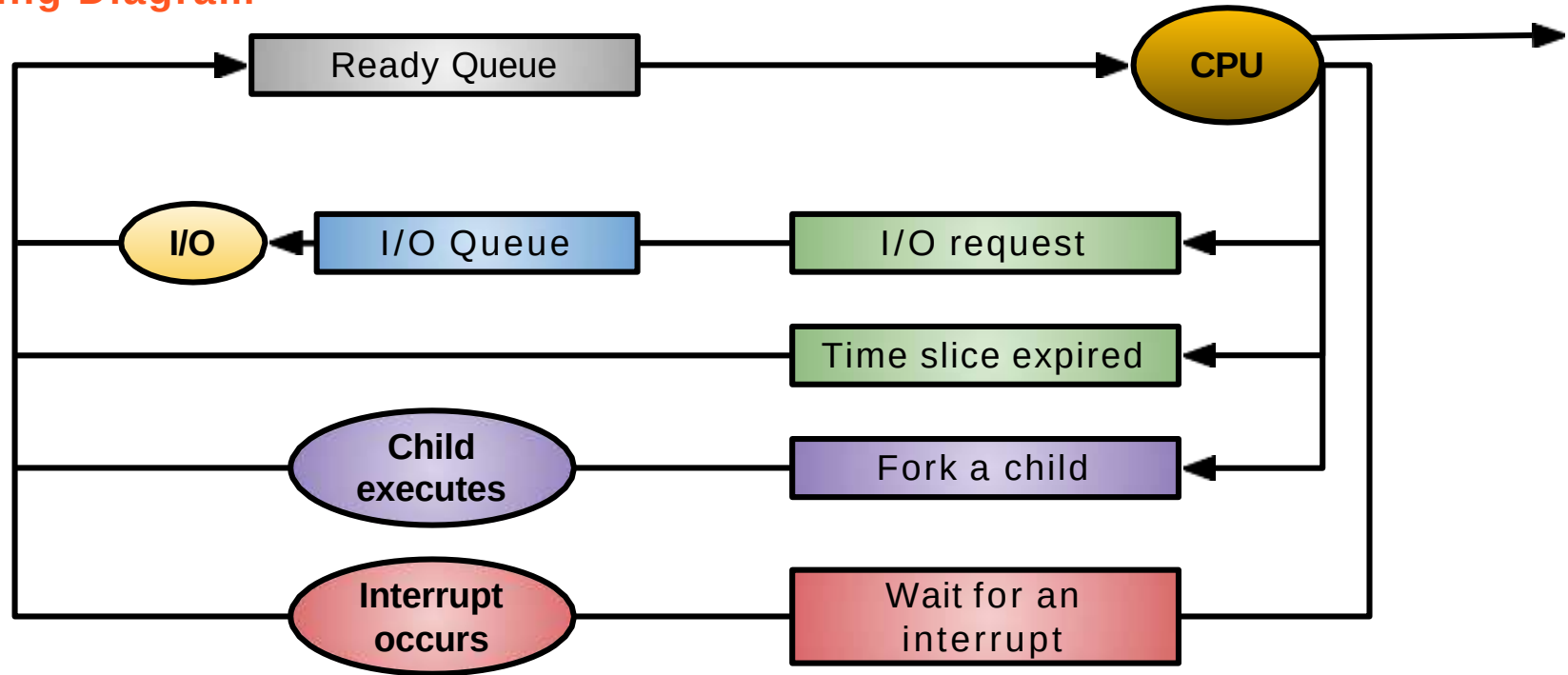


I/O-bound process is one that spends more of its time doing I/O than it spends doing computations.

A **CPU-bound process**, in contrast, generates I/O requests infrequently, using more of its time doing computations.

Process Scheduling Queues

Queuing Diagram



Once the process is allocated a CPU core and is executing, one of several events could occur:

- The process could issue an I/O request and then be placed in an I/O wait queue.
- The process could create a new child process and then be placed in a wait queue while it awaits the child's termination.
- The process could be removed forcibly from the core, as a result of an interrupt or having its time slice expire, and be put back in the ready queue.

Process - Context switching

- When CPU switches to another process, the system must save the state of the old process and load the saved state of new process via a **context switch**
- Context of the process is represented by PCB
- Context switch time is pure overhead, the system does no useful work while switching
- Time depends on hardware support - some hardware provides multiple set of registers so that multiple process contexts loaded/stored

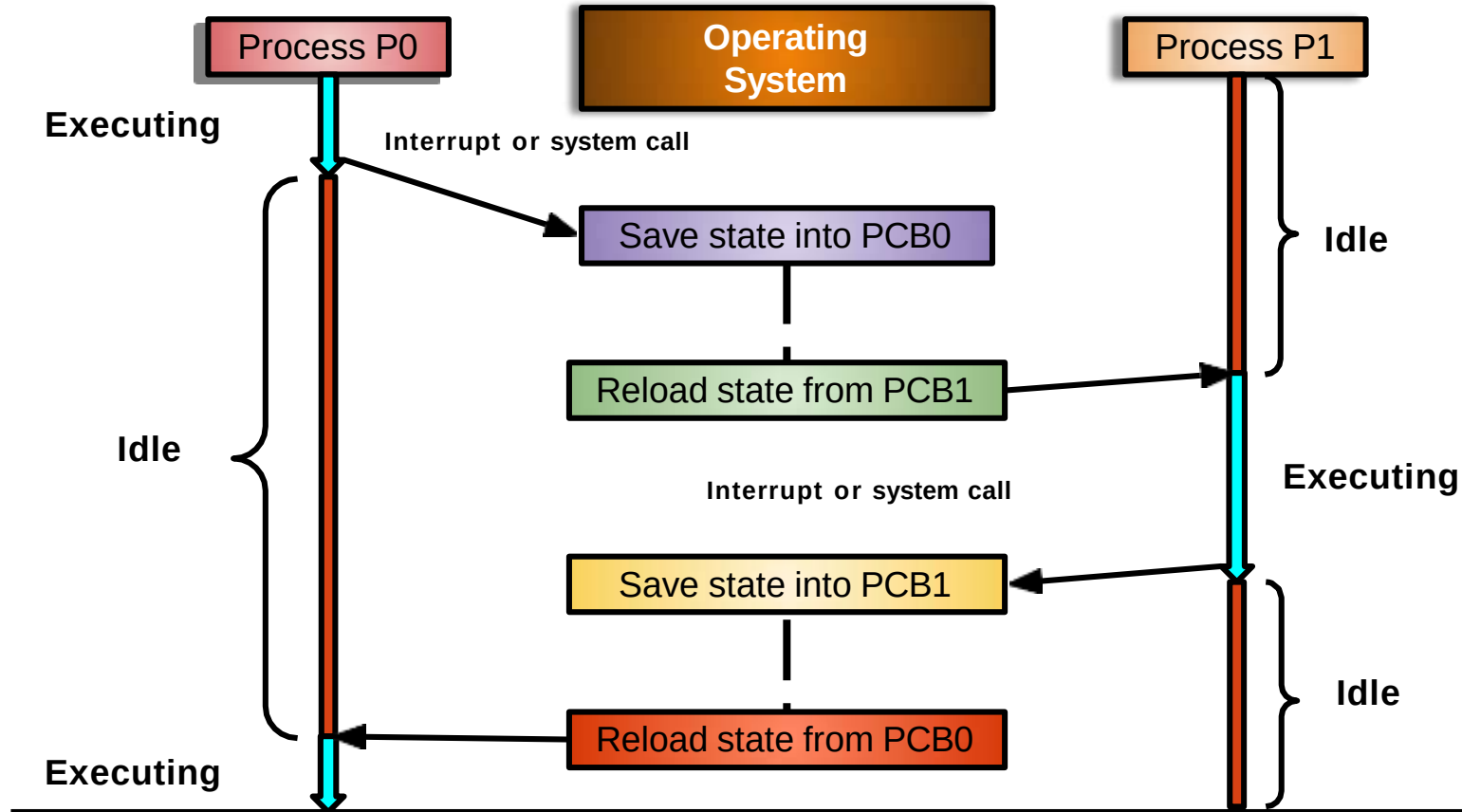
Why is Context Switch Time Called "Pure Overhead"?

Because during a context switch:

- The CPU is not performing any real computation or user task.**
- It is only saving and loading process states (registers, program counters, etc).**
- No progress is made in executing user-level instructions.**

So, the time spent in switching is overhead — it's necessary but not productive from a user or application point of view.

Process - Context switching



Switching the CPU to another process requires performing a state save of the current process and a state restore of a different process.



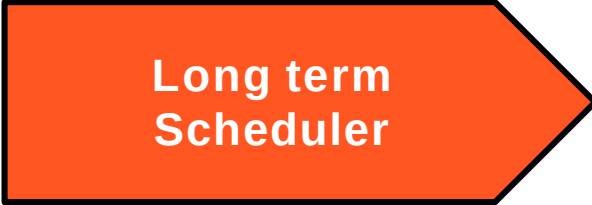
State Save



State Restore

This task is known as a **context switch**.

Schedulers



Long term Scheduler

1. Selects the processes from the storage pool in the secondary memory and loading them into the ready queue in the main memory for execution.
2. The long-term scheduler controls the degree of multiprogramming.
3. It must select a careful mixture of I/O bound and CPU bound processes to yield optimum system throughput.
4. If it selects too many CPU bound processes then the I/O devices are idle and if it selects too many I/O bound processes then the processor has nothing to do.

1. Long-Term Scheduler (Job Scheduler)

Function:

Selects processes from the job pool and loads them into memory for execution.

Main Tasks:

- Controls the degree of multiprogramming (number of processes in memory).
- Decides which jobs to admit into the system.

When it acts:

Runs less frequently than other schedulers.

Example:

In a batch system, when multiple jobs are submitted, the long-term scheduler

Schedulers

Short term Scheduler

1. Selects which process to be executed next and allocates CPU
2. Sometimes only scheduler
3. Short term scheduler is invoked frequently (in milliseconds), must be invoked very fast
4. The choices of the short term scheduler are very important. If it selects a process with a long burst time, then all the processes after that will have to wait for a long time in the ready queue.
5. This is known as starvation and it may happen if a wrong decision is made by the short-term scheduler.

2. Short-Term Scheduler (CPU Scheduler)

Function:

Selects from the processes that are ready to execute, and allocates the CPU to one of them.

Main Tasks:

- Handles process switching on the CPU.
- Runs frequently, sometimes every few milliseconds.

When it acts:

Occurs during context switching (i.e., when a process is waiting or terminated).

Example:

Choosing between a running web browser, a text editor, and a file download

Schedulers

Medium term Scheduler

1. Medium-term scheduling involves swapping out a process from main memory.
2. The process can be swapped in later from the point it stopped executing.
3. This can also be called as suspending and resuming the process and is done by the medium-term scheduler.

3. Medium-Term Scheduler (Swapper)

Function:

Temporarily removes processes from memory (i.e., swapping them out) and may bring them back later.

Main Tasks:

- Improves system performance by reducing the load on memory.
- Manages the transition between memory and disk.

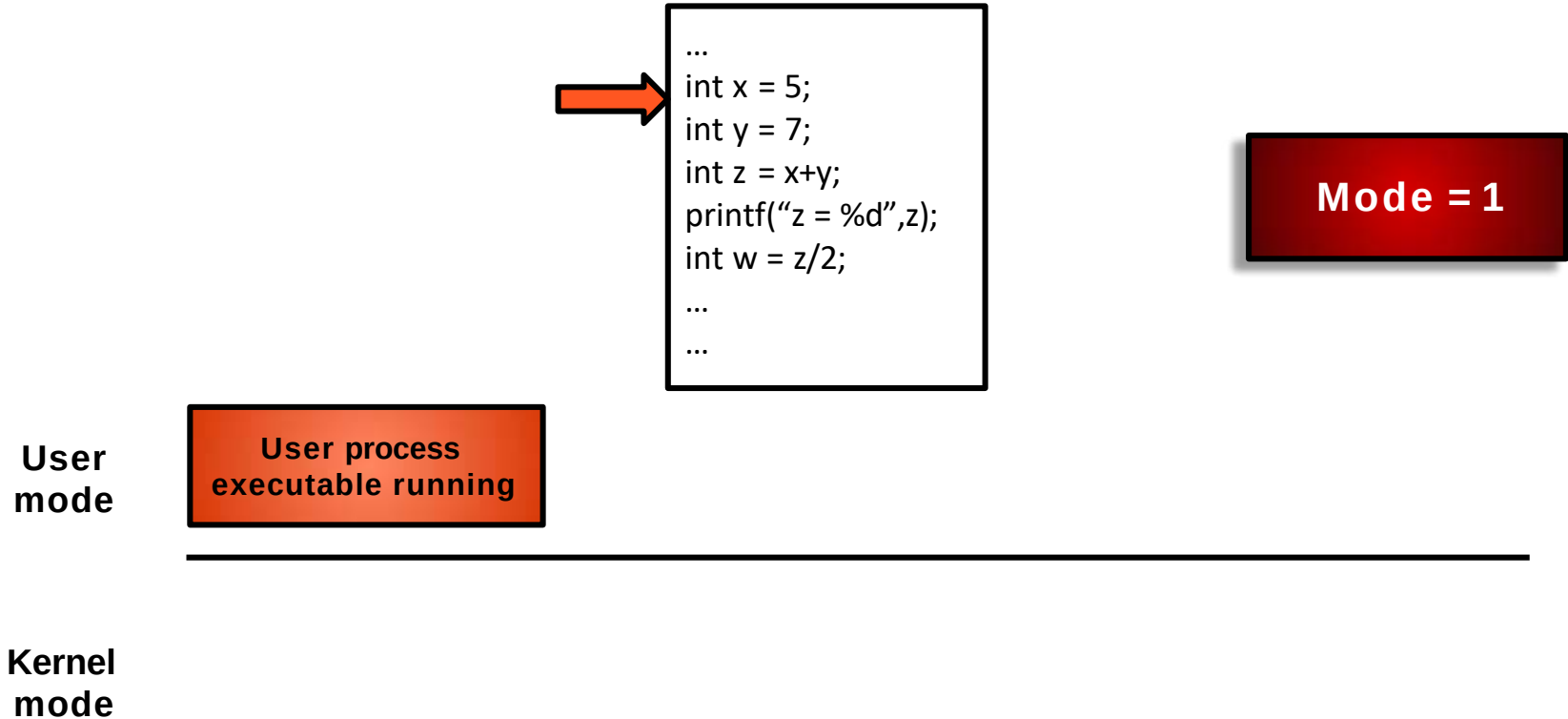
When it acts:

Used when memory is overused and swapping is needed.

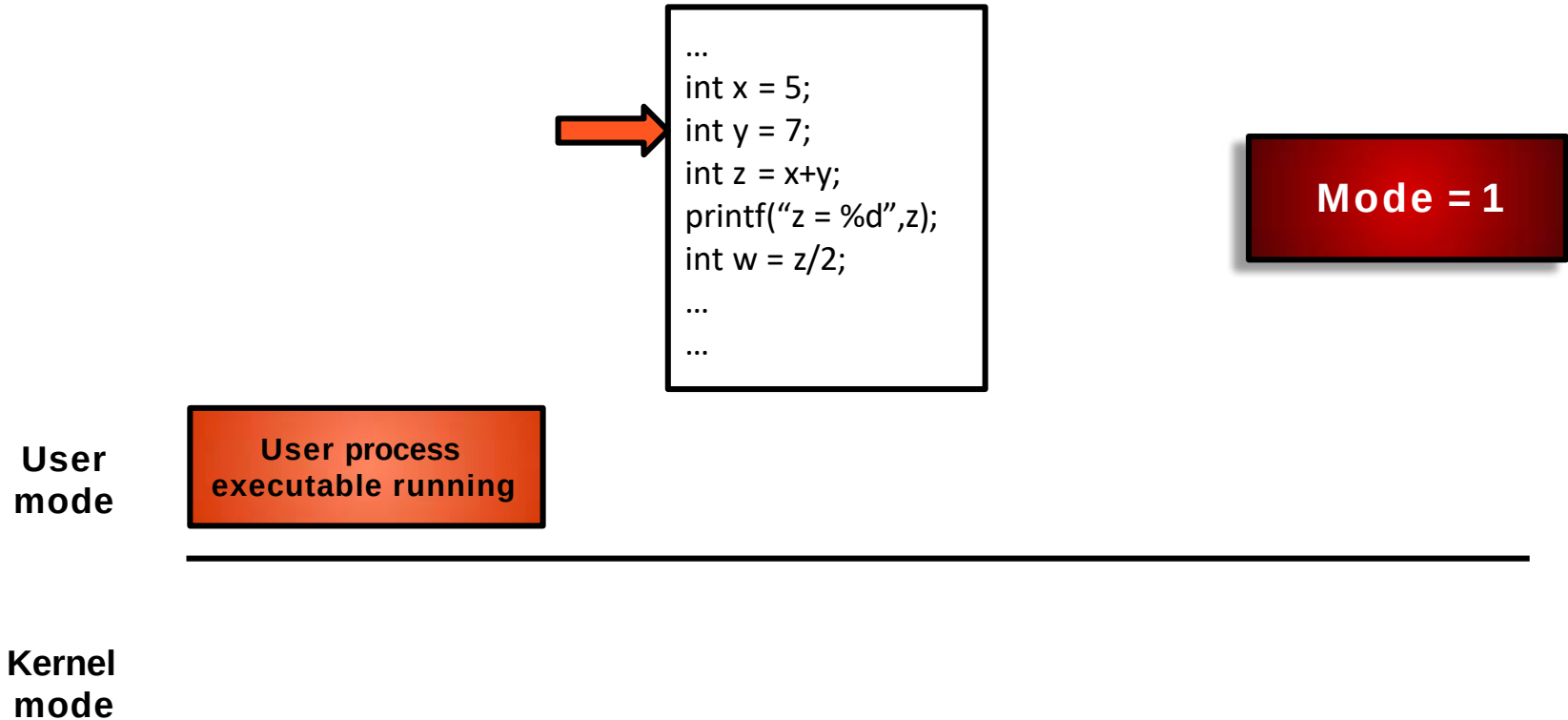
Example:

If the RAM is full, the OS may swap a paused process to disk to make space.

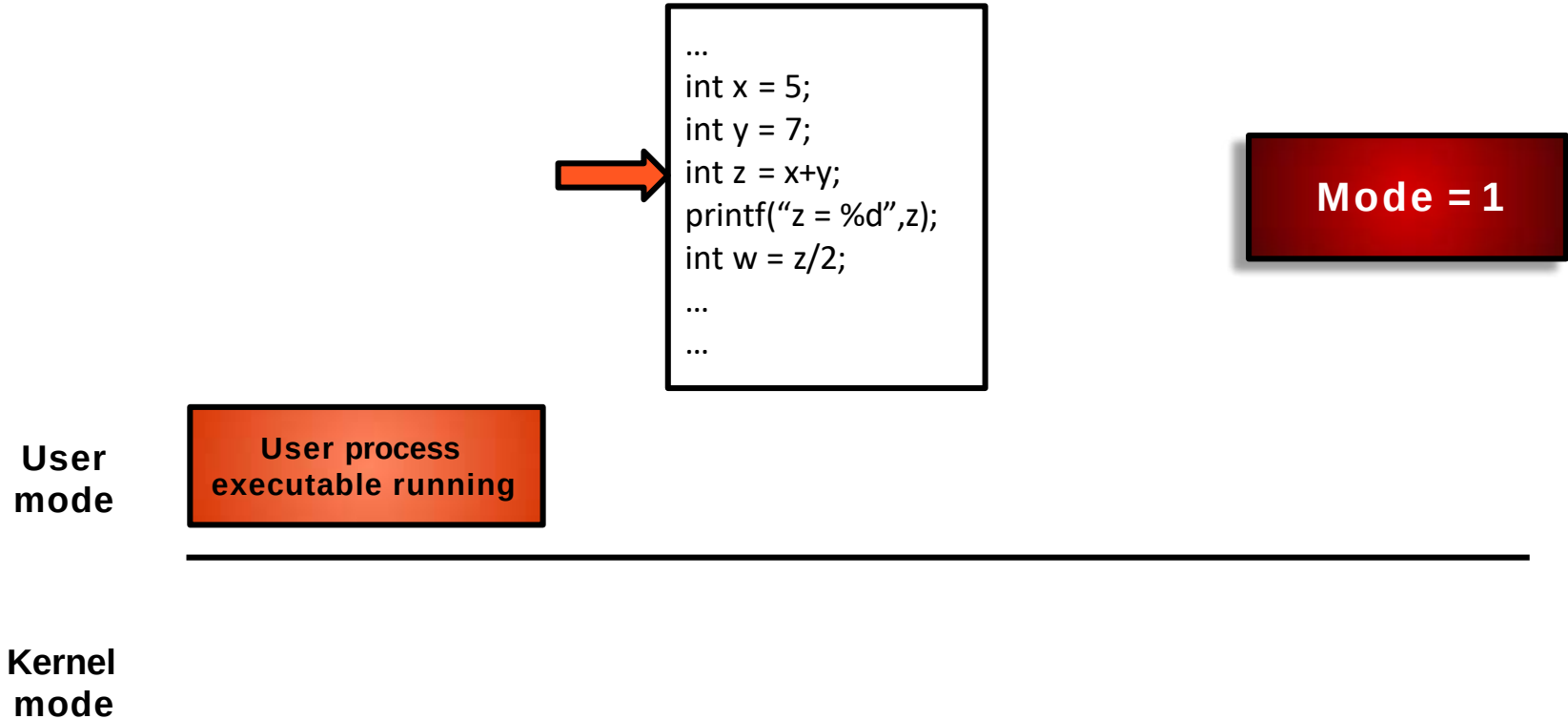
User mode and kernel mode of execution



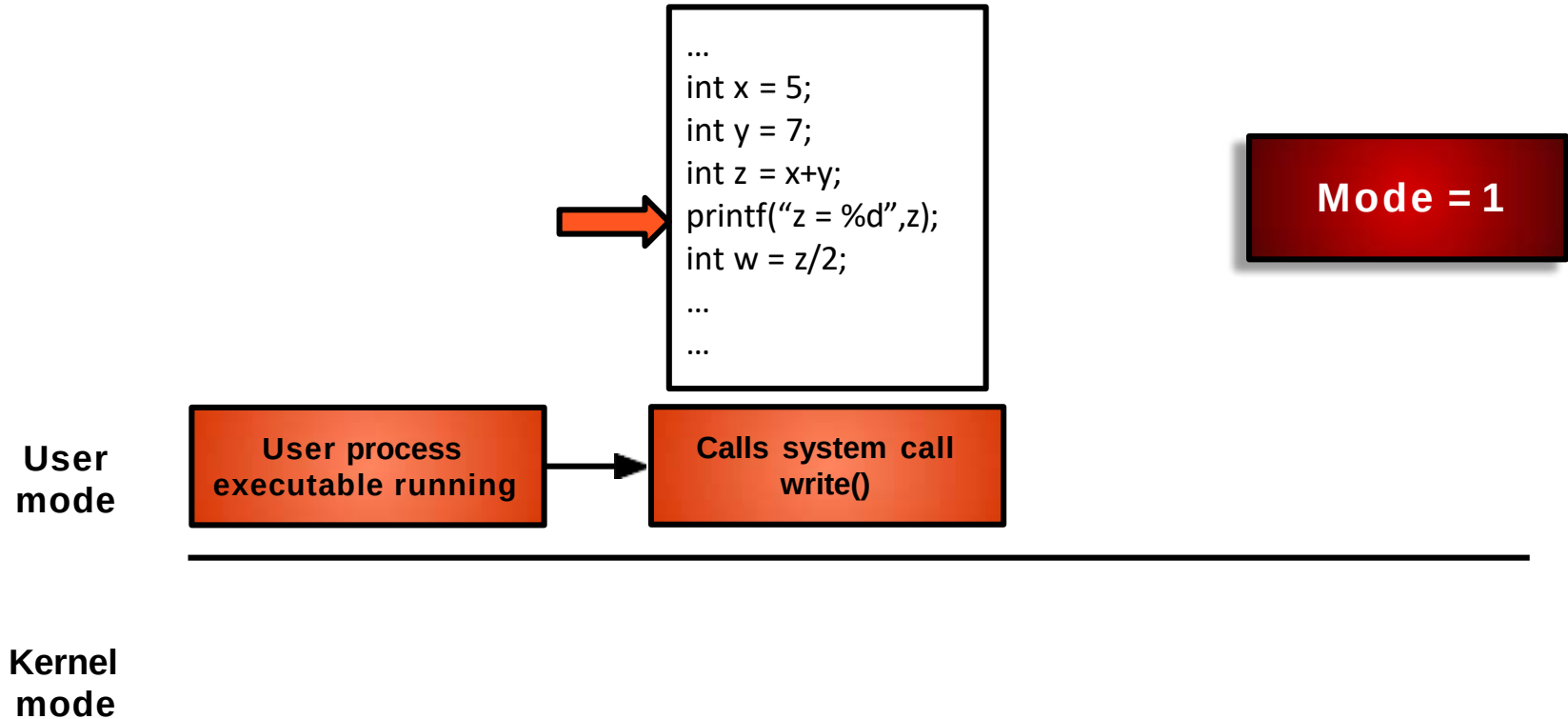
User mode and kernel mode of execution



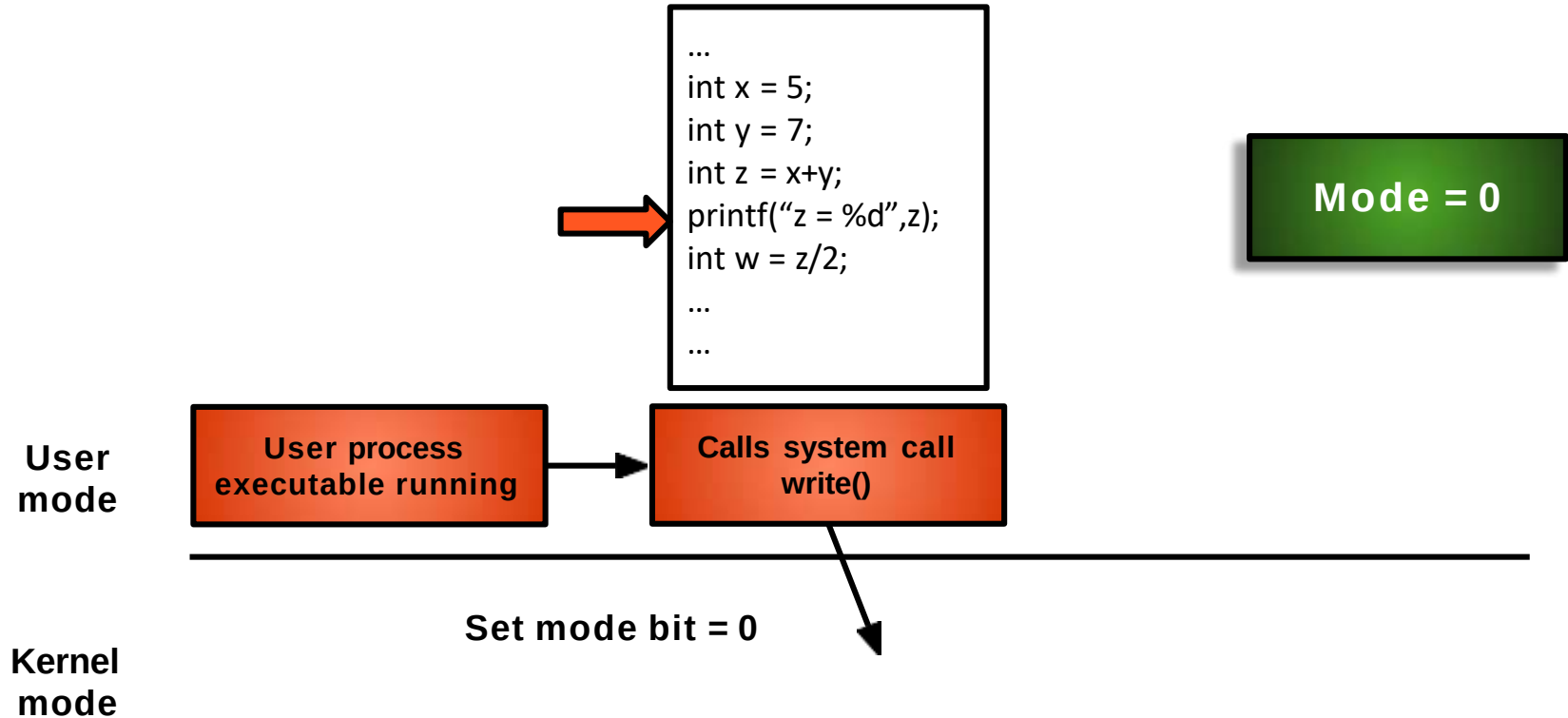
User mode and kernel mode of execution



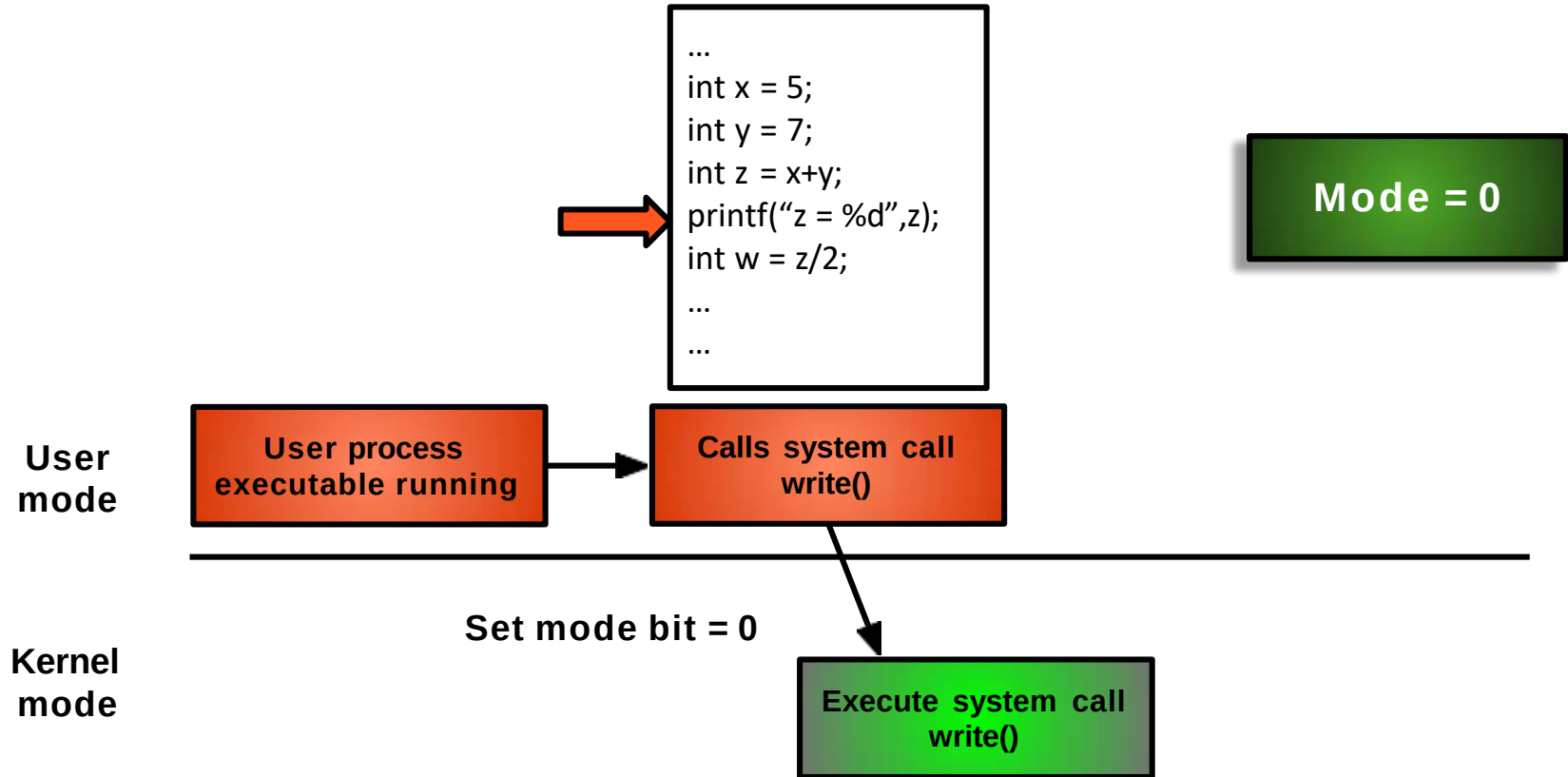
User mode and kernel mode of execution



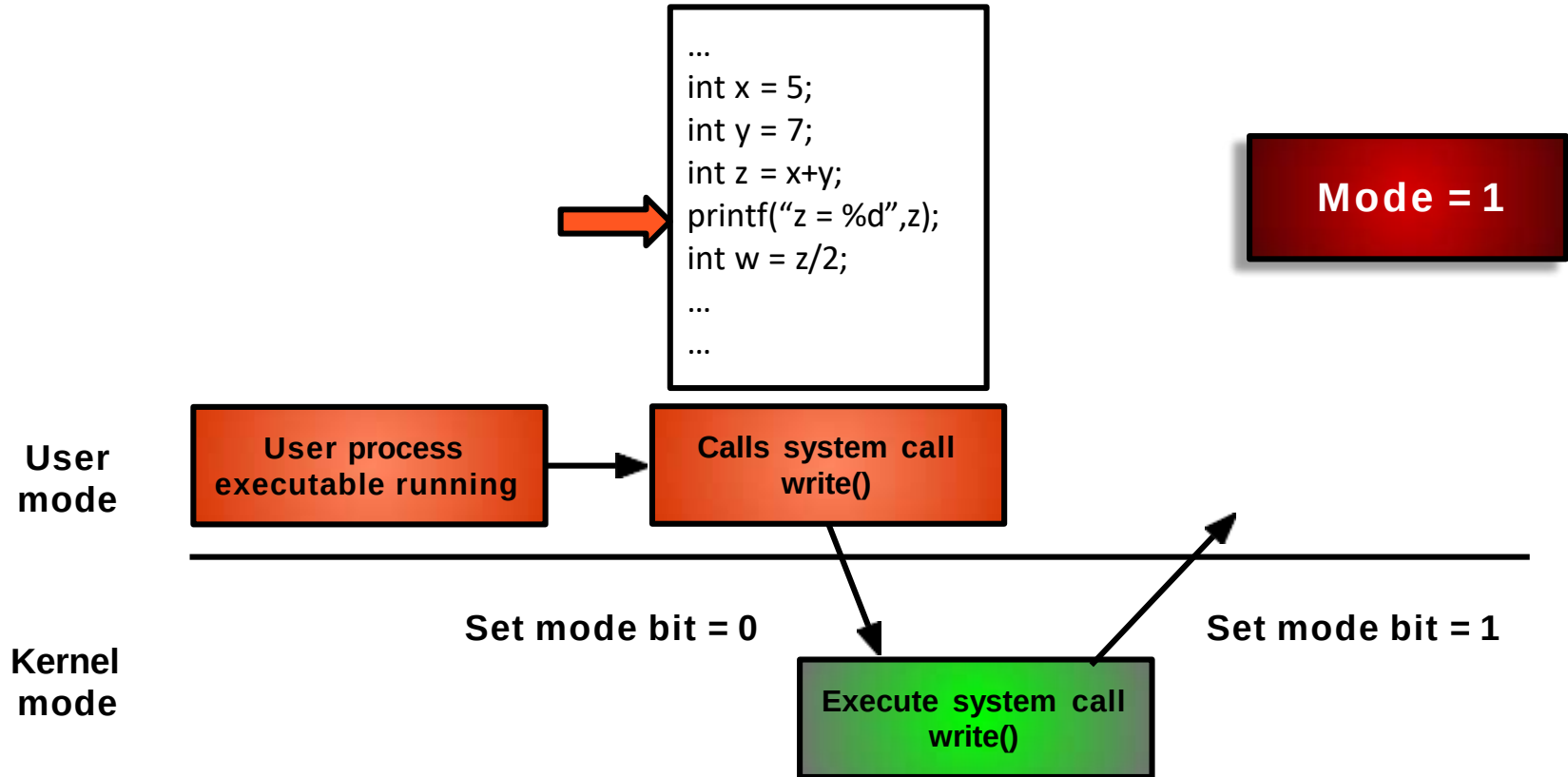
User mode and kernel mode of execution



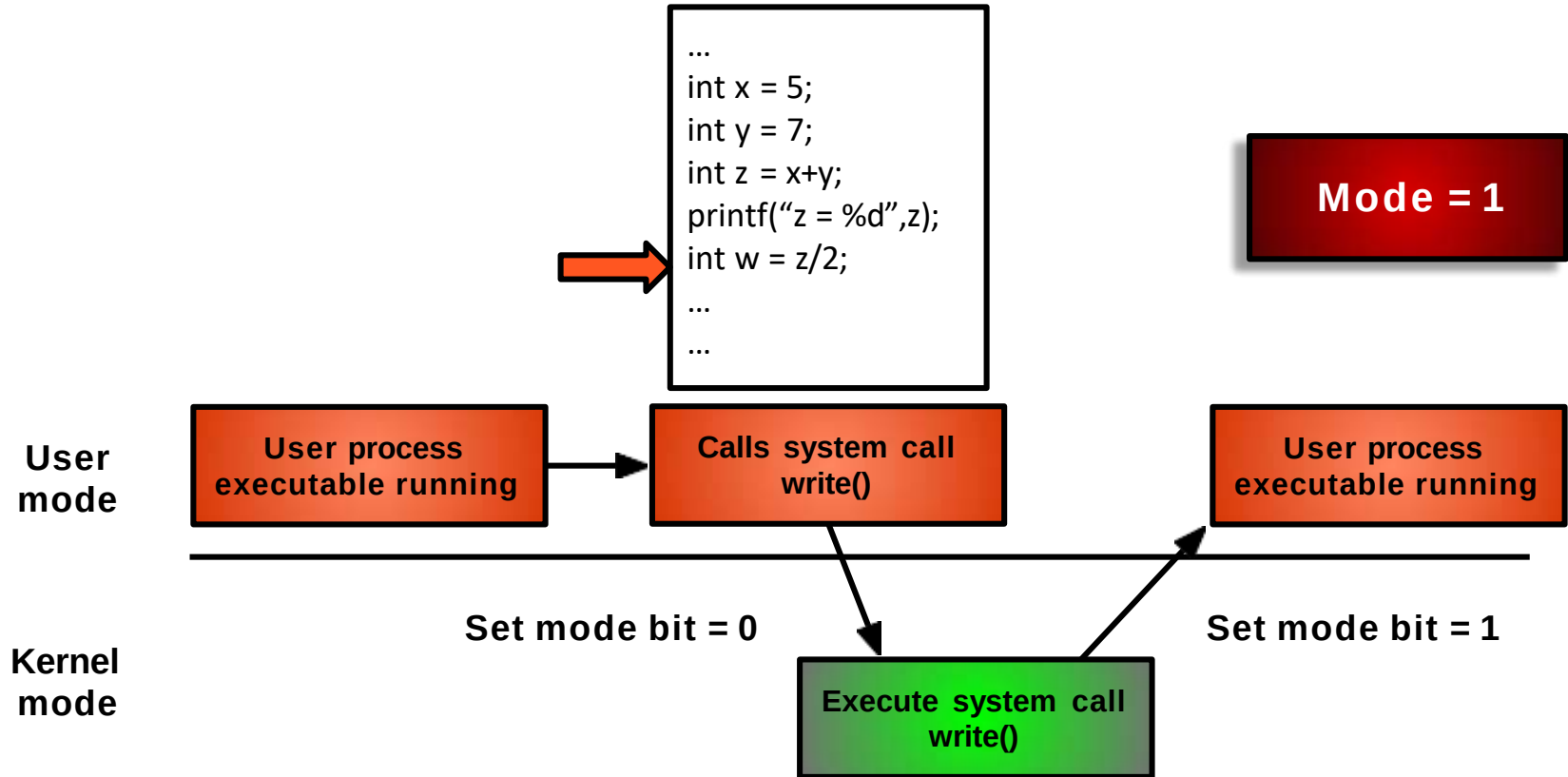
User mode and kernel mode of execution



User mode and kernel mode of execution



User mode and kernel mode of execution



System Call

- A **system call** is the programmatic way in which a computer program requests a service from the kernel of the operating system it is executed on.
- A system call is a way for programs to **interact with the operating system**.
- A computer program makes a system call when it makes a request to the operating system's kernel.
- System call **provides** the services of the operating system to the user programs via **Application Program Interface(API)**.

System Call

- For example, the C programming language gives you `printf()` that lets you write data in many different formats.
- So, `printf()` is a function that convert data into a formatted sequence of bytes.
- Then it calls `write()` to write those bytes onto the output device.
- Application programs don't have direct access to system calls, it requests OS provide access to the call



Program

`printf()` in C
`Cout` in C++
`system.out.println()` in Java
`print()` in Python



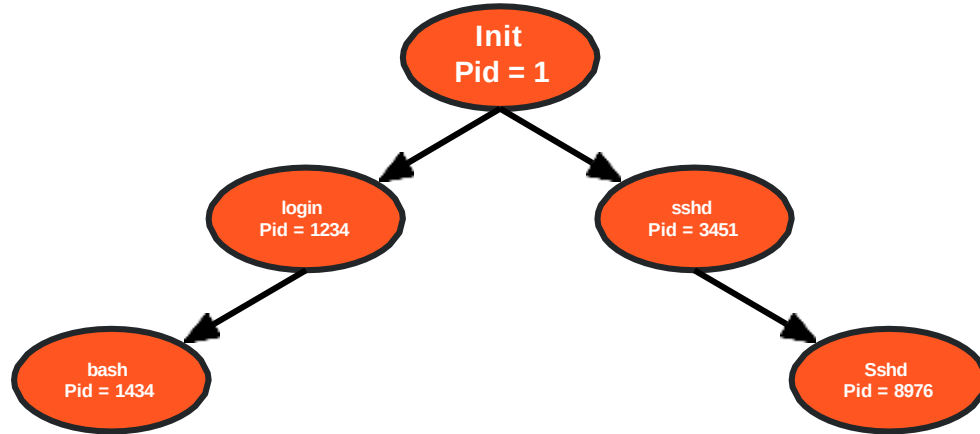
`write()`



OS

Process Creation

- A process may create other processes
- Parent process creates children processes, which in turn create other processes, forming a tree of processes
- Generally a process is identified and managed via process identifier (PID)



A tree of processes in Unix

Process Creation

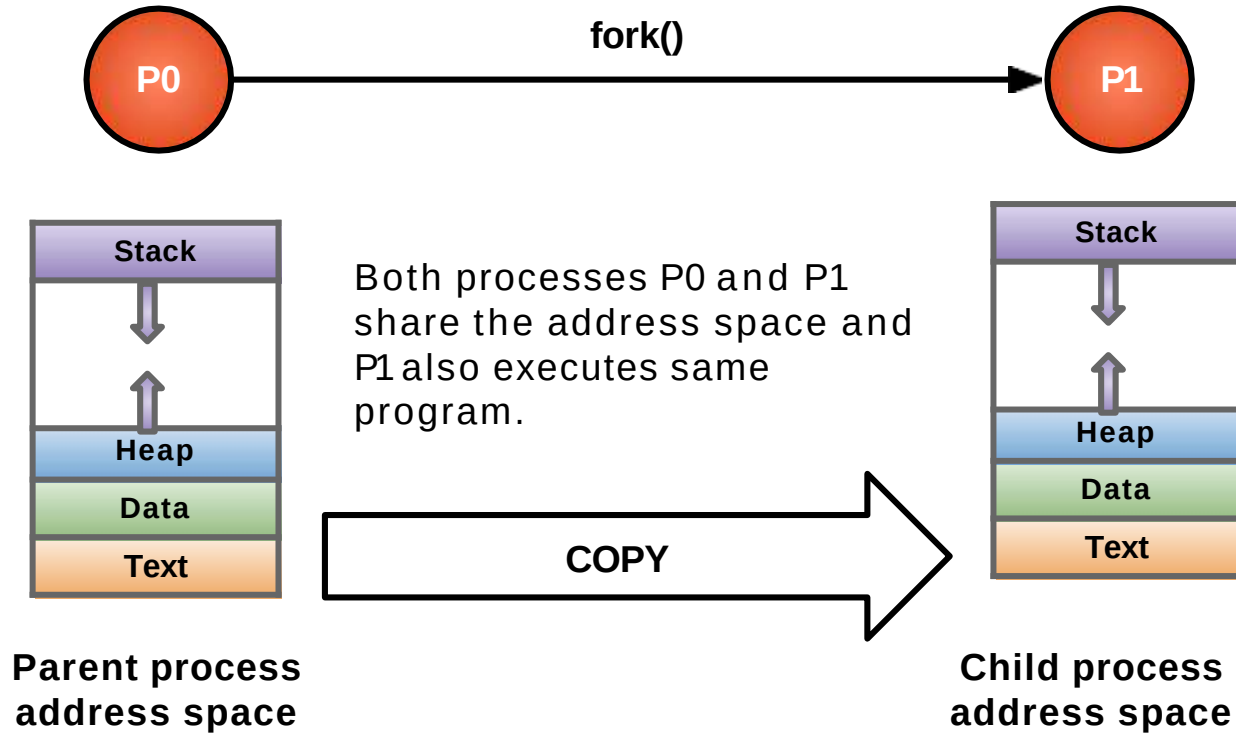
- Fork system call is used for creating a new process, which is called *child process*, which runs concurrently with the process that makes the fork() call (parent process).
- After a new child process is created, both processes will execute the next instruction following the fork() system call.
- A child process uses the same pc(program counter), same CPU registers, same open files which use in the parent process.
- It takes no parameters and returns an integer value. Below are different values returned by fork().

Negative Value: creation of a child process was unsuccessful.

Zero: Returned to the newly created child process.

Positive value: Returned to parent or caller. The value contains process ID of newly created child process.

Process Creation - fork()



Process Creation - fork() program

```
1. #include <stdio.h>
2. #include <sys/types.h>
3. #include <unistd.h>
4. int main()
5. {
6.     // make two process which run same
7.     // program after this instruction
8.     fork();
9.     printf("Hello world!\n");
10.    return 0;
11. }
```

Output: -
Hello world!
Hello world!

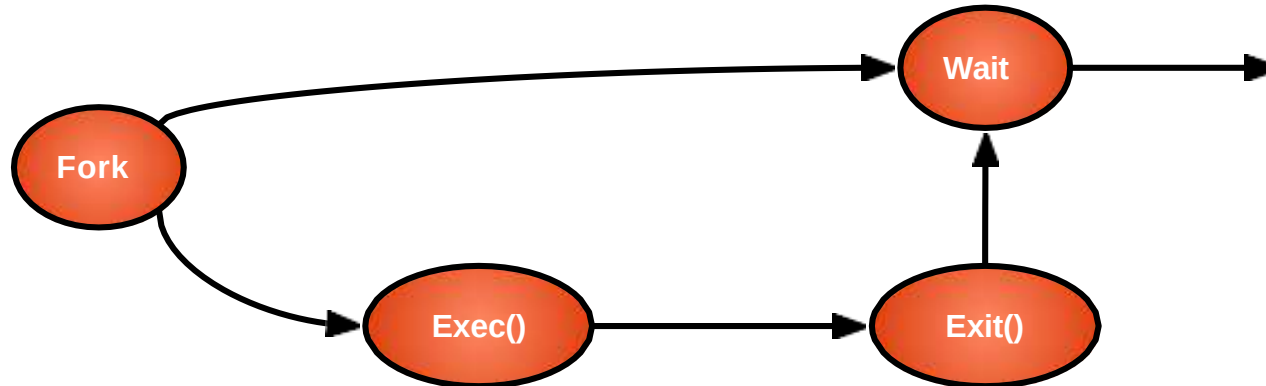
```
1. #include <stdio.h>
2. #include <sys/types.h>
3. int main()
4. {
5.     fork();
6.     fork();
7.     fork();
8.     printf("hello\n");
9.     return 0;
10. }
```



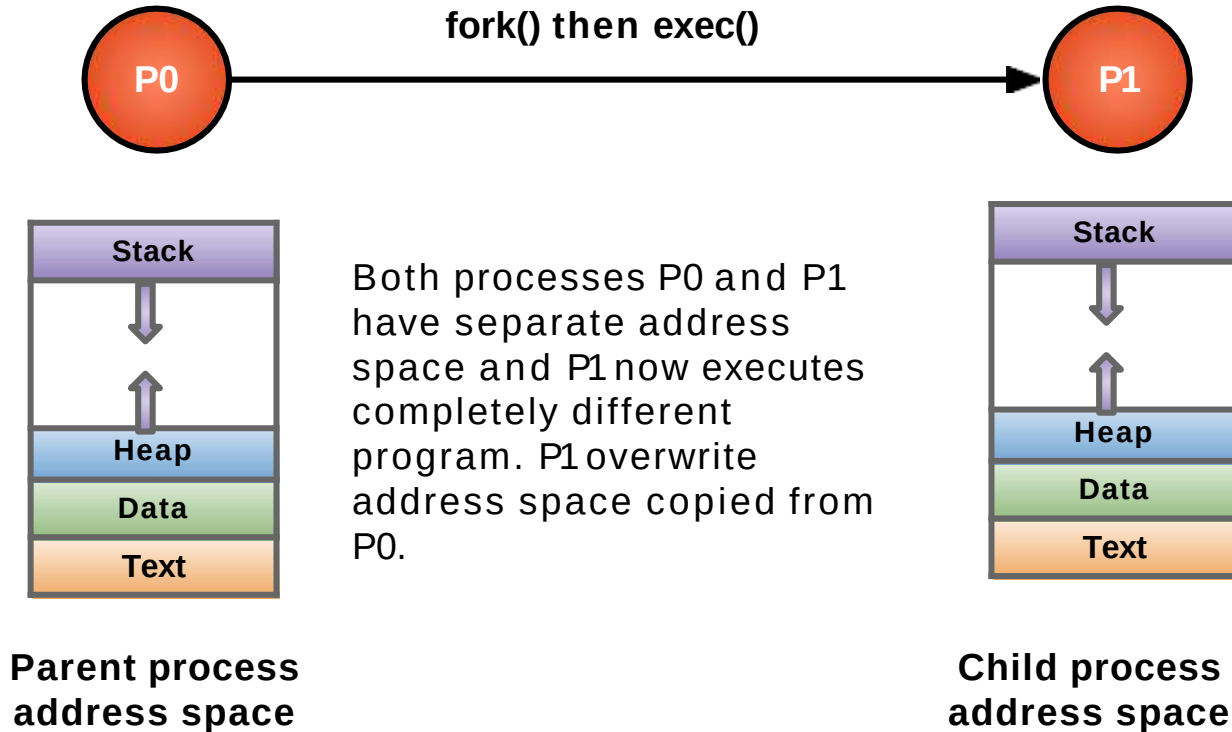
Output?

Process Creation - fork() and exec()

- **fork()** system call is used to create a new process
- **exec()** system call is used after a **fork()** replaces the process's memory space with a new program
- Using **fork()** a child is duplicate of the parents address space
- Using **fork()** A child loads program into address space



Process Creation - fork() and exec()



Process Creation - Exec() program

```
1.  //EXEC.c
2.  #include<stdio.h>
3.  #include<unistd.h>
4.  int main()
5.  {
6.      int i;
7.      printf("I am EXEC.c called by
8.      execvp() ");
9.      printf("\n");
10.     return 0;
10. }
```

COMPILE AND CREATE EXECUTABLE

```
gcc EXEC.c -o EXEC
```

```
1.  #include<stdio.h>
2.  #include<stdlib.h>
3.  #include<unistd.h>
4.  int main()
5.  {
6.      //A null terminated array of character
7.      //pointers
8.      char *args[]={"./EXEC",NULL};
9.      execvp(args[0],args);
10.     printf("Ending-----");
11.     return 0;
12. }
```

Compile and Run this program

```
gcc execDemo.c -o execDemo
```

Process Termination

- A process terminates when it executes its final statement and ask OS to delete it by using **exit() system call**
- All the resources allocated to process gets deallocated by the operating system
- A parent may terminate the execution of child process using abort system call, some reasons to do so are
 1. Child has exceeded allocated resources
 2. Task assigned to child is no longer required
 3. The parent is exiting and OS does not allow a child to continue if the parent terminates.

Process Termination

1. A parent process may wait till child process terminates using **wait() system call**. The call returns status and pid of terminated process.

```
pid = wait(&status);
```

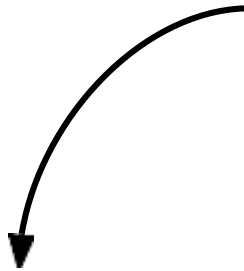
2. A process that has finished the execution but still has an entry in the process table called as **Zombie process**
3. An **orphan process** is a computer process whose parent process has finished or terminated, though it (child process) remains running itself.

Process Termination - wait() example

```
// C program to demonstrate working of wait()
#include<stdio.h>
#include<sys/wait.h>
#include<unistd.h>
int main()
{
    if (fork()== 0)
        printf("HC: hello from child\n");
    else
    {
        printf("HP: hello from parent\n");
        wait(NULL);
        printf("CT: child has terminated\n");
    }

    printf("Bye\n");
    return 0;
}
```

This will never get printed before
HC



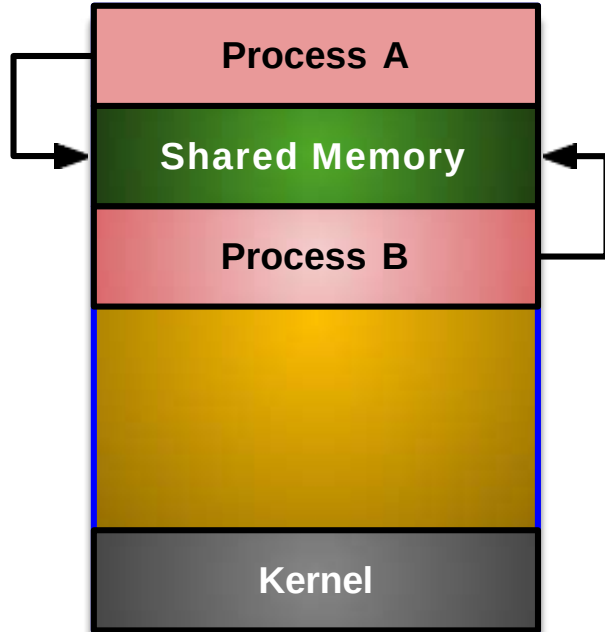
Inter Process Communication (IPC)

1. Processes in the system may be independent or cooperating.
2. Cooperating processes may affect or be affected by other processes including sharing data.
3. Independent processes cannot affect other processes.

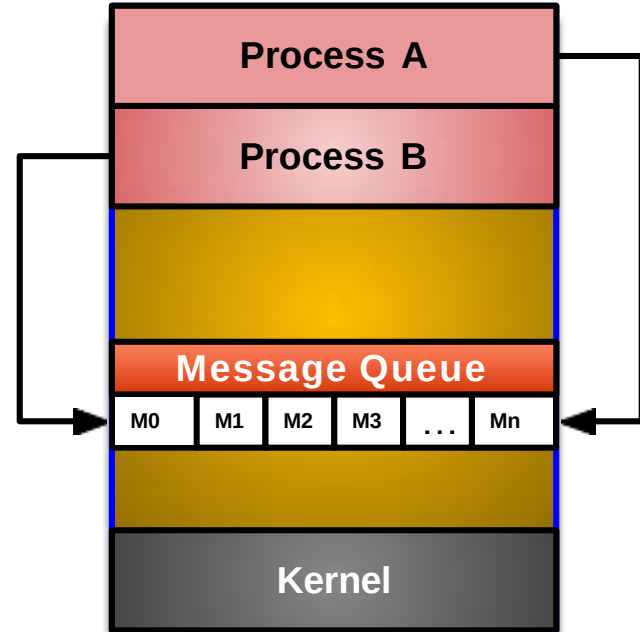
Reasons for having cooperating processes:-

1. Information sharing
2. Computation speedup (multiple processes running in parallel)
3. Modularity
4. Convenience

Two models of IPC

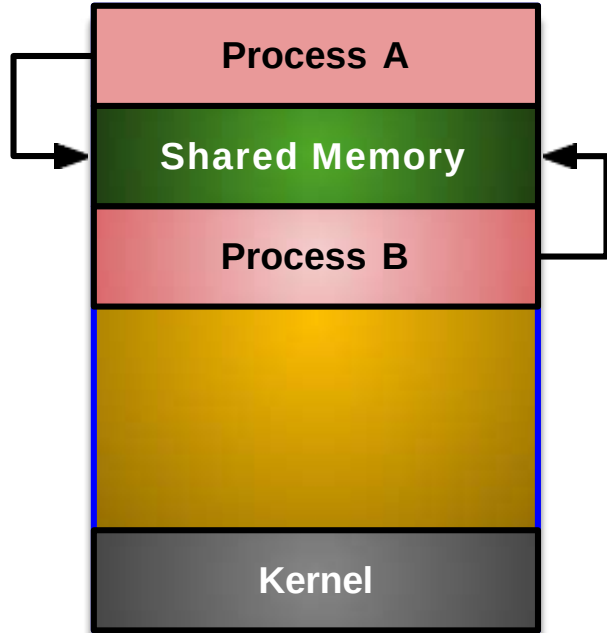


Shared Memory



Message Passing

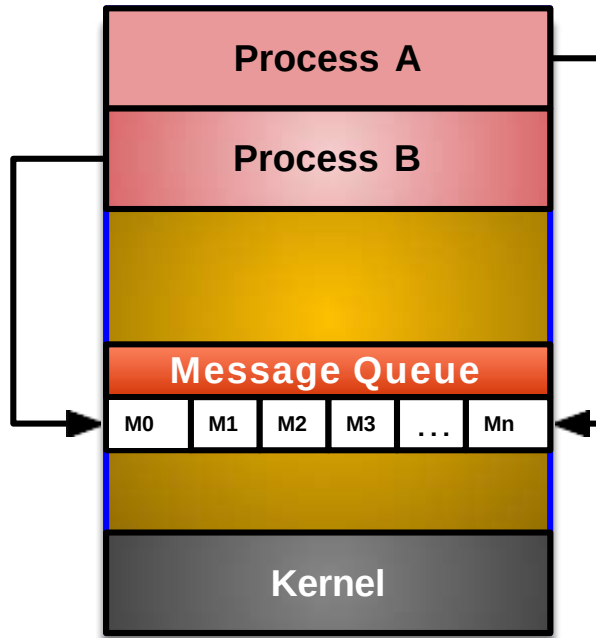
Two models of IPC



Shared Memory

- Processes communicate by **read/write on shared memory**
- Multiple processes writing/reading from same memory location may result in **data inconsistency**
- To ensure data consistency, a proper **synchronization** is required

Two models of IPC



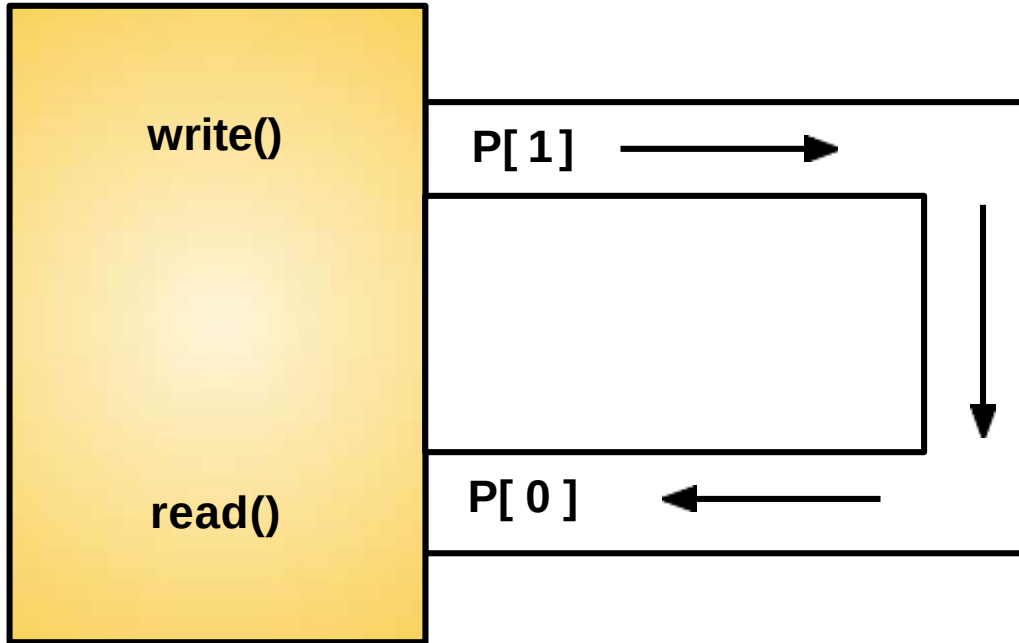
- Processes communicate by **sending or receiving messages**
- IPC facility provide two operations
 1. **send (message)**
 2. **receive (message)**
- Message size either fixed or variable
- If process P and Q wants to communicate
 - Establish **a communication link** between them
 - Exchange messages via send/receive

Pipe

- **A pipe** is a connection between two processes, such that the standard output from one process becomes the standard input of the other process.
- In UNIX Operating System, Pipes are useful for communication between related processes (inter-process communication).
- Pipe is one-way communication only i.e we can use a pipe such that One process write to the pipe, and the other process reads from the pipe.
- If a process tries to read before something is written to the pipe, the process is suspended until something is written.

Pipe

Process



Refer pipe1.c from programs folder

```
int pipe(int fds[2]);
```

Parameters :

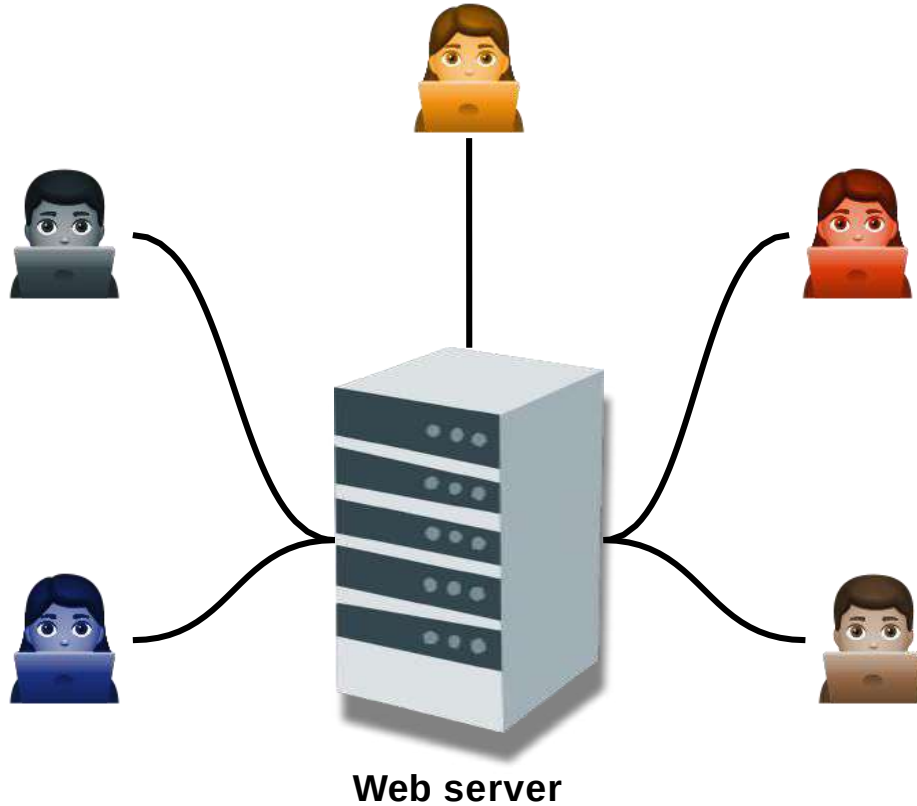
fd[0] will be the fd(file descriptor) for the read end of pipe.

fd[1] will be the fd for the write end of pipe.

Returns : 0 on Success. -1 on error.

Threads

Web server example



- If we handle one user at a time it may lead to starvation to others
- If for each user a separate process is created:-
- Each process contains data, text, stack and heap region
- If multiple processes gets created it becomes very heavy to operate

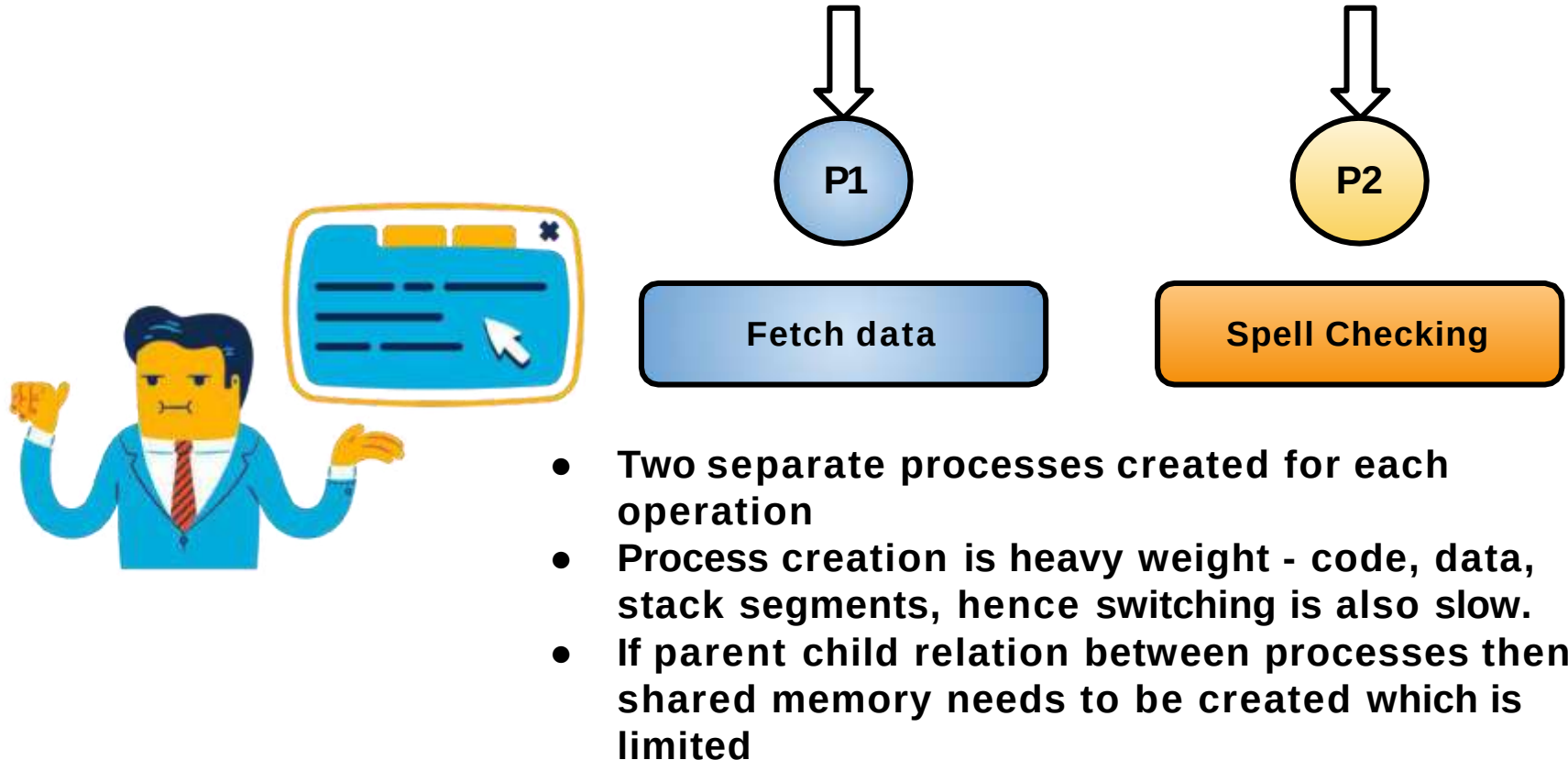
Motivation for threads



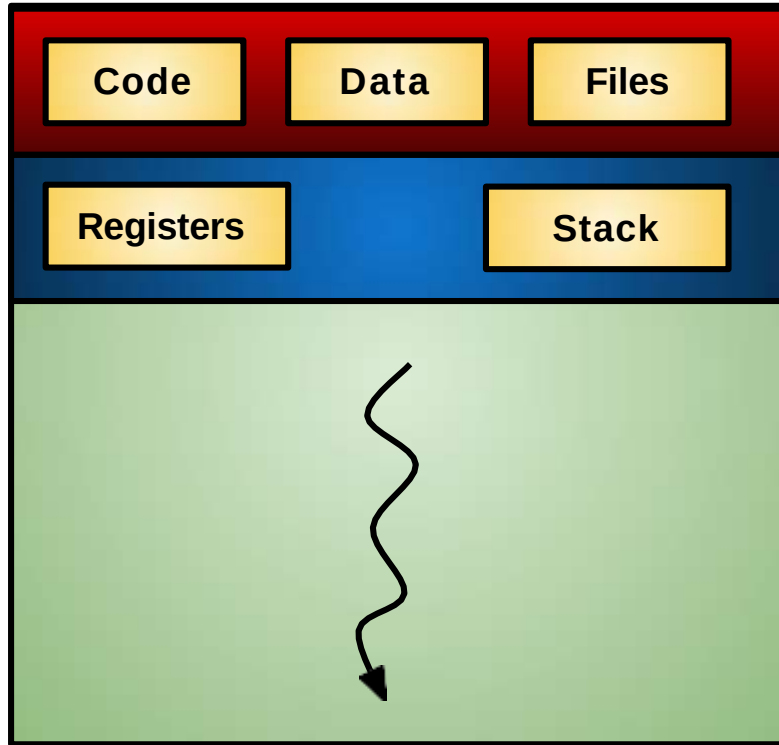
Let's say there are multiple tasks with a computer application

1. Update display
2. Fetch data from web
3. Spell check
4. Answer to a network request

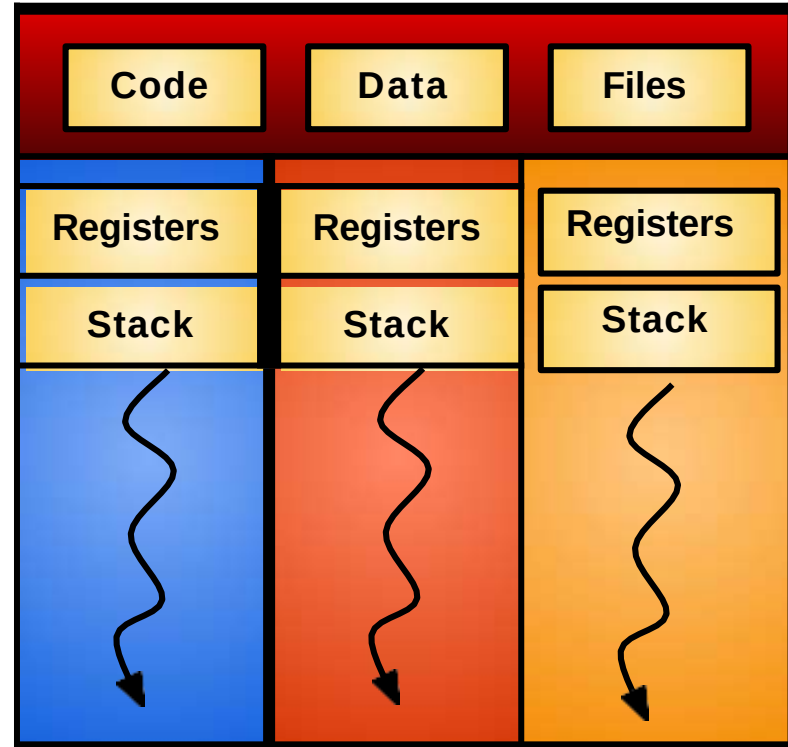
Motivation for threads



Single vs multi threaded process

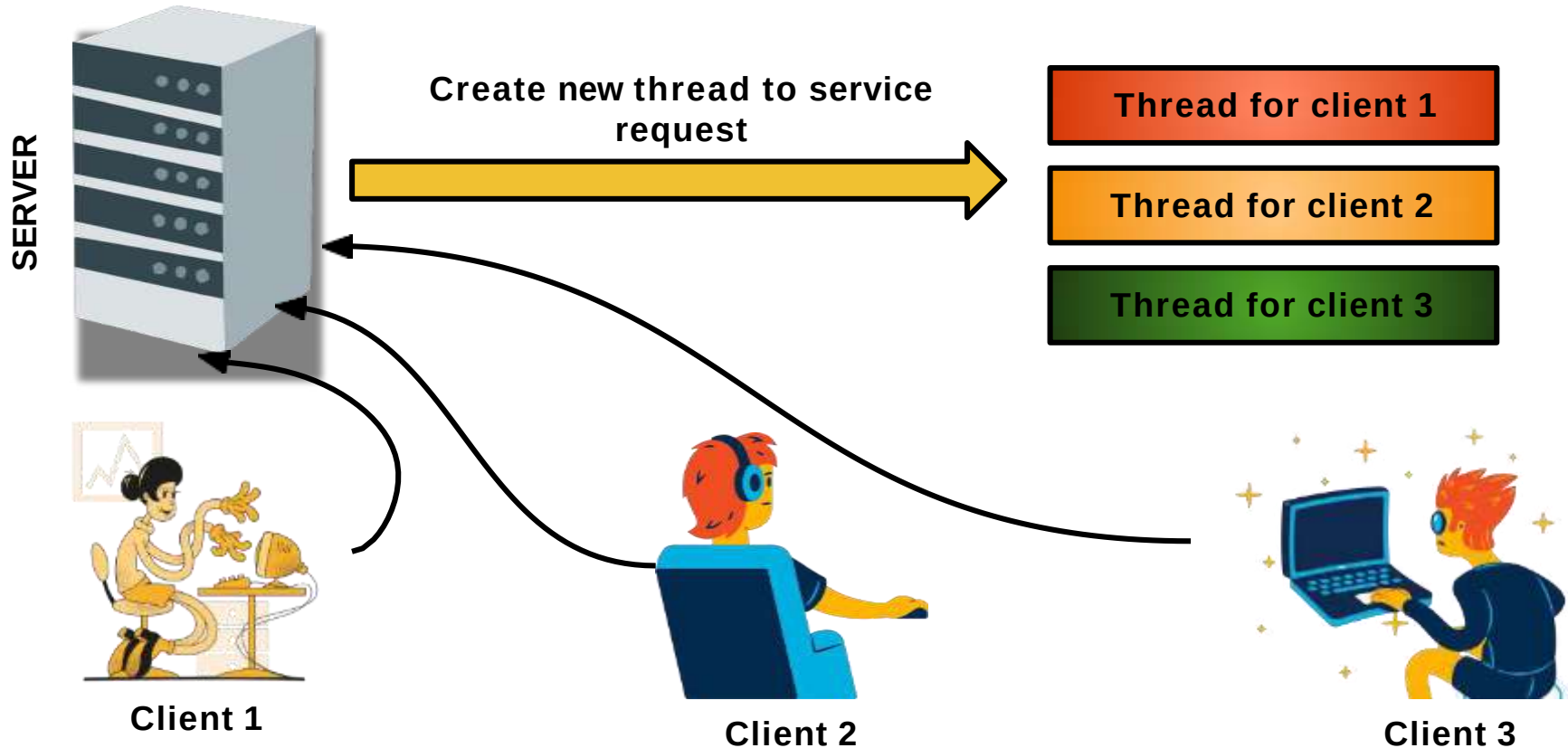


Single threaded process



Multi-threaded process

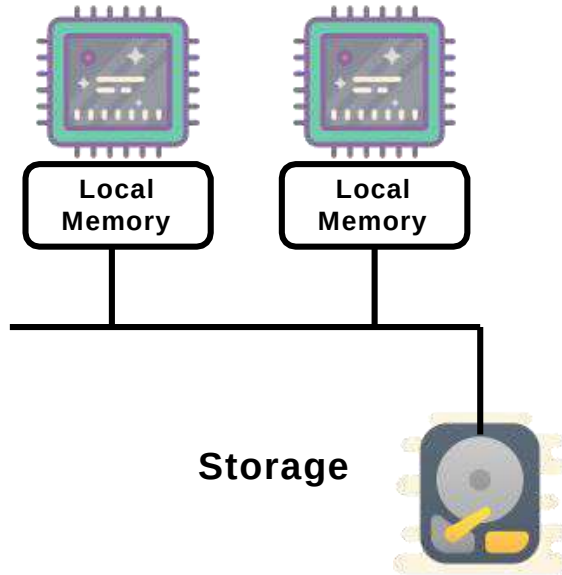
Multi-threaded server architecture



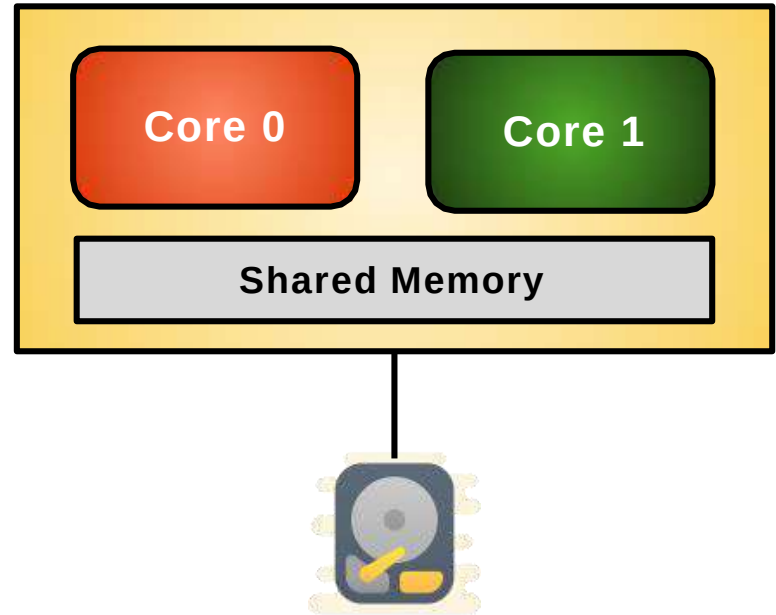
Benefits of multi-threading

- 1** **Responsiveness** - May allow continued execution if part of process is blocked, especially important for user interfaces
- 2** **Resource sharing** - Thread share resources of process easier than shared memory and message passing IPC
- 3** **Economy** - Cheaper than process creation, thread switching overheads are lesser than context switch process overhead
- 4** **Scalability** - process can take advantage of multiprocessor architecture

Multi-core vs Multi-CPU

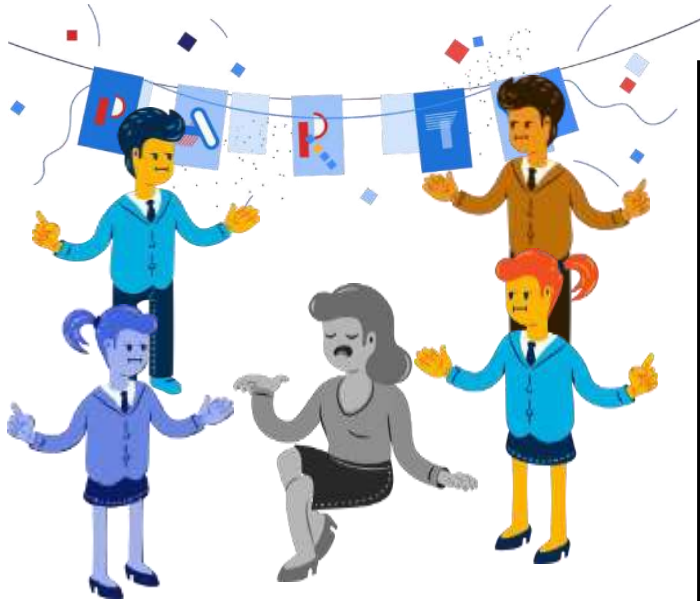


- Multiple CPUs placed in a computer
- Each processor is having its own local memory



- Multiple cores placed on a single processor chip
Each core appears as separate CPU for OS

Serial vs concurrency vs parallelism

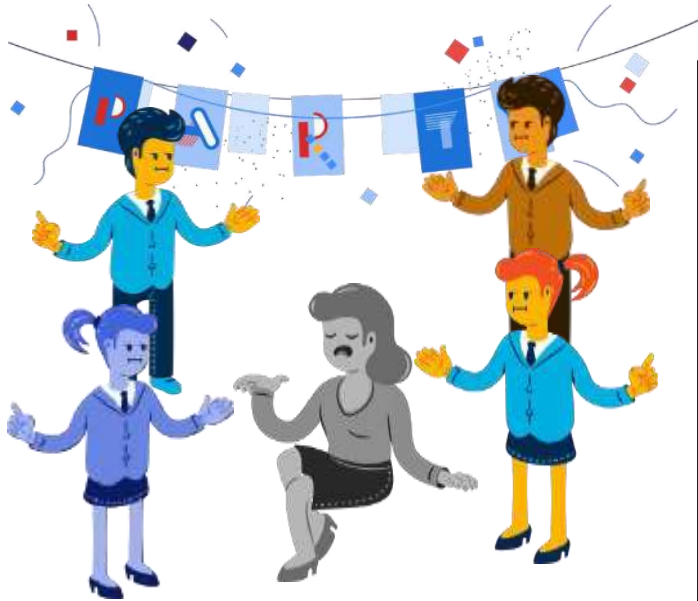


Host

Serial

- Host is talking with one guest at a time
- While host talking with one guest other guests wait till his/her chance
- Host communicates with a guest till it finishes, it doesn't switch to another guest in between

Serial vs concurrency vs parallelism

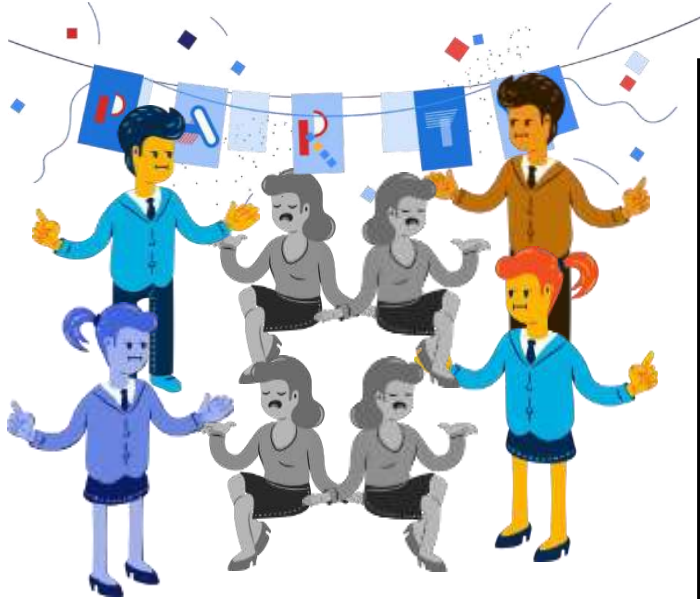


Host

Concurrent

- Host is talking with one guest at a time for one minute
- While host talking with one guest other guests wait till his/her chance
- Host switch to another guest after one minute
- Each guest time to communicate with host for some time

Serial vs concurrency vs parallelism



Host

Parallel

- Multiple host copies (ghost :-) are available
- Each guest is communicating with host without any intervention

Serial vs concurrency vs parallelism



Host

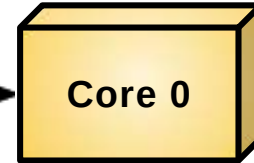
**Concurrency +
Parallelism**

- Multiple host copies (ghost :-) are available
- Each guest is communicating with two host concurrently

Types of parallelism

Data Parallel:- Distributes subset of same data across multiple cores and perform same operation on each

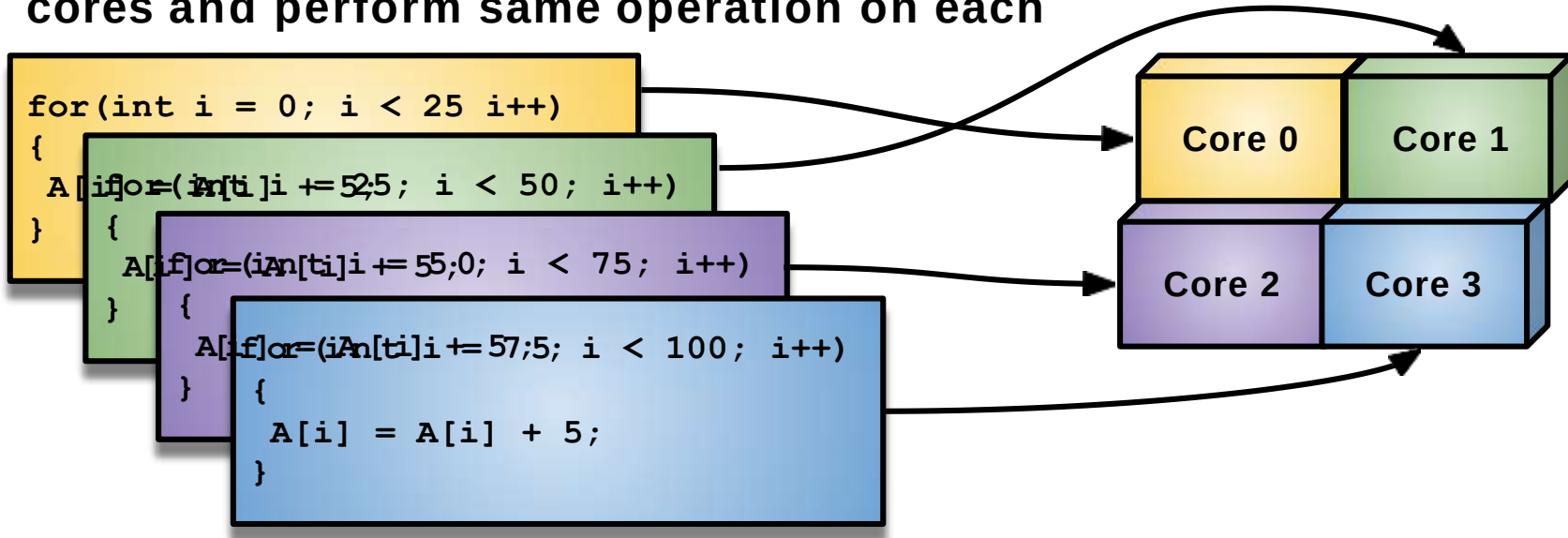
```
for(int i = 0; i < 100; i++)  
{  
    A[i] = A[i] + 5;  
}
```



Sequential:- All computations performed on core 0

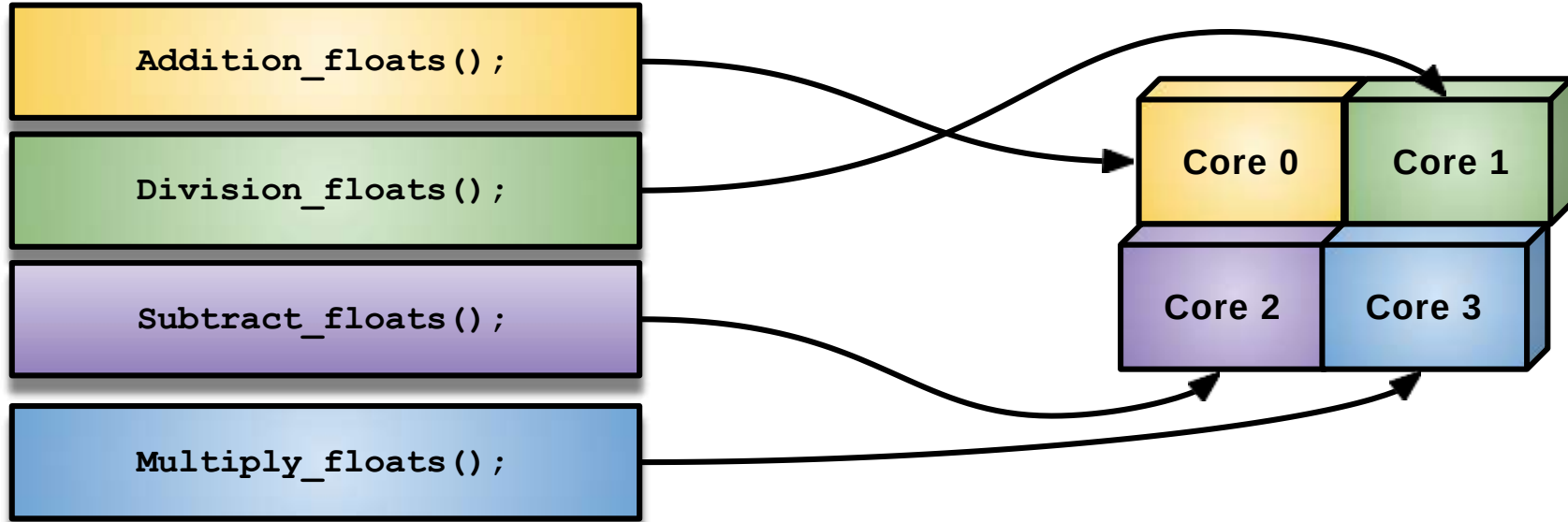
Types of parallelism

Data Parallel:- Distributes subset of same data across multiple cores and perform same operation on each



Types of parallelism

Task Parallel:- Distributes threads across cores, each core performing unique operation.



Multicore and multiprocessor programming Challenges on programmer

- **Dividing Activities:-** Divide the tasks among threads/cores
- **Balancing:-** Each thread/core should get equal work
- **Data splitting:-** Each core gets only required data
- **Data dependency:-** Output of one thread's execution is input to other thread. Serializes the application.
- **Testing and debugging:-** Compare to sequential programming testing and debugging is difficult

User threads and kernel threads

Support for threads also provided at two levels

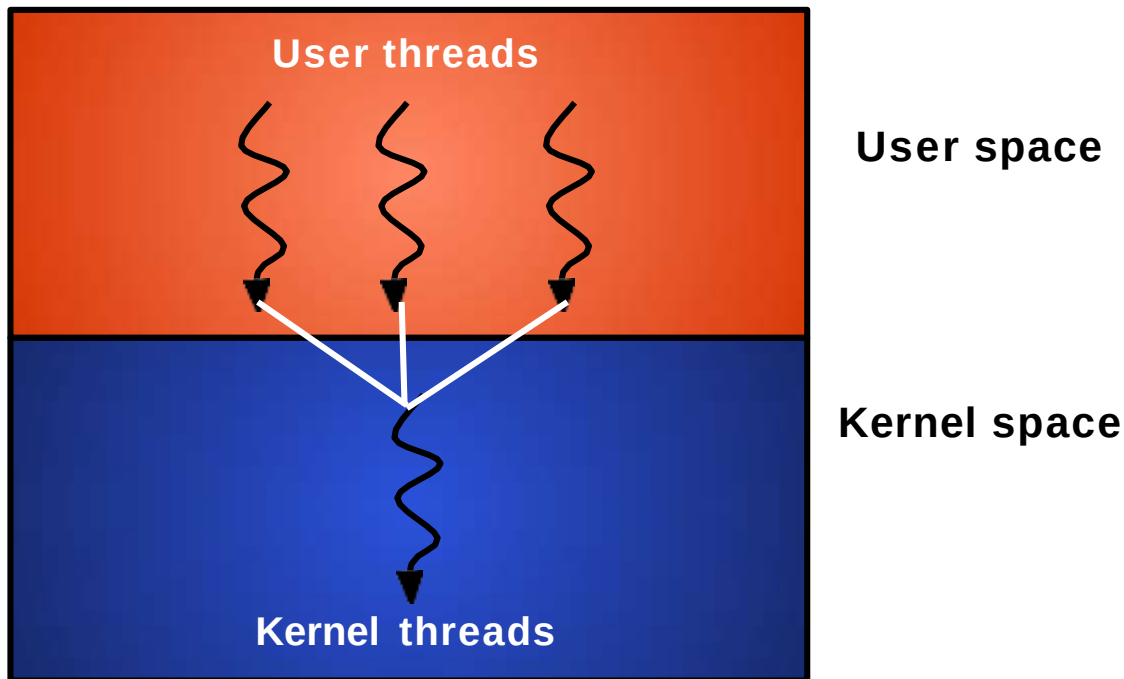
- **User threads** - Supported above the kernel and are managed without support of the kernel.
Generally handled by a programmer or by a threads library
- **Kernel threads** - Supported and managed directly by OS

Relationship between user and kernel threads

- **Many-to-one**
- **One-to-one**
- **Many-to-many**

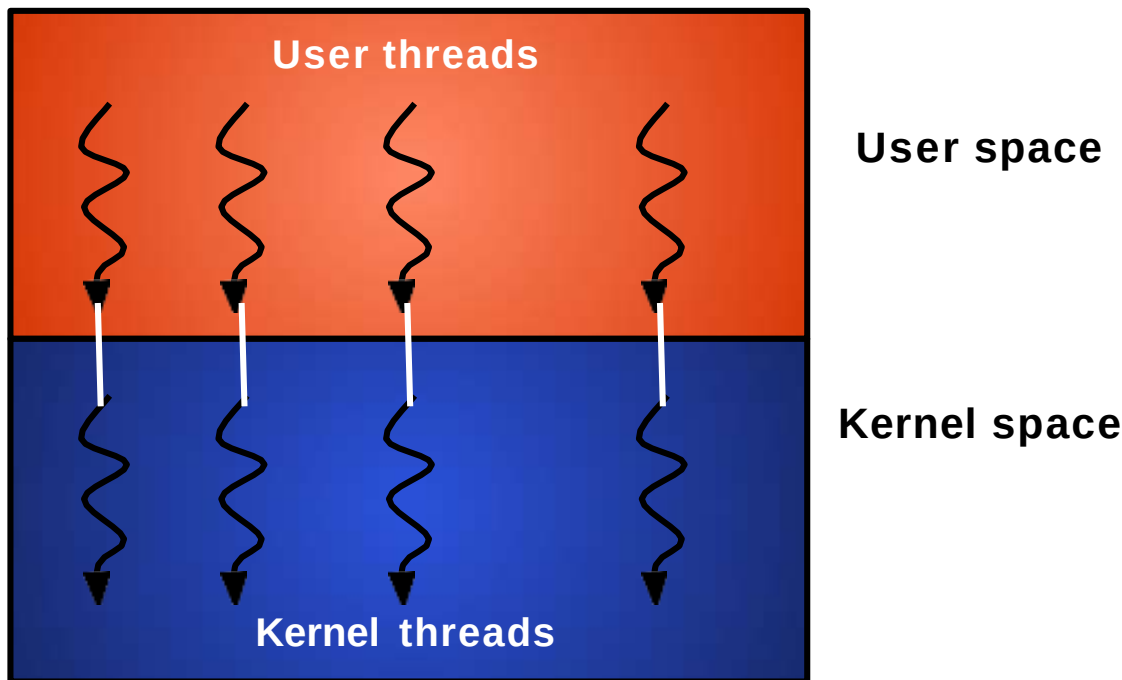
Relationship between user and kernel threads

- **Many-to-one**
- **One-to-one**
- **Many-to-many**
- Many user level threads mapped to one kernel thread
- One thread blocking cause all to block
- Multiple threads may not run in parallel because only one may be in kernel at a time



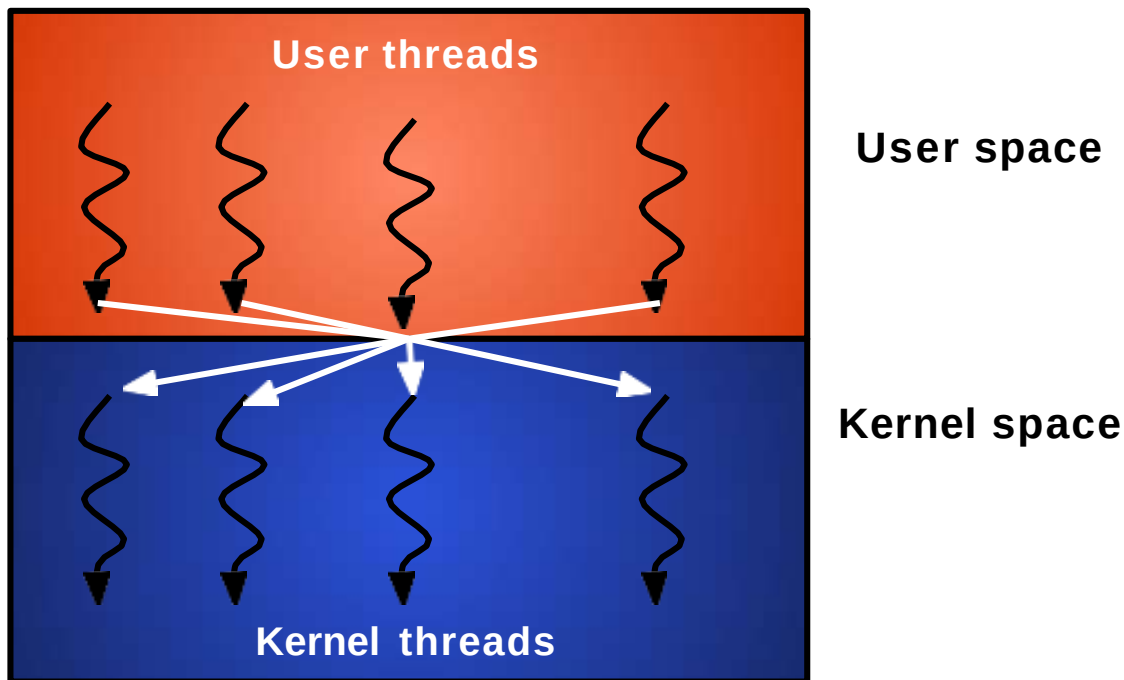
Relationship between user and kernel threads

- Many-to-one
- **One-to-one**
- Many-to-many
- Whenever a new thread is created in user space, a new thread is created in kernel space
- Each user thread, kernel thread handles system calls
- Less blocking at OS level



Relationship between user and kernel threads

- Many-to-one
- One-to-one
- **Many-to-many**
- Allows many user threads mapped to many kernel threads
- Allows OS to create sufficient number of kernel threads



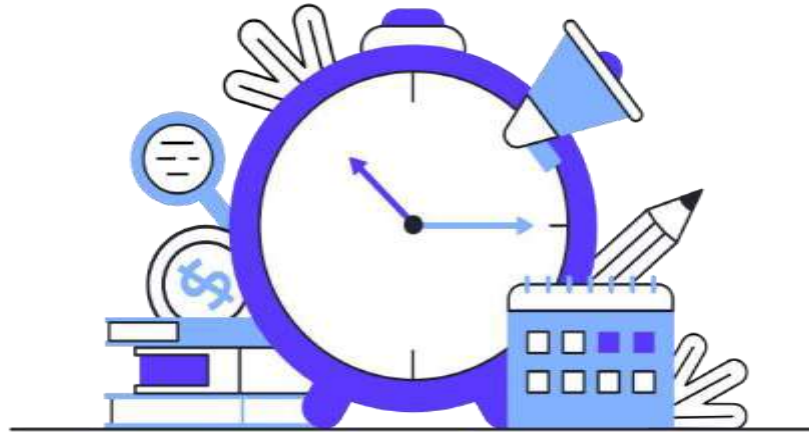
Thread Libraries

- Thread libraries provide programmer with API creating and managing threads
- Two primary ways of implementing
 1. Library entirely in user space
 2. Kernel library supported by OS
- Three primary thread libraries
 1. POSIX Pthreads
 2. Windows threads
 3. Java threads
 4. OpenMP

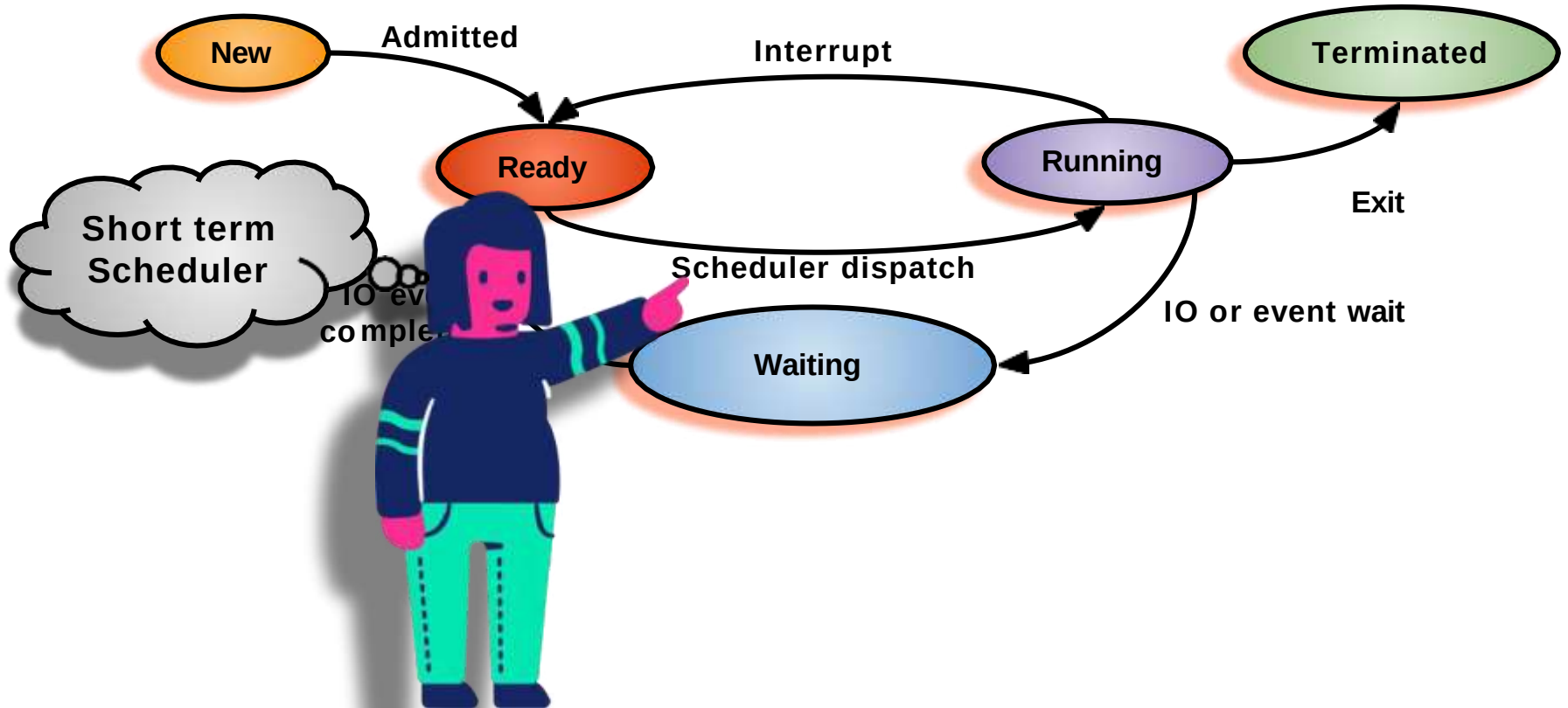


Refer programs from “pthread and openMP” folder

CPU Scheduling

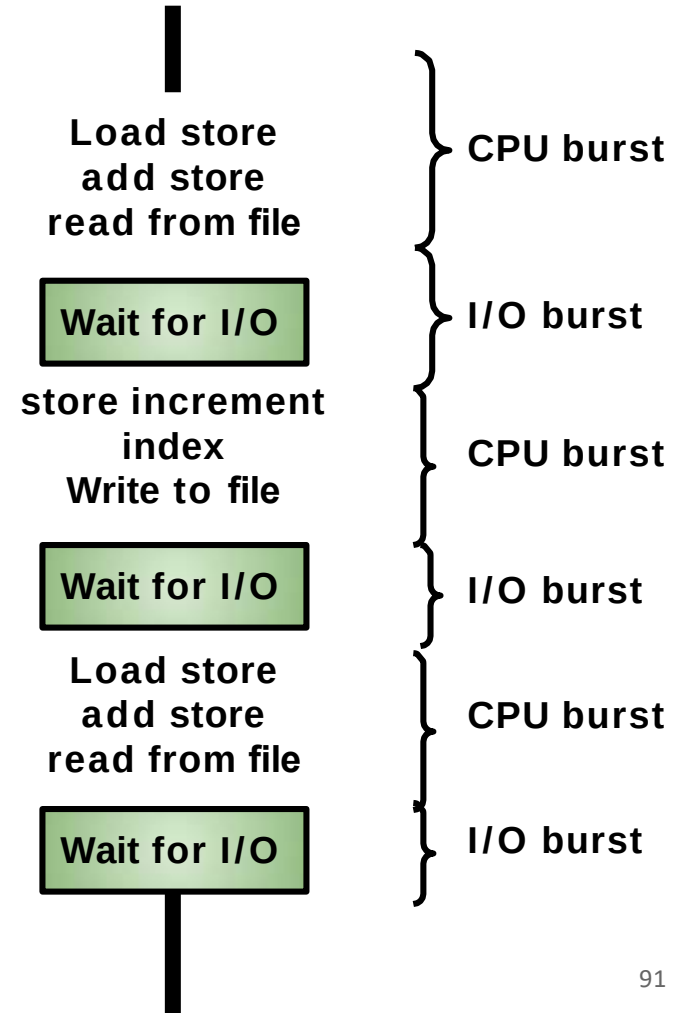


CPU Scheduling



Process execution

- Goal of a scheduler is to maximize CPU utilization
- A process continuously switch between I/O and CPU burst
- CPU burst is of main concern
- Scheduler is an overhead hence it must be as simple as possible
- If a scheduler is complex then it also consumes more CPU burst



CPU Scheduler

- Whenever CPU becomes idle, the OS must select one of the processes in the ready queue to be executed.
- The selection process is carried out by the CPU scheduler.
- The ready queue may be ordered in various ways.
- CPU scheduling decisions may takes place when a process
 1. Process switches from running state to waiting state
 2. Process switches from running state to ready state
 3. Process switches from waiting state to ready state
 4. When a process terminates
- For situations 1and 4, there is no choice in terms of scheduling.
- There is a choice however for situations 2 and 3.

CPU Scheduling types

Preemptive Scheduling

1. CPU resource can be taken away from a process before it finishes
 - Process can be moved from running state to ready
 - Preempted when higher priority process comes
2. Overheads of switching process states
3. No starvation

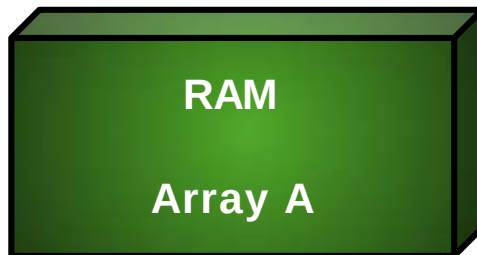
Non preemptive Scheduling

1. Once CPU has been allocated to process, the process keeps until it releases the CPU:
 - Either by terminating
 - Or by switching to the waiting state
 - There is no other way by which CPU can be taken away from the process
2. No overheads of switching states
3. May lead to starvation

Preemptive scheduling may result into race conditions

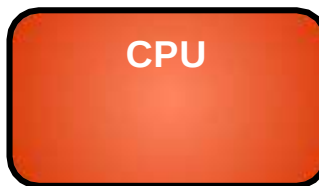
```
...  
...  
for(int i = 0; i < 1000; i++)  
{  
    A[i] = A[i] + 5;  
}  
...  
...
```

Writing array A



```
...  
...  
for(int i = 0; i < 1000; i++)  
{  
    B[i] = A[i];  
}  
...  
...
```

Reading array A



CPU is idle

Preemptive scheduling may result into race conditions

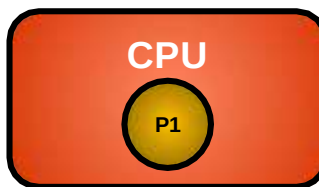
```
...  
...  
for(int i = 0; i < 1000; i++)  
{  
    A[i] = A[i] + 5;  
}  
...  
...
```

Writing array A



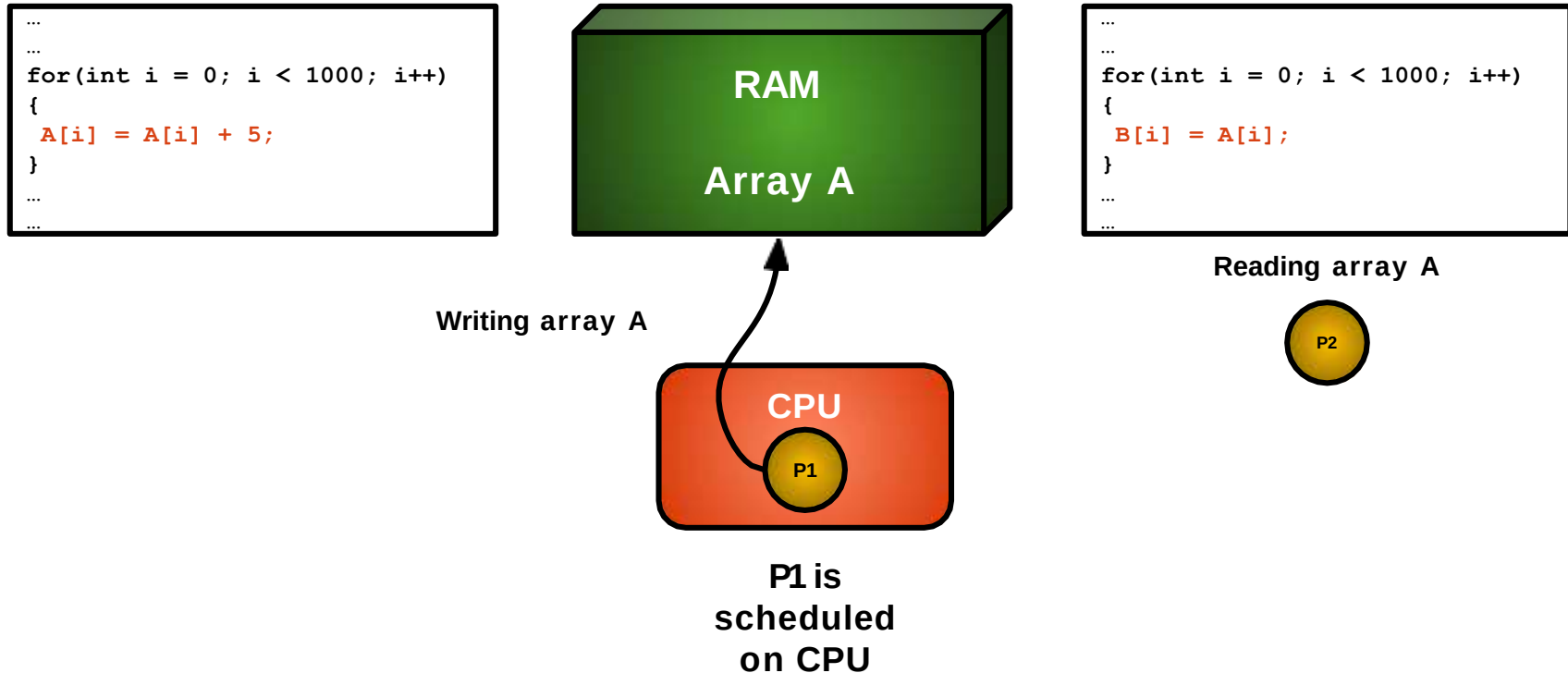
```
...  
...  
for(int i = 0; i < 1000; i++)  
{  
    B[i] = A[i];  
}  
...  
...
```

Reading array A

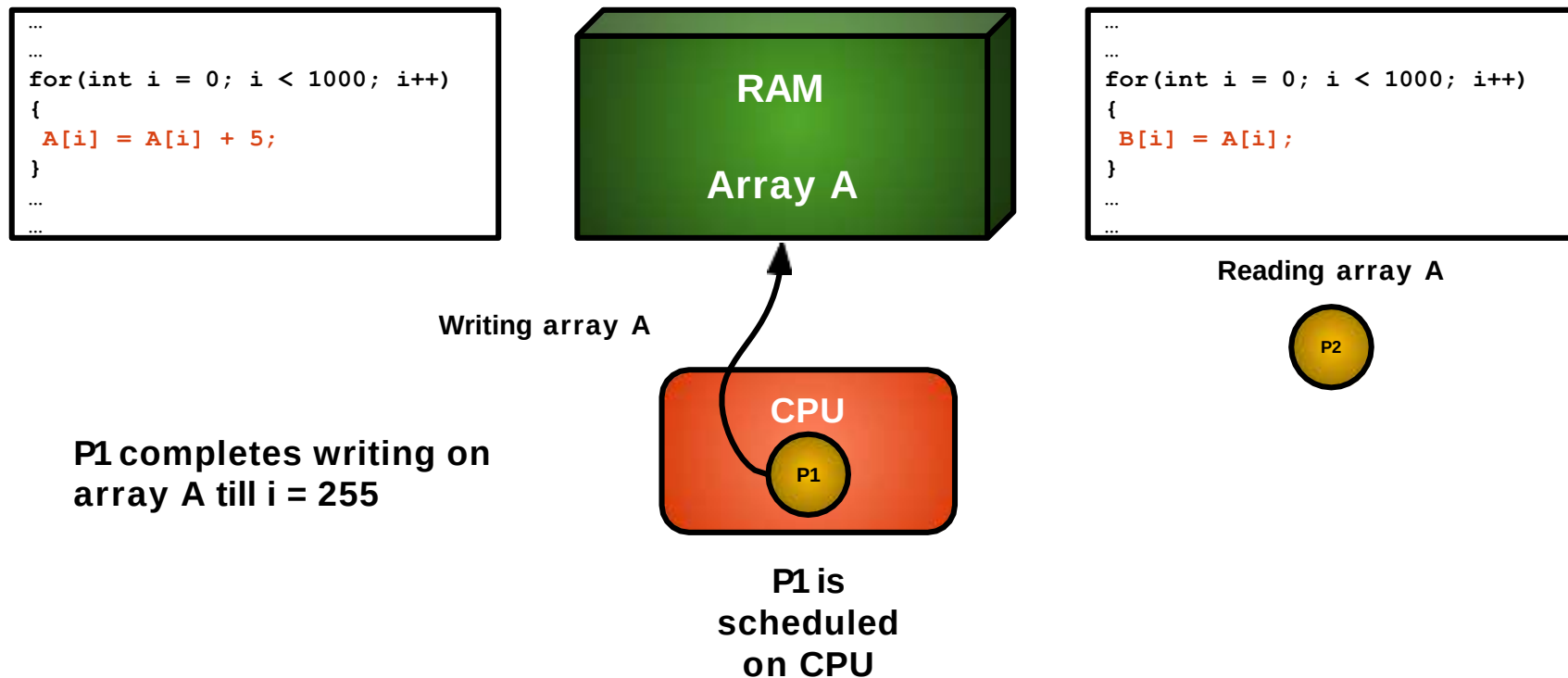


P1 is
scheduled
on CPU

Preemptive scheduling may result into race conditions



Preemptive scheduling may result into race conditions



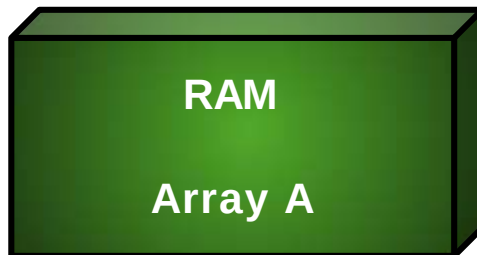
Preemptive scheduling may result into race conditions

```
...  
for(int i = 0; i < 1000; i++)  
{  
  A[i] = A[i] + 5;  
}  
...  
...
```

P1

P1 preempted from CPU

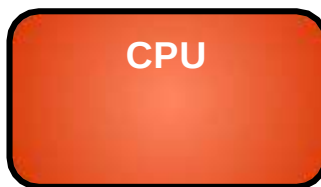
Array is partly updated
by P1



```
...  
...  
for(int i = 0; i < 1000; i++)  
{  
  B[i] = A[i];  
}  
...  
...
```

Reading array A

P2



CPU

CPU is idle

Preemptive scheduling may result into race conditions

```
...  
...  
for(int i = 0; i < 1000; i++)  
{  
    A[i] = A[i] + 5;  
}  
...  
...
```

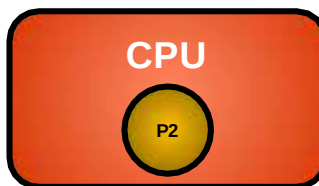
P1

P2 Scheduled on CPU



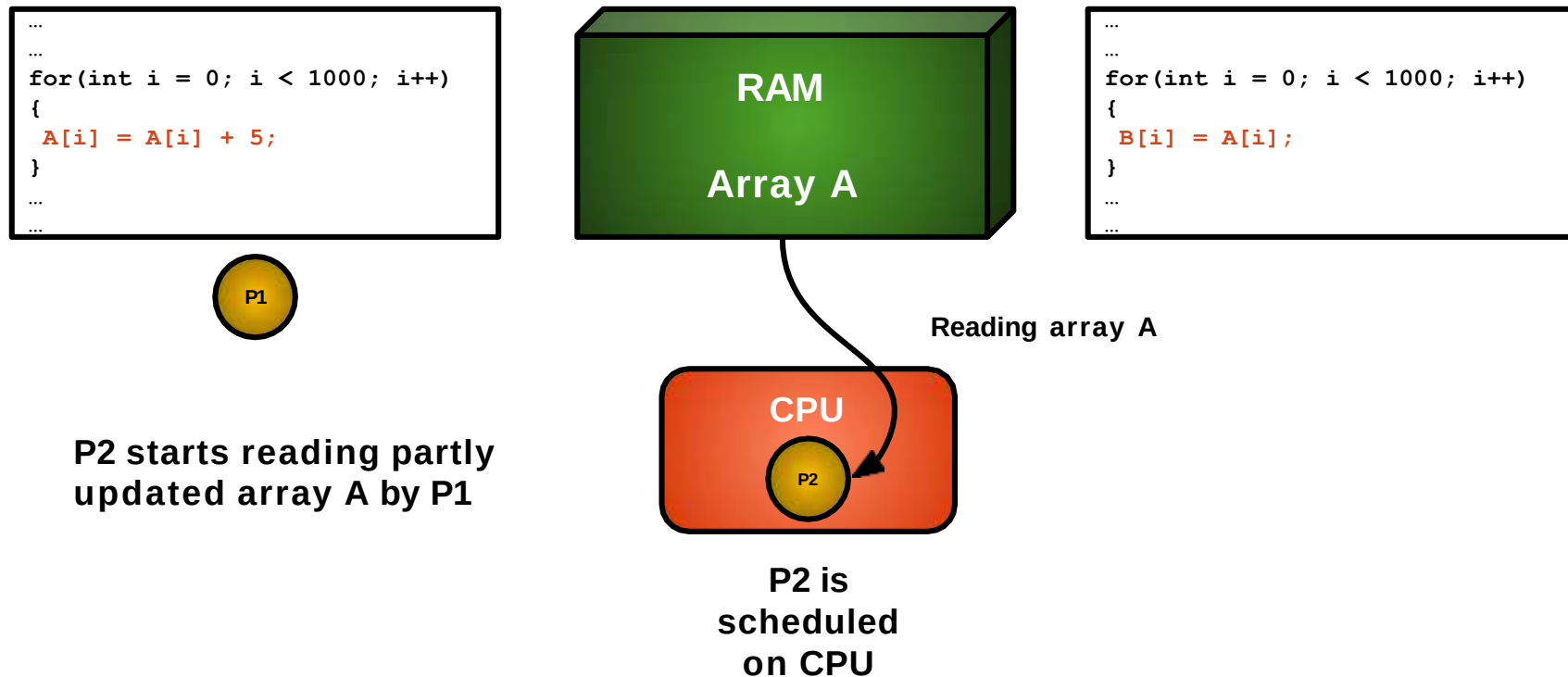
```
...  
...  
for(int i = 0; i < 1000; i++)  
{  
    B[i] = A[i];  
}  
...  
...
```

Reading array A

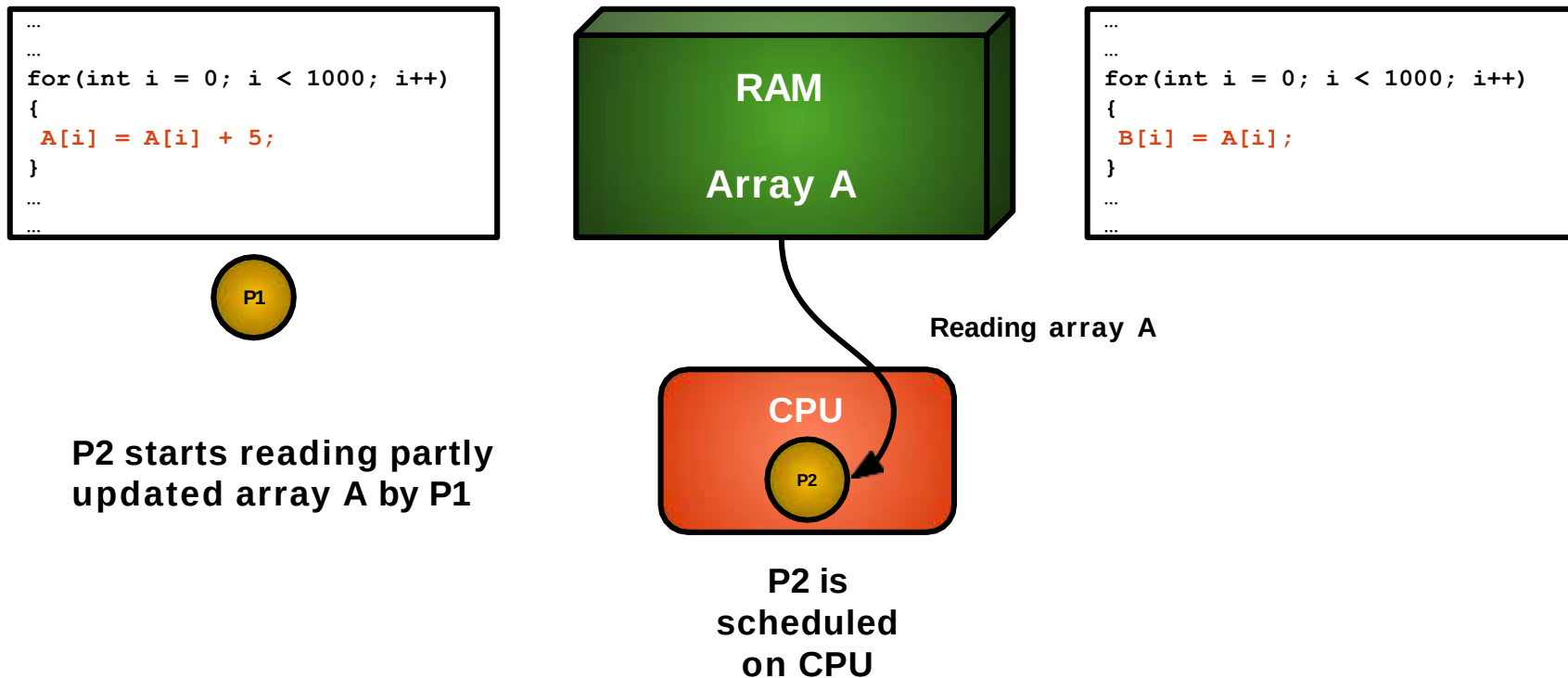


P2 is
scheduled
on CPU

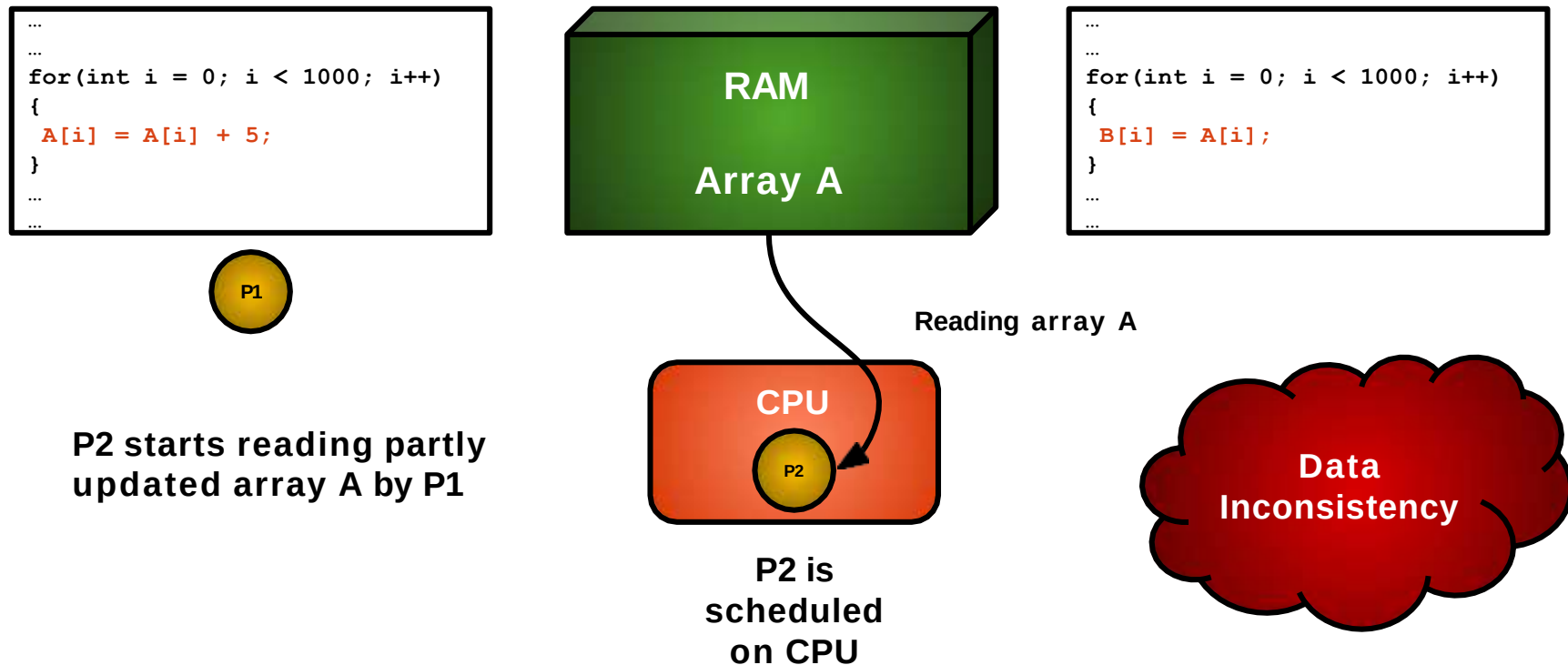
Preemptive scheduling may result into race conditions



Preemptive scheduling may result into race conditions



Preemptive scheduling may result into race conditions



Dispatcher

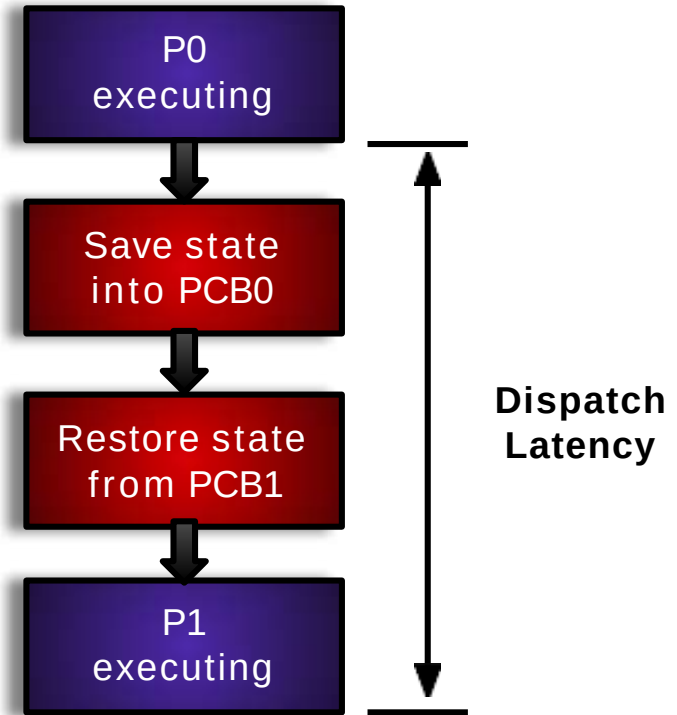
Dispatcher module gives control of the CPU to the process selected by CPU scheduler.

This involves:-

1. Switching context
2. Switching to user mode
3. Jumping to a proper location in the user program to resume that program

Dispatch Latency:- Time it takes for the dispatcher to stop one process and start another running.

It is expected that dispatch latency should be as less as possible



Scheduling criteria

- **CPU utilization:-** Amount of time where CPU is busy. Ideally 100 % CPU utilization is expected but it is less due to context switching overheads.
- **Throughput:-** Number of processes that complete their execution per unit time.
- **Turnaround time:-** Amount of time to execute a process
- **Waiting time:-** total amount of time a process has been waiting in the ready queue.
- **Response time:-** Amount of time it takes from when a process request was submitted until the first response is produced (no output) for time sharing environment.

Optimization criteria

- **Maximize CPU utilization**
- **Maximize throughput**
- **Minimize turnaround time**
- **Minimize waiting time**
- **Minimize response time**

Scheduling Algorithms

- **First-come-first serve (FCFS)**
- **Shortest job first (SJF)**
- **Round robin scheduling (RR)**
- **Priority Scheduling (PR)**
- **Multi-level queue scheduling**



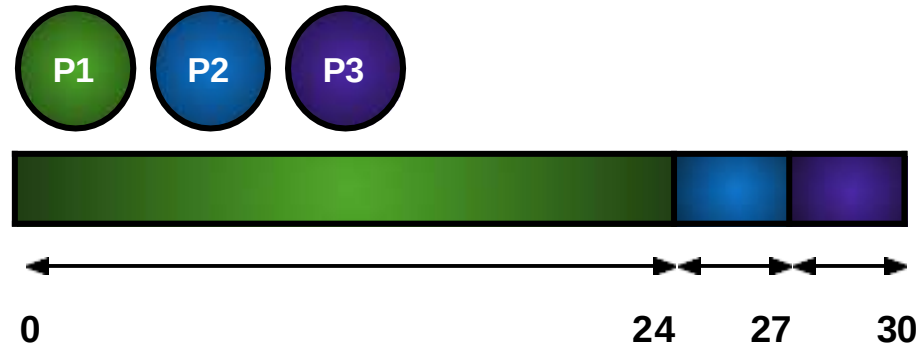
First Come First Serve

First come first serve scheduling

Scenario 1

Consider following three processes and their burst time
Consider processes arrives in order: P1, P2, P3

Process	Burst time
P1	24
P2	3
P3	3



Waiting time for P1 :- 0

Waiting time for P2 :- 24

Waiting time for P3 :- 27

$$\text{Average Waiting time} = (0 + 24 + 27)/3 = 17$$

First come first serve scheduling

Scenario 2

Consider following three processes and their burst time
Consider processes arrives in order: P1, P2, P3

Process	Burst time
P1	24
P2	3
P3	3



Waiting time for P1 :- 6

Waiting time for P2 :- 0

Waiting time for P3 :- 3

$$\text{Average Waiting time} = (0 + 6 + 3)/3 = 3$$

Convoy effect in FCFS

- FCFS algorithm is **non-preemptive in nature**, that is, once CPU time has been allocated to a process, other processes can get CPU time only after the current process has finished.
- This property of FCFS scheduling leads to the situation called Convoy Effect.



Convoy effect in FCFS

Suppose there is one CPU intensive (large burst time) process in the ready queue, and several other processes with relatively less burst times but are Input/Output (I/O) bound (Need I/O operations frequently).

Steps are as following below:

- The I/O bound processes are first allocated CPU time. As they are less CPU intensive, they quickly get executed and goto I/O queues.
- Now, the CPU intensive process is allocated CPU time. As its burst time is high, it takes time to complete.
- While the CPU intensive process is being executed, the I/O bound processes complete their I/O operations and are moved back to ready queue.
- However, the I/O bound processes are made to wait as the CPU intensive process still hasn't finished. This leads to I/O devices being idle.
- When the CPU intensive process gets over, it is sent to the I/O queue so that it can access an I/O device.
- Meanwhile, the I/O bound processes get their required CPU time and move back to I/O queue.
- However, they are made to wait because the CPU intensive process is still accessing an I/O device. As a result, the CPU is sitting idle now.

Advantages of FCFS

- **Simple** to implement
- Eventually, every process will get a chance to run, so **starvation doesn't occur**
- Scheduling **overheads are less** as no preemption

Disadvantages of FCFS

- There is **no option for pre-emption** of a process. If a process is started, then CPU executes the process until it ends.
- The process with less execution time suffer i.e. **waiting time is often quite long.**
- May suffer from **Convoy effect**, This effect results in lower CPU and device utilization.
- FCFS algorithm is particularly troublesome for time-sharing systems, where it is important that each user get a share of the CPU at regular intervals.

Assignment 1

Process	Burst Time	Arrival Time
P1	6	2
P2	2	4
P3	8	1
P4	3	0
P5	4	5

Using FCFS Scheduling calculate average waiting time for the processes



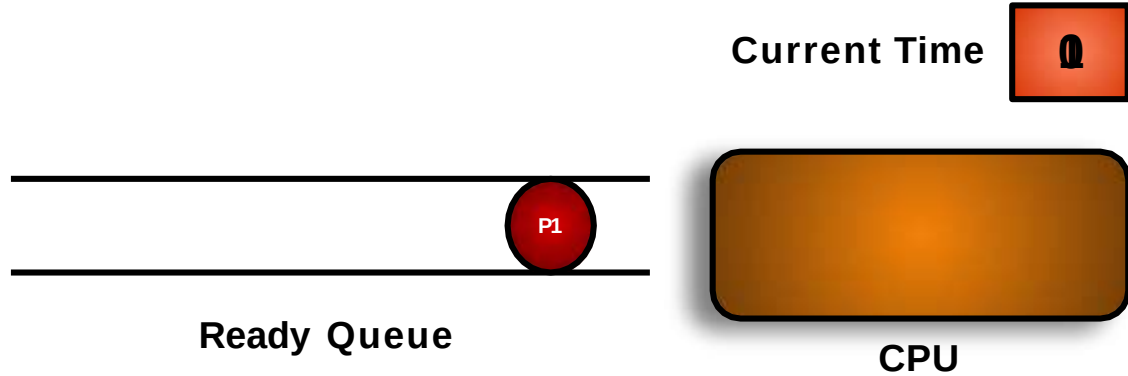
Shortest Job First Scheduling

Shortest Job First Scheduling

- **Shortest job first (SJF) or Shortest Job Next (SJN)**, is a scheduling policy that selects the waiting process with the smallest execution time to execute next. SJN is a non-preemptive algorithm.
- Shortest Job first has the advantage of having a minimum average waiting time among all scheduling algorithms.
- It is a **Greedy Algorithm**.
- It may cause **starvation** if shorter processes keep coming. This problem can be solved using the concept of **ageing**.
- It is practically infeasible as Operating System may not know burst time and therefore may not sort them. While it is not possible to predict execution time, several methods can be used to estimate the execution time for a job, such as a weighted average of previous execution times.
- SJF can be used in specialized environments where accurate estimates of running time are available.

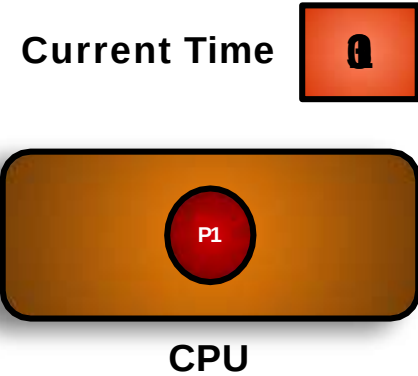
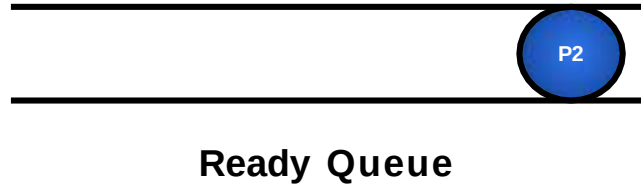
Shortest Job First Scheduling

PID	Arrival Time	Burst Time
1	1	7
2	3	3
3	6	2
4	7	10
5	9	8



Shortest Job First Scheduling

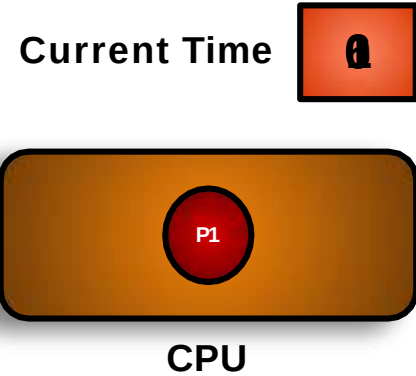
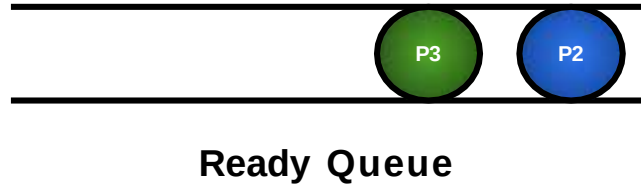
PID	Arrival Time	Burst Time
1	1	7
2	3	3
3	6	2
4	7	10
5	9	8



1 3

Shortest Job First Scheduling

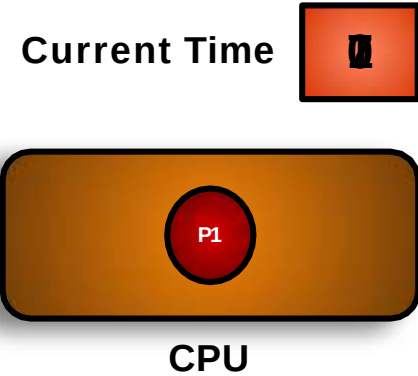
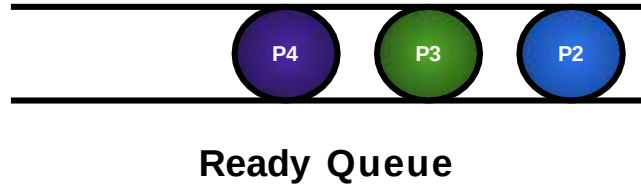
PID	Arrival Time	Burst Time
1	1	7
2	3	3
3	6	2
4	7	10
5	9	8



1 3

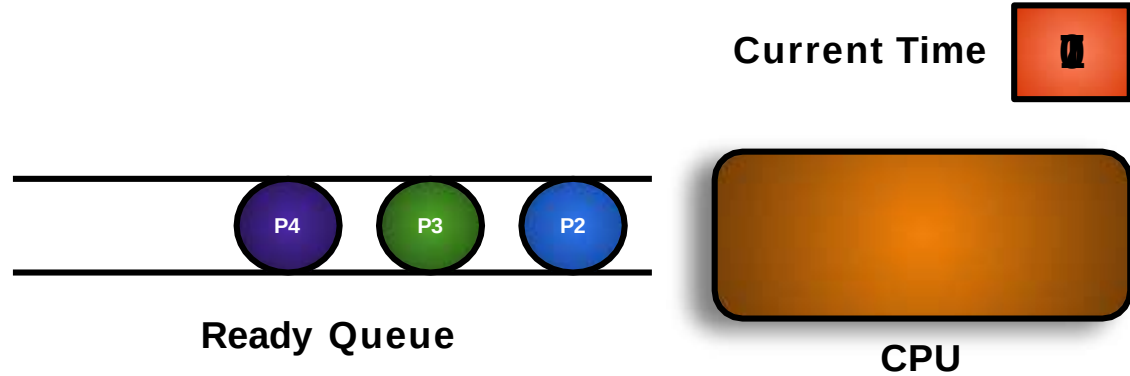
Shortest Job First Scheduling

PID	Arrival Time	Burst Time
1	1	7
2	3	3
3	6	2
4	7	10
5	9	8

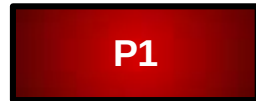


Shortest Job First Scheduling

PID	Arrival Time	Burst Time
1	1	7
2	3	3
3	6	2
4	7	10
5	9	8



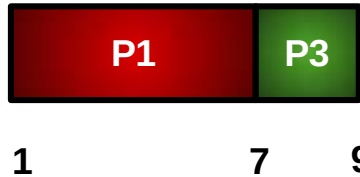
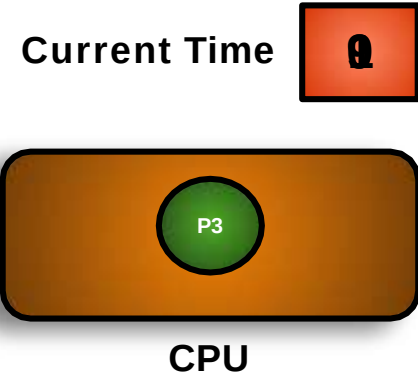
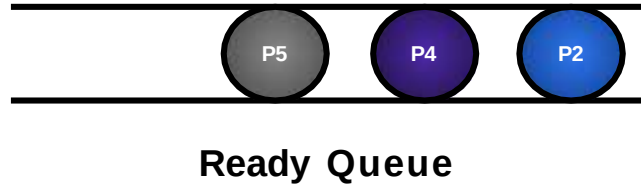
P3 has shortest burst time in ready queue



1 7

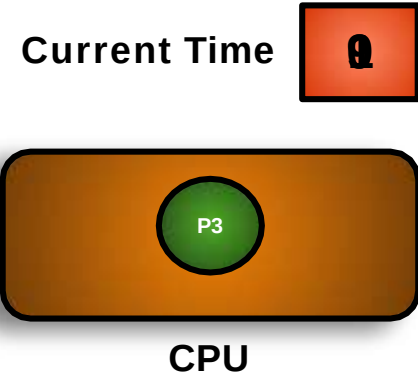
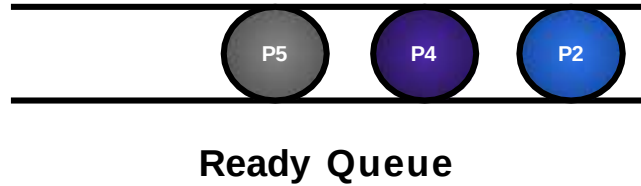
Shortest Job First Scheduling

PID	Arrival Time	Burst Time
1	1	7
2	3	3
3	6	2
4	7	10
5	9	8

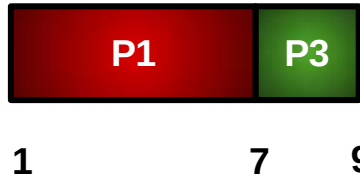


Shortest Job First Scheduling

PID	Arrival Time	Burst Time
1	1	7
2	3	3
3	6	2
4	7	10
5	9	8

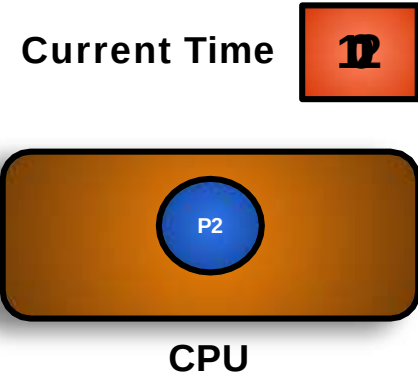
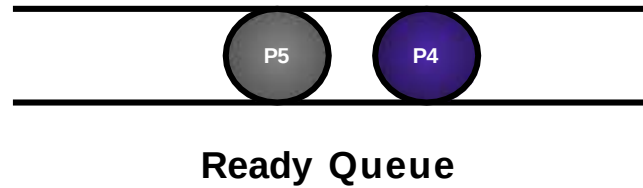


P2 has shortest burst time in ready queue



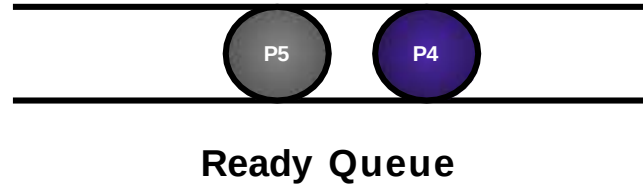
Shortest Job First Scheduling

PID	Arrival Time	Burst Time
1	1	7
2	3	3
3	6	2
4	7	10
5	9	8

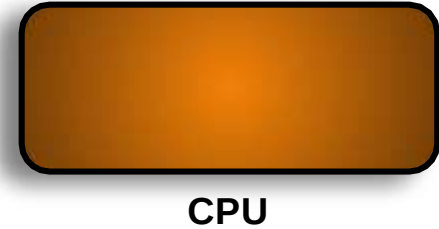


Shortest Job First Scheduling

PID	Arrival Time	Burst Time
1	1	7
2	3	3
3	6	2
4	7	10
5	9	8



Current Time 



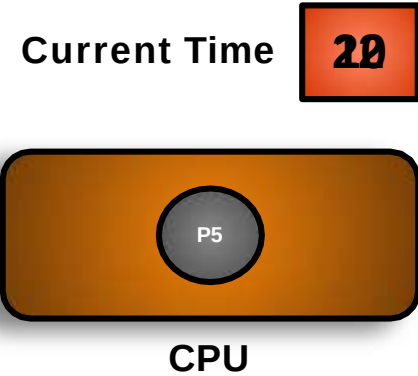
P5 has shortest burst time in ready queue



1 7 9 12

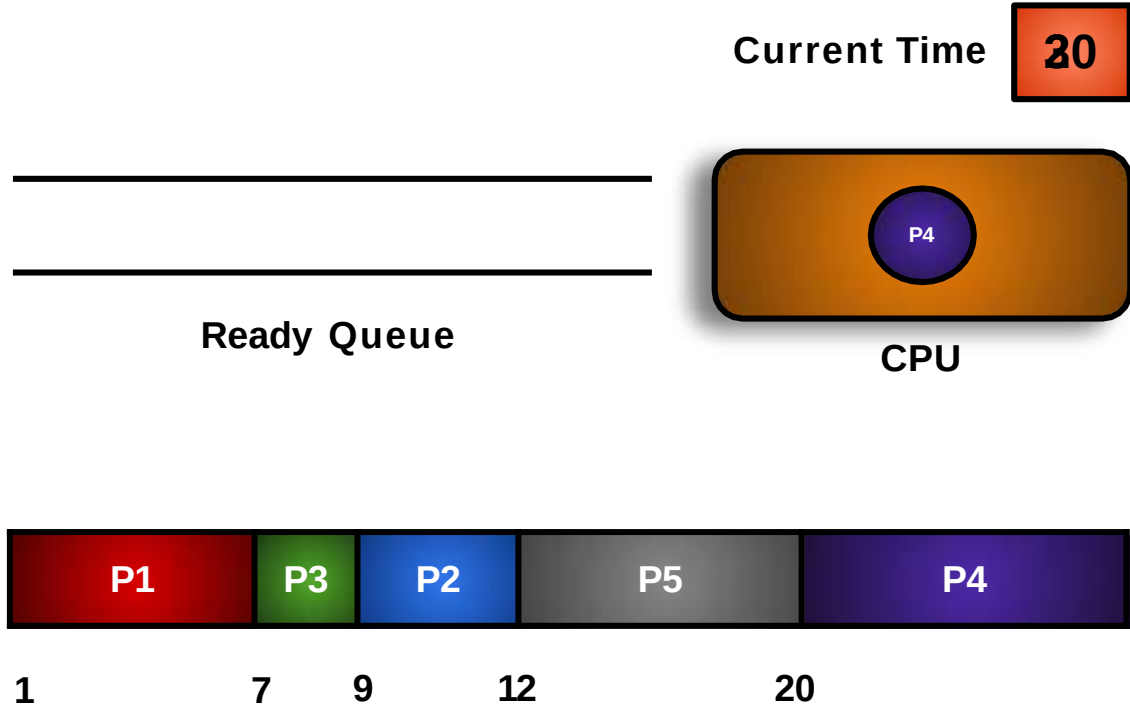
Shortest Job First Scheduling

PID	Arrival Time	Burst Time
1	1	7
2	3	3
3	6	2
4	7	10
5	9	8



Shortest Job First Scheduling

PID	Arrival Time	Burst Time
1	1	7
2	3	3
3	6	2
4	7	10
5	9	8



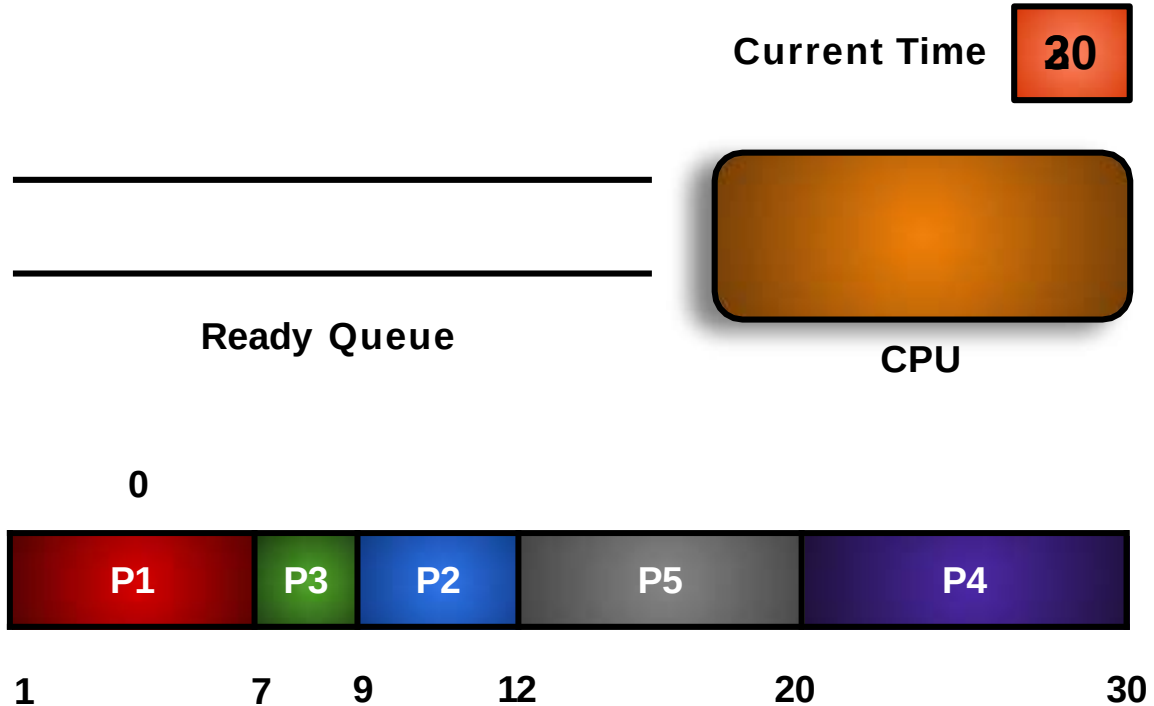
Shortest Job First Scheduling

PID	Arrival Time	Burst Time
1	1	7
2	3	3
3	6	2
4	7	10
5	9	8

waiting time =
start time - arrival time



$$1 - 1 = 0$$



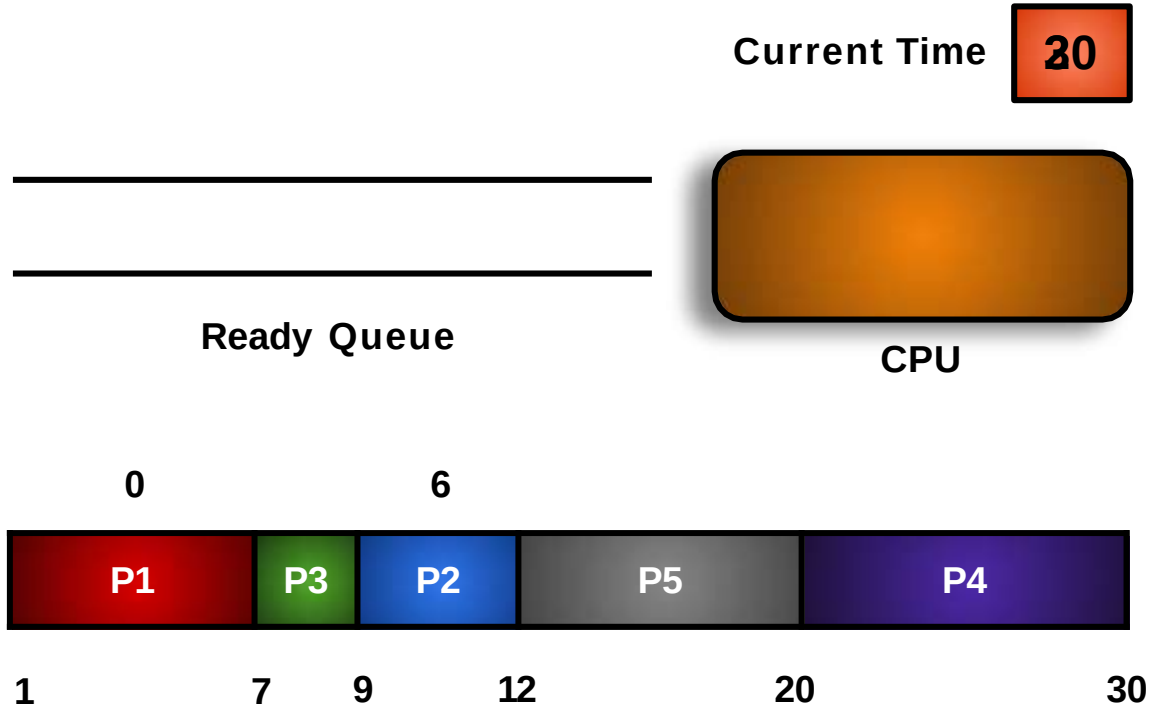
Shortest Job First Scheduling

PID	Arrival Time	Burst Time
1	1	7
2	3	3
3	6	2
4	7	10
5	9	8

waiting time =
start time - arrival time



$$9 - 3 = 6$$



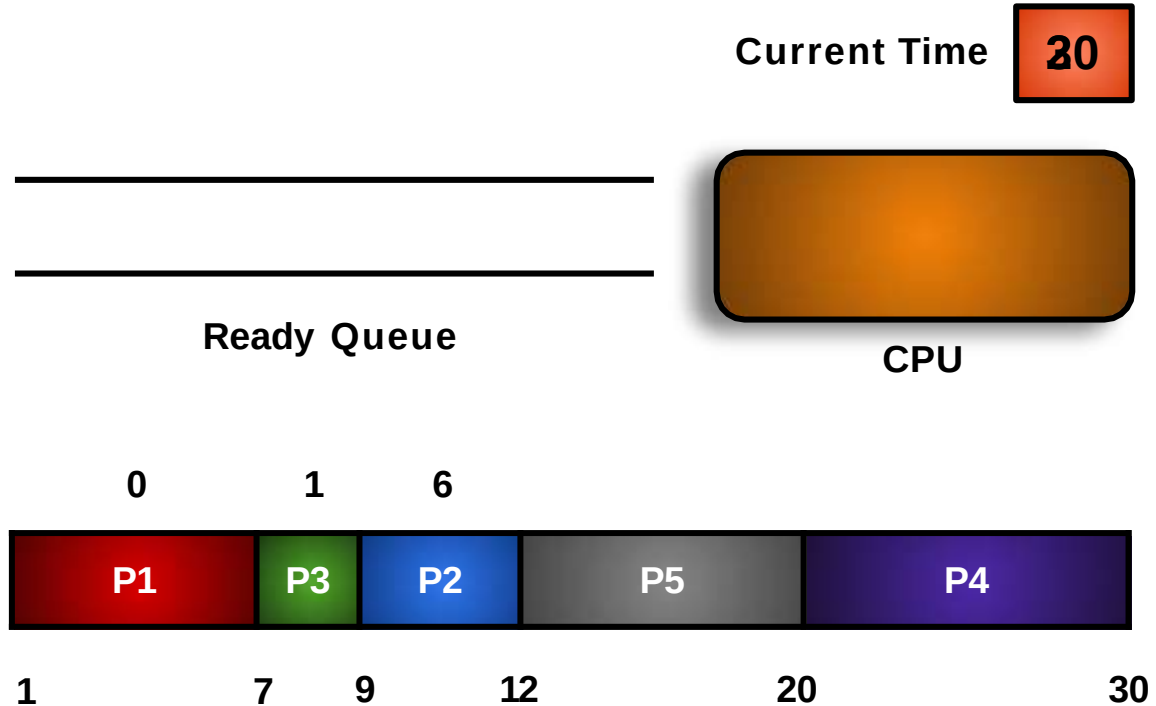
Shortest Job First Scheduling

PID	Arrival Time	Burst Time
1	1	7
2	3	3
3	6	2
4	7	10
5	9	8

waiting time =
start time - arrival time



$$7 - 6 = 1$$



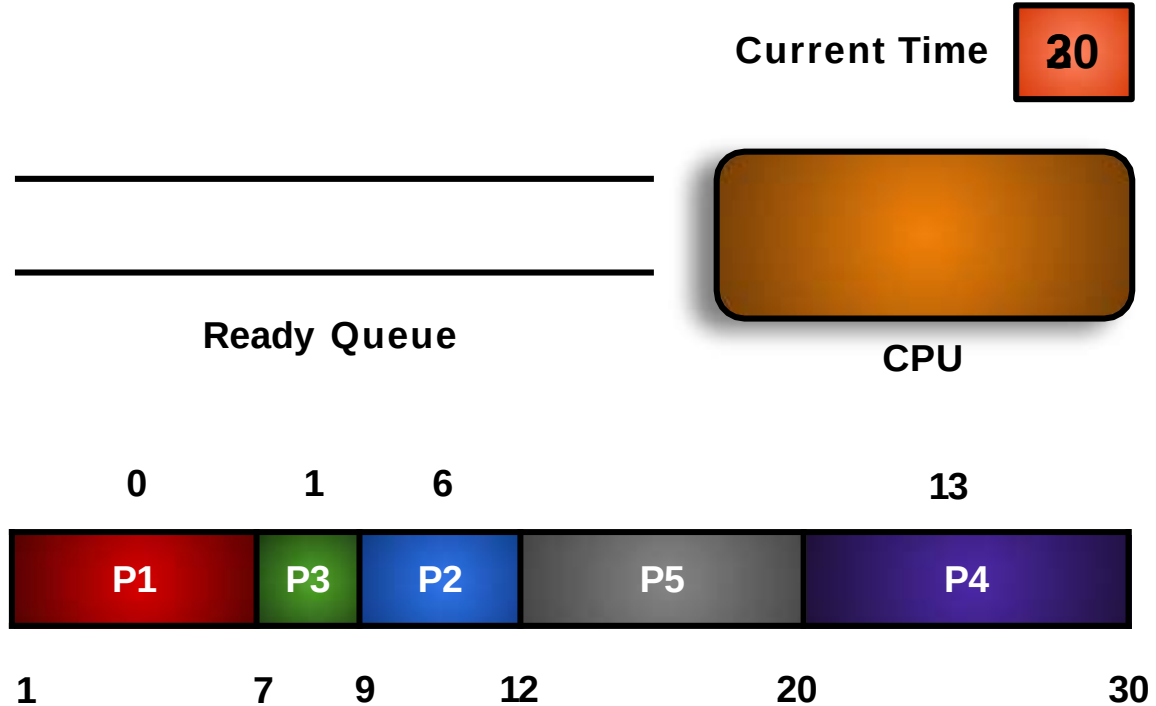
Shortest Job First Scheduling

PID	Arrival Time	Burst Time
1	1	7
2	3	3
3	6	2
4	7	10
5	9	8

waiting time =
start time - arrival time



$$20 - 7 = 13$$



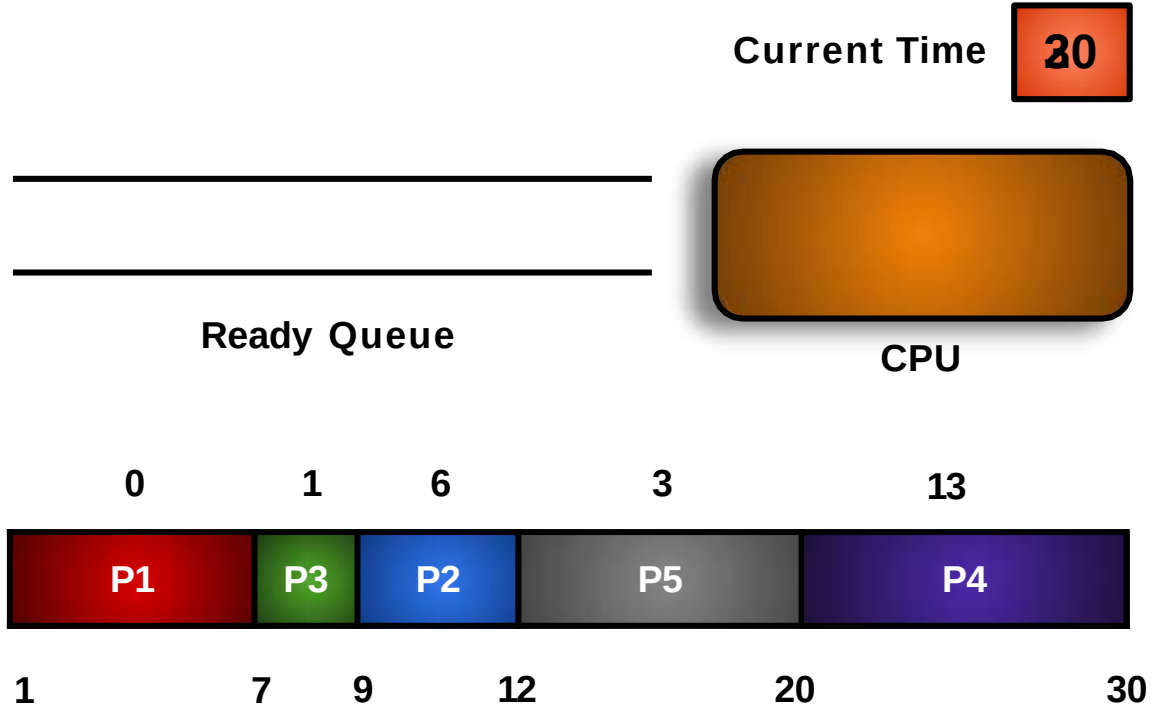
Shortest Job First Scheduling

PID	Arrival Time	Burst Time
1	1	7
2	3	3
3	6	2
4	7	10
5	9	8

waiting time =
start time - arrival time



$$12 - 9 = 3$$

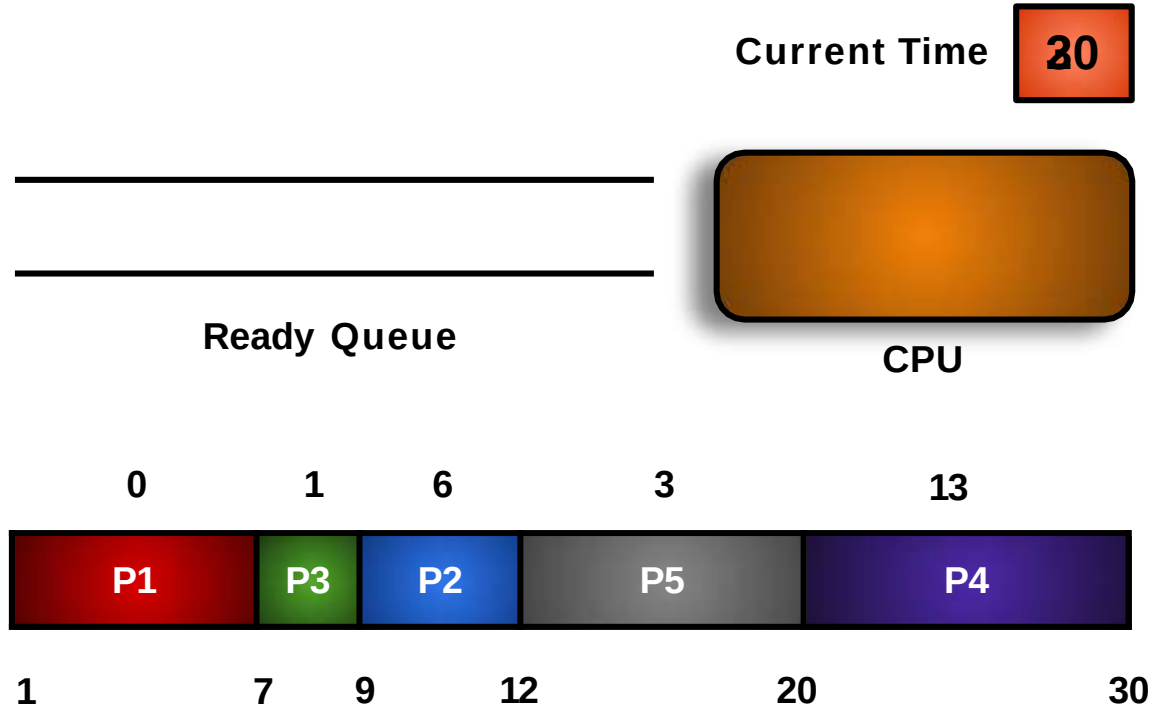


Shortest Job First Scheduling

PID	Arrival Time	Burst Time
1	1	7
2	3	3
3	6	2
4	7	10
5	9	8

Average waiting Time

$$\begin{aligned} & (0 + 6 + 1 + 13 + 3)/5 \\ & = 23/5 \\ & = 4.6 \end{aligned}$$



Assignment 2

Process	Arrival Time	Burst Time
P1	0.0	6
P2	2.0	8
P3	4.0	7
P4	5.0	3

Find average waiting time for the given processes considering SJF scheduling

Problem with SJF

- Not possible to know next CPU burst time in advance
- While it is not possible to predict next CPU burst accurately, some **approximation methods** can be used.
- The next CPU burst of a process can be **predicted from previous burst**
- But this is **only an estimate**, it may or may not work
- **Exponential averaging** of previous CPU burst can be used to predict next CPU burst

$$T_{n+1} = \alpha t_n + (1 - \alpha)T_n$$

where α = is smoothing factor and $0 \leq \alpha \leq 1$,

t_n = actual burst time of n th process,

T_n = predicted burst time of n th process.

Problem with SJF

- Not possible to know next CPU burst time in advance
- While it is not possible to predict next CPU burst accurately, some **approximation methods** can be used.
- The next CPU burst of a process can be **predicted from previous burst**
- But this is **only an estimate**, it may or may not work
- **Exponential averaging** of previous CPU burst can be used to predict next CPU burst

$$T_{n+1} = \alpha t_n + (1 - \alpha)T_n$$

$$\alpha t_n + (1 - \alpha)\alpha t_{n-1} + (1 - \alpha)^2 \alpha t_{n-2} \dots + (1 - \alpha)^j \alpha t_{n-j} \dots + (1 - \alpha)^{n+1} T_0$$

Advantages of SJF

- Shortest jobs are favored.
- It is optimal, it gives the **minimum average waiting time** for a given set of processes and used as **benchmark for other scheduling algorithms**.
- The **throughput is increased** because more processes can be executed in less amount of time.

Disadvantages of SJF

- SJF may cause **starvation**, if shorter processes keep coming.
- **Longer processes will have more waiting time**, eventually they'll suffer starvation.
- The time taken by a process must be known by the CPU beforehand, which is **not possible**.

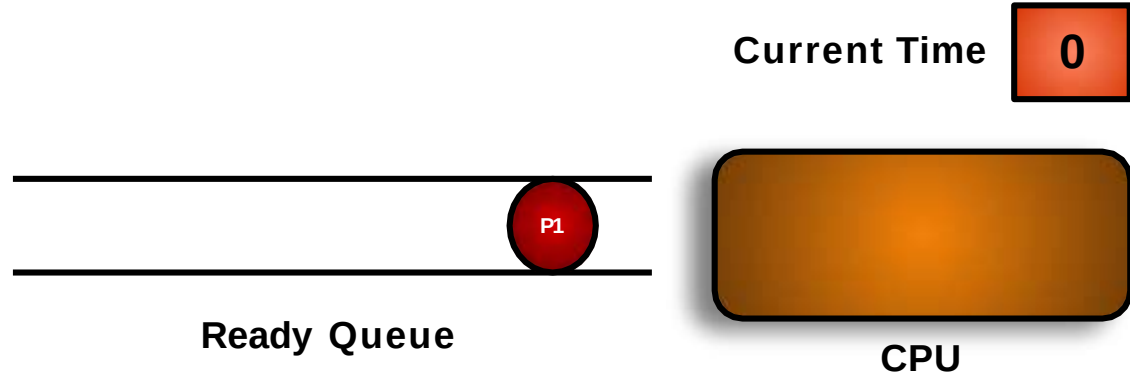


Shortest Remaining Time First

Shortest Remaining Time First (SRTF) - Preemptive SJF

PID	Arrival Time	Burst Time
1	0	8
2	1	4
3	2	9
4	3	5

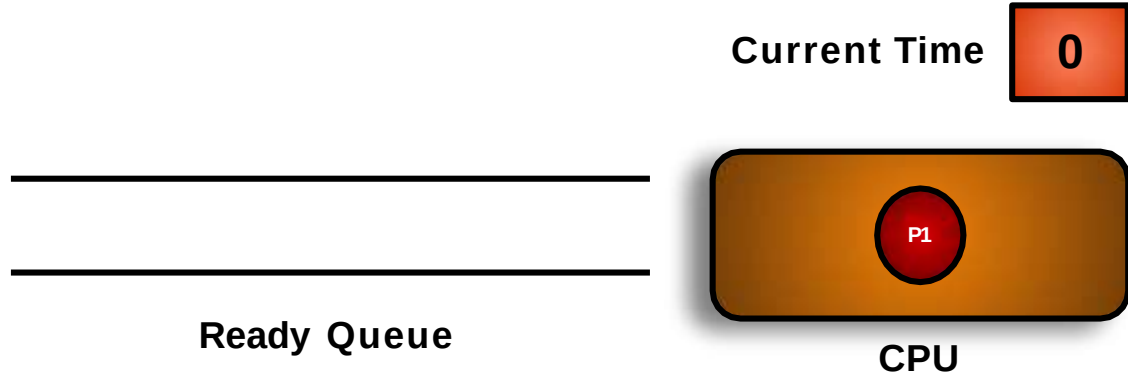
P1 arrives in ready queue



Shortest Remaining Time First (SRTF) - Preemptive SJF

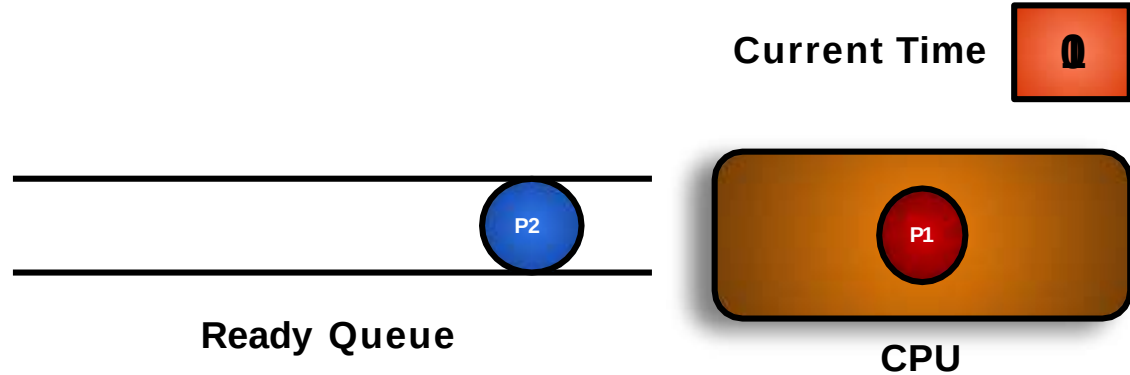
PID	Arrival Time	Burst Time
1	0	8
2	1	4
3	2	9
4	3	5

P1 starts execution



Shortest Remaining Time First (SRTF) - Preemptive SJF

PID	Arrival Time	Burst Time
1	0	8
2	1	4
3	2	9
4	3	5



P2 arrives at time 1.

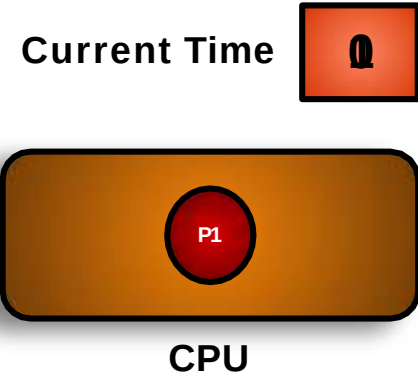
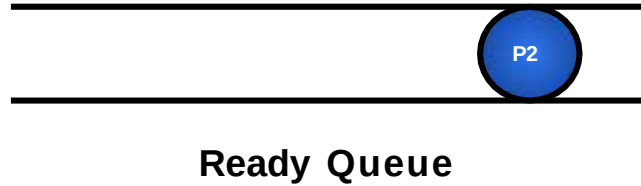
Remaining time of P1 is 7 and remaining time of P2 is 4



0 1

Shortest Remaining Time First (SRTF) - Preemptive SJF

PID	Arrival Time	Burst Time
1	0	8
2	1	4
3	2	9
4	3	5



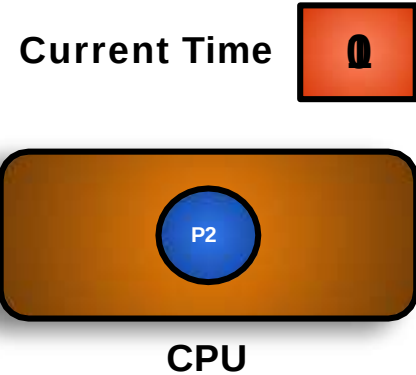
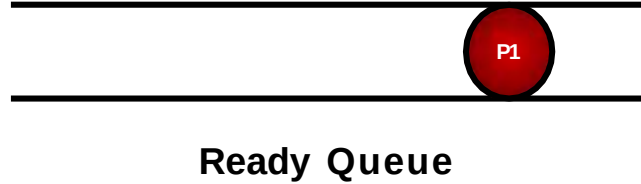
Preempt P1 and keep in ready queue, schedule P2 for execution



0 1

Shortest Remaining Time First (SRTF) - Preemptive SJF

PID	Arrival Time	Burst Time
1	0	8
2	1	4
3	2	9
4	3	5



Preempt P1 and keep in ready queue, schedule P2 for execution



0 1

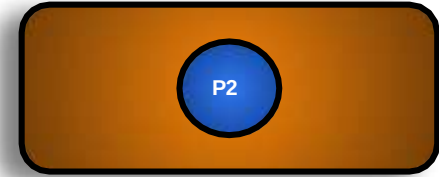
Shortest Remaining Time First (SRTF) - Preemptive SJF

PID	Arrival Time	Burst Time
1	0	8
2	1	4
3	2	9
4	3	5



Ready Queue

Current Time



CPU

Execute P2 for one time unit

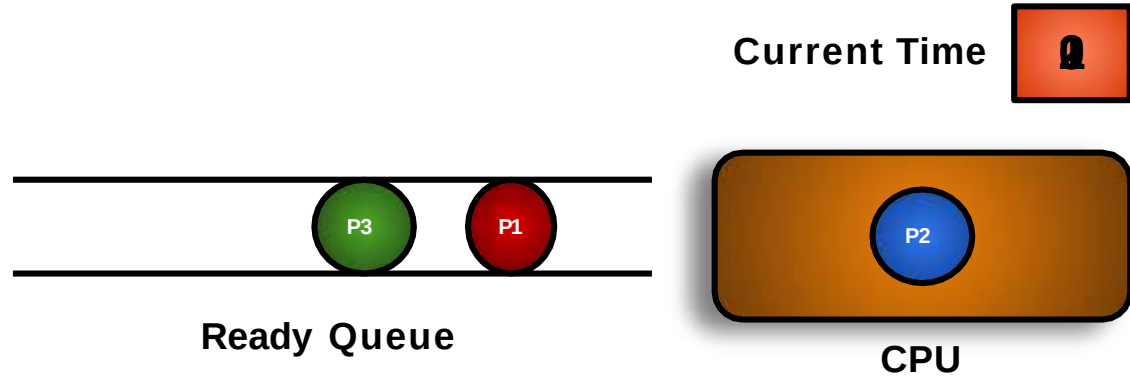
P3 arrives at time 2 and placed in ready queue



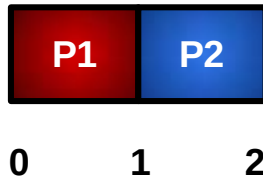
0 1 2

Shortest Remaining Time First (SRTF) - Preemptive SJF

PID	Arrival Time	Burst Time
1	0	8
2	1	4
3	2	9
4	3	5

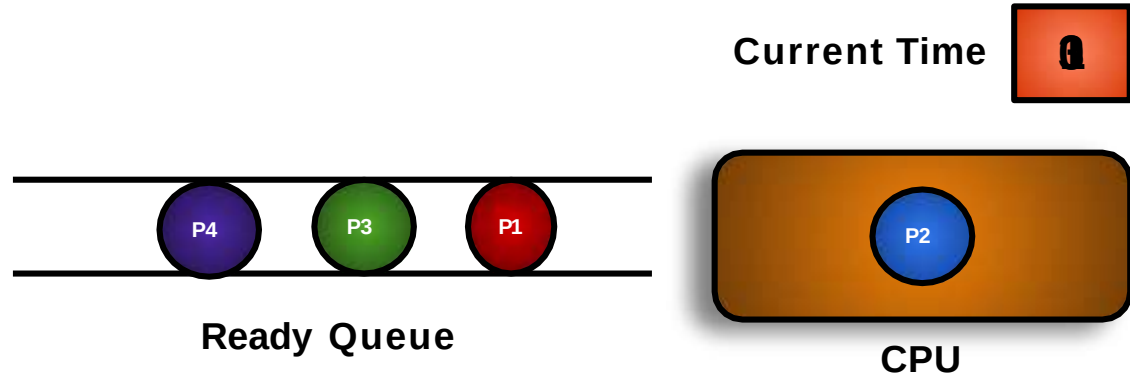


In ready queue remaining time for P1 is 7, P3 is 9 and for P2 is 3 so continue execution of P2



Shortest Remaining Time First (SRTF) - Preemptive SJF

PID	Arrival Time	Burst Time
1	0	8
2	1	4
3	2	9
4	3	5

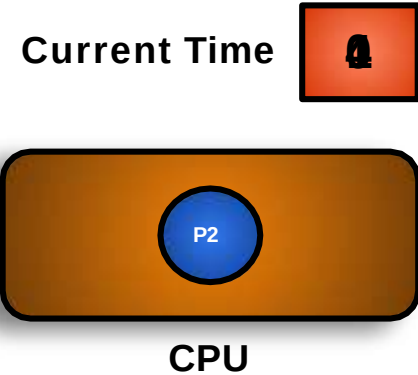
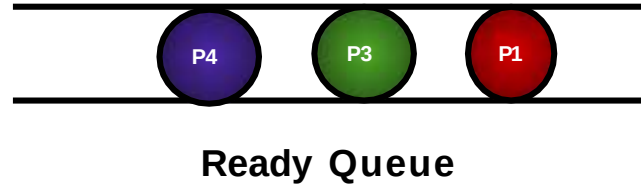


In ready queue remaining time for P1 is 7, P3 is 9, P4 is 5 and for P2 is 2 so continue execution of P2 and P4 arrives in ready queue



Shortest Remaining Time First (SRTF) - Preemptive SJF

PID	Arrival Time	Burst Time
1	0	8
2	1	4
3	2	9
4	3	5

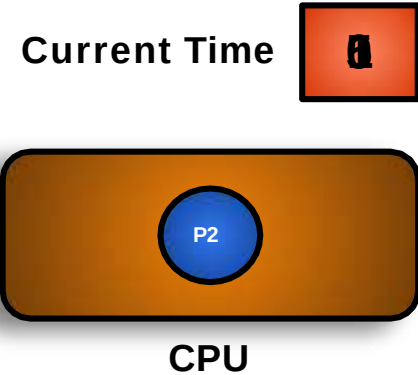
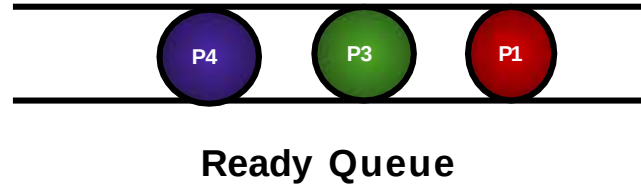


In ready queue remaining time for P1 is 7, P3 is 9, P4 is 5 and for P2 is 2 so continue execution of P2

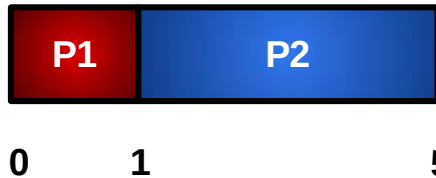


Shortest Remaining Time First (SRTF) - Preemptive SJF

PID	Arrival Time	Burst Time
1	0	8
2	1	4
3	2	9
4	3	5

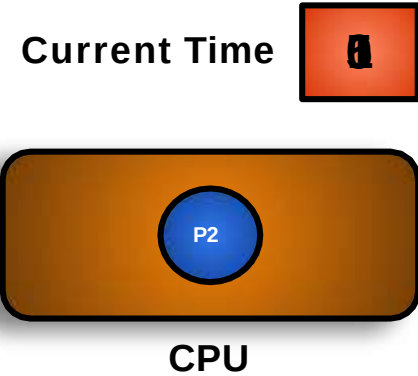
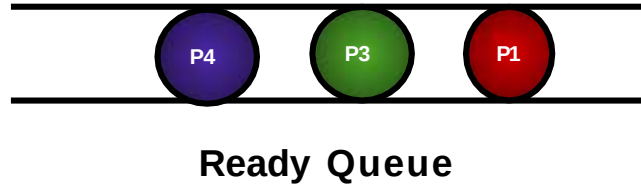


In ready queue remaining time for P1 is 7, P3 is 9, P4 is 5 and for P2 is 1 so continue execution of P2

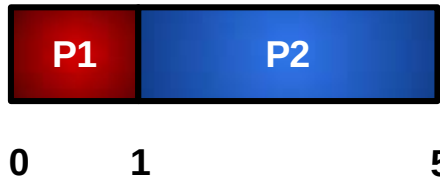


Shortest Remaining Time First (SRTF) - Preemptive SJF

PID	Arrival Time	Burst Time
1	0	8
2	1	4
3	2	9
4	3	5

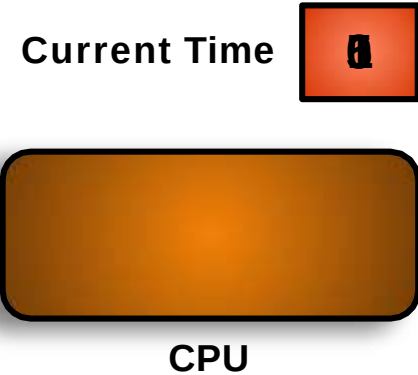
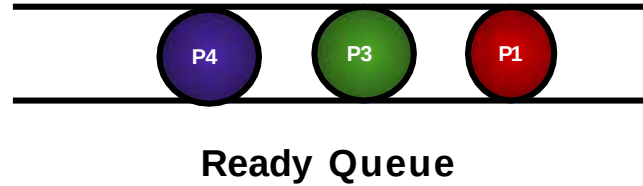


P2 completed its execution

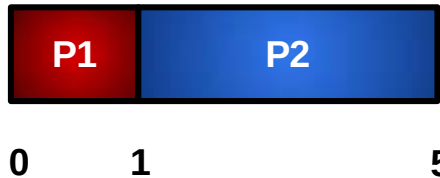


Shortest Remaining Time First (SRTF) - Preemptive SJF

PID	Arrival Time	Burst Time
1	0	8
2	1	4
3	2	9
4	3	5

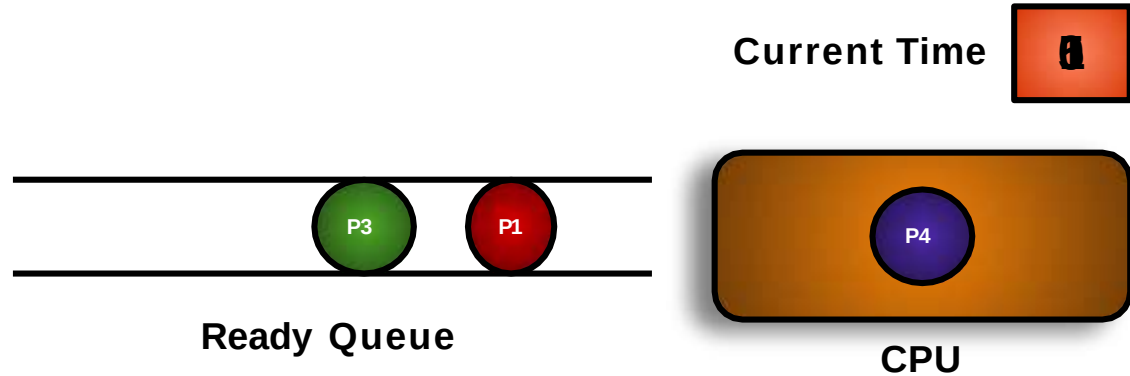


The remaining time of P1 is 7 P3 is 9 and P4 is 5 so P4 gets CPU

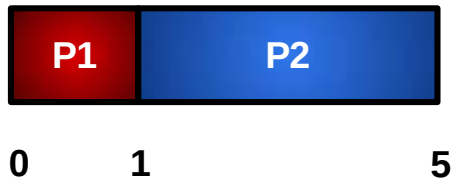


Shortest Remaining Time First (SRTF) - Preemptive SJF

PID	Arrival Time	Burst Time
1	0	8
2	1	4
3	2	9
4	3	5

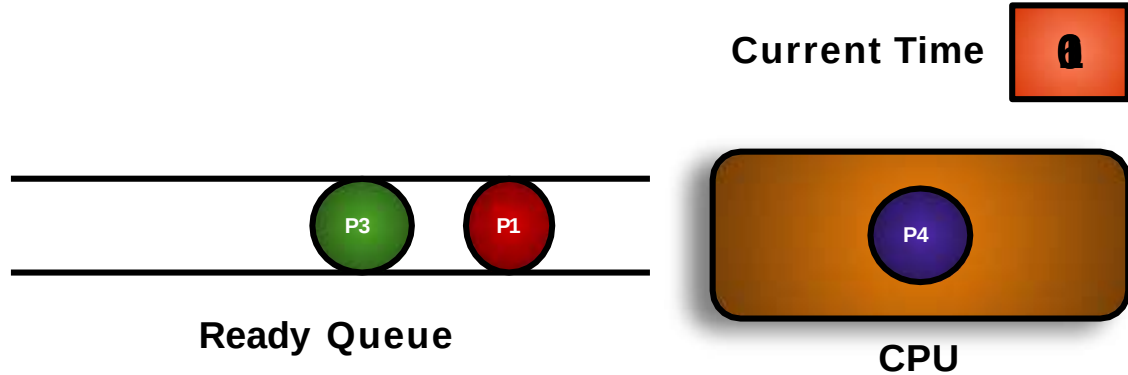


The remaining time of P1 is 7 P3 is 9 and P4 is 5 so P4 gets CPU

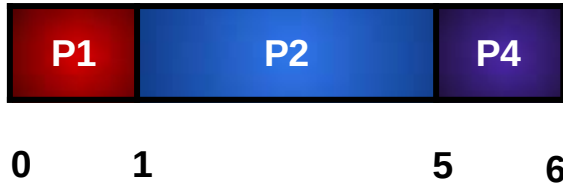


Shortest Remaining Time First (SRTF) - Preemptive SJF

PID	Arrival Time	Burst Time
1	0	8
2	1	4
3	2	9
4	3	5

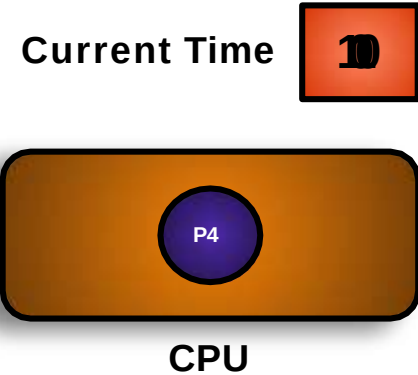
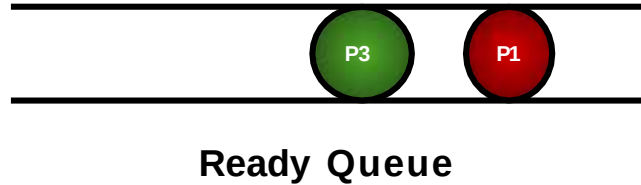


P4 executes for one unit time. The remaining time of P1 is 7 P3 is 9 and P4 is 4 so P4 continue execution

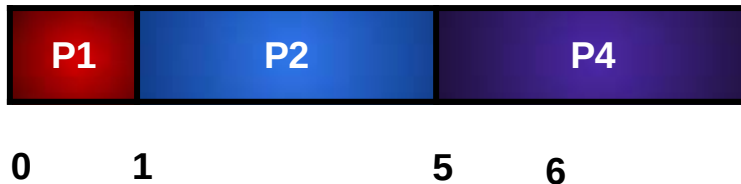


Shortest Remaining Time First (SRTF) - Preemptive SJF

PID	Arrival Time	Burst Time
1	0	8
2	1	4
3	2	9
4	3	5

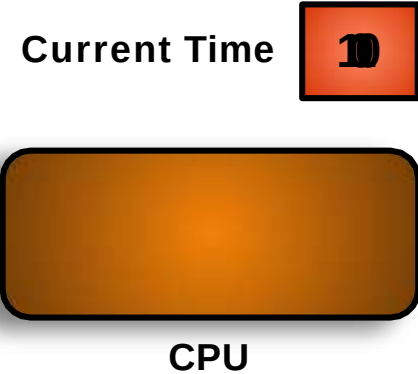
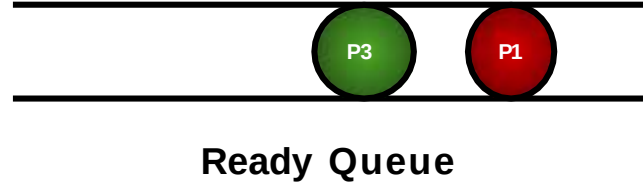


P4 executes for one unit time. The remaining time of P1 is 7 P3 is 9 and P4 is 4 so P4 continue execution till it completes

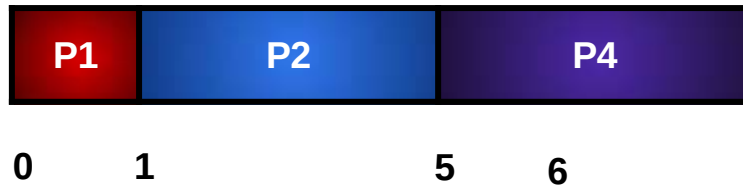


Shortest Remaining Time First (SRTF) - Preemptive SJF

PID	Arrival Time	Burst Time
1	0	8
2	1	4
3	2	9
4	3	5

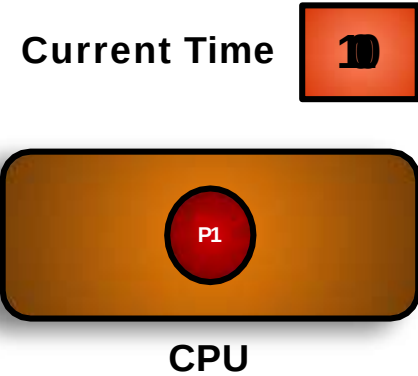
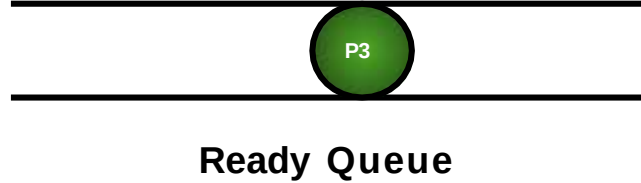


The remaining time of P1 is 7 P3 is 9. So P1 gets scheduled

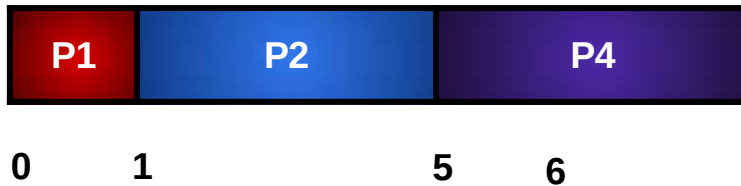


Shortest Remaining Time First (SRTF) - Preemptive SJF

PID	Arrival Time	Burst Time
1	0	8
2	1	4
3	2	9
4	3	5

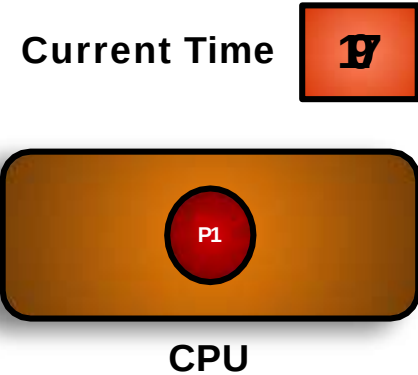
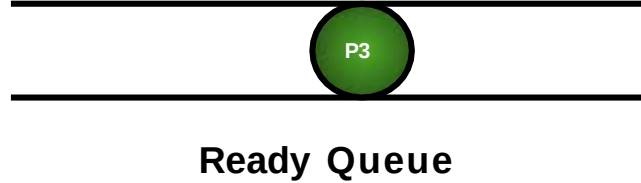


The remaining time of P1 is 7 P3 is 9. So P1 gets scheduled



Shortest Remaining Time First (SRTF) - Preemptive SJF

PID	Arrival Time	Burst Time
1	0	8
2	1	4
3	2	9
4	3	5

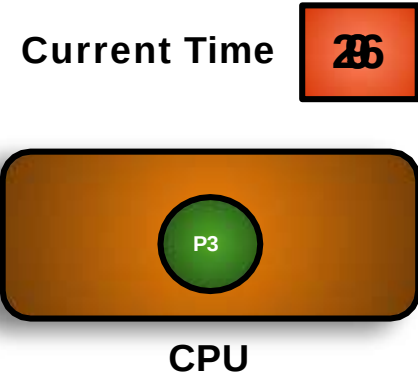
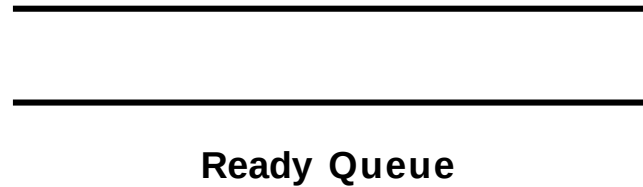


P1 completes execution

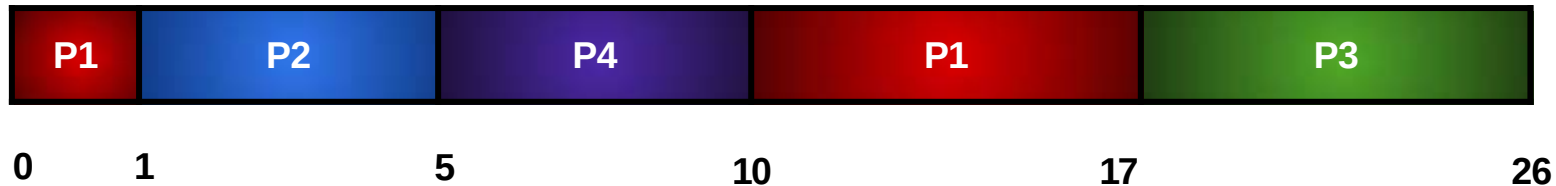


Shortest Remaining Time First (SRTF) - Preemptive SJF

PID	Arrival Time	Burst Time
1	0	8
2	1	4
3	2	9
4	3	5



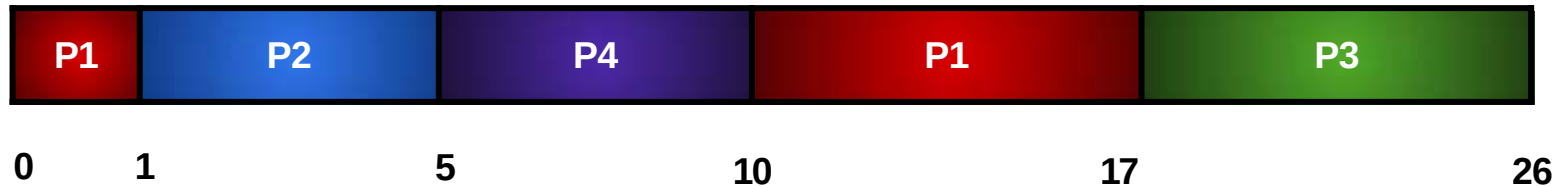
Only P3 in ready queue so it completes the execution



Shortest Remaining Time First (SRTF) - Preemptive SJF

PID	Arrival Time (AT)	Burst Time (BT)	Completion Time (CT)	Turn Around Time (TAT) = CT - AT	Waiting Time (WT) = TAT - BT
1	0	8	17	17	9
2	1	4	5	4	0
3	2	9	26	24	15
4	3	5	10	7	2

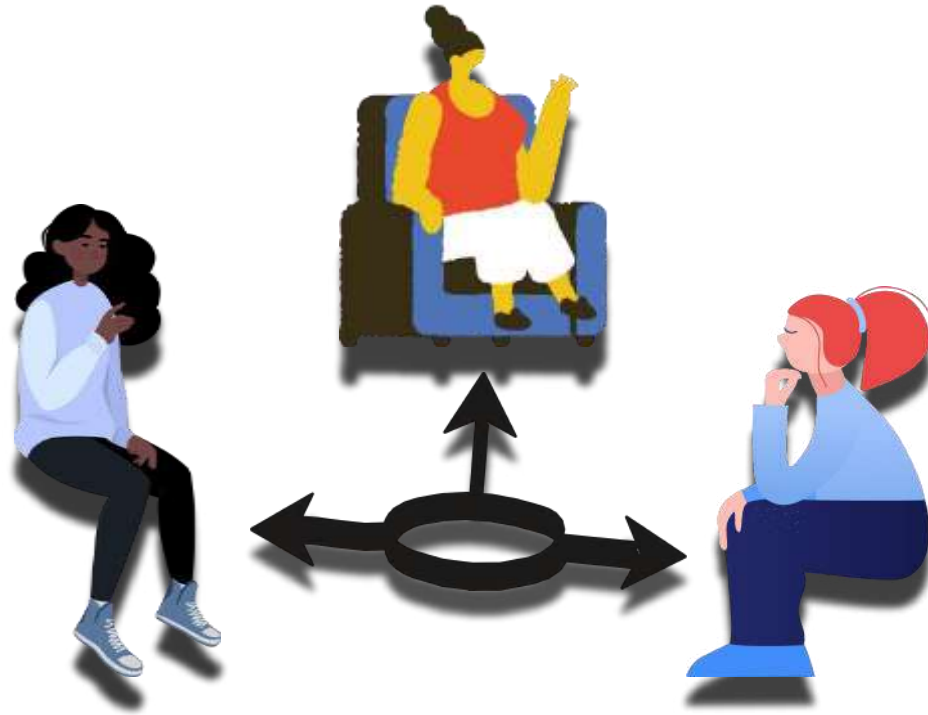
Average waiting time = $(9 + 0 + 15 + 2)/4 = 6.5$



Assignment 3

PID	Arrival Time (AT)	Burst Time (BT)
1	0	8
2	1	4
3	2	2
4	3	1
5	4	3
6	5	2

**Consider above processes with Arrival time and Burst time.
Calculate CT, TAT, WT and average wait time using SRTF algorithm**



Round Robin Scheduling

Round Robin Scheduling

- Each process gets small unit of CPU time (**time quantum q**).
- After time ' q ' elapsed, the **process is preempted** and added to the end of the **ready queue**.
- If there are ' N ' processes in the ready queue and the time quantum is ' q ', then each process gets **$1/N$ of the CPU time** in chunks of at most ' q ' time units at once.
- No process waits for more than **$(N-1)*q$ time units**.
- Timer **interrupts every quantum** to schedule process
- In case a process needs less time than ' q ' then also next process gets scheduled

Round Robin Scheduling

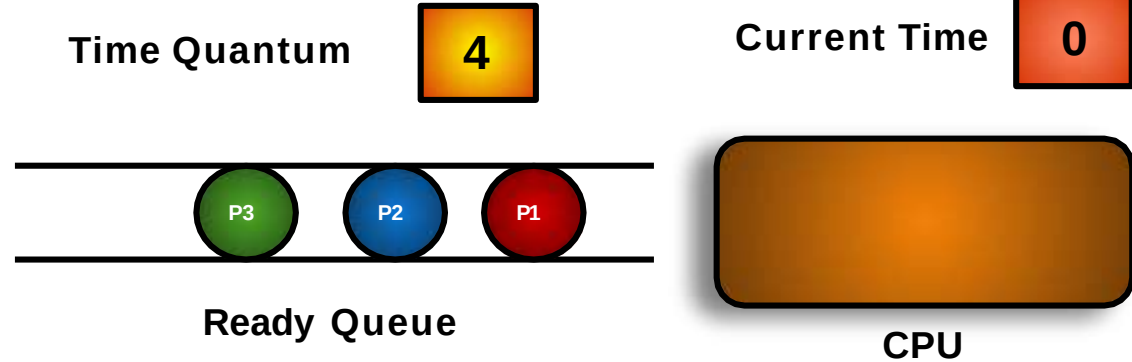
- **Performance:-**

If 'q' is large the RR behaves as FCFS

If 'q' is too small then it increases context switch overheads

Round Robin Scheduling Example

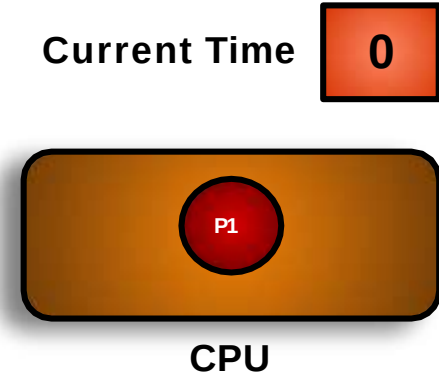
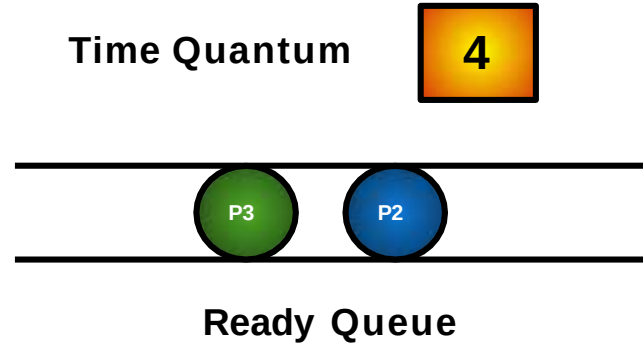
PID	Arrival Time	Burst Time
1	0	24
2	0	3
3	0	3



As arrival time is 0 for all processes, all of them enters into ready queue

Round Robin Scheduling Example

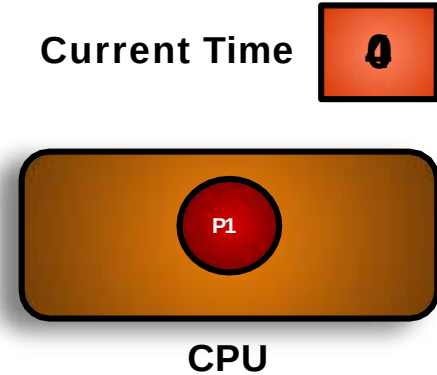
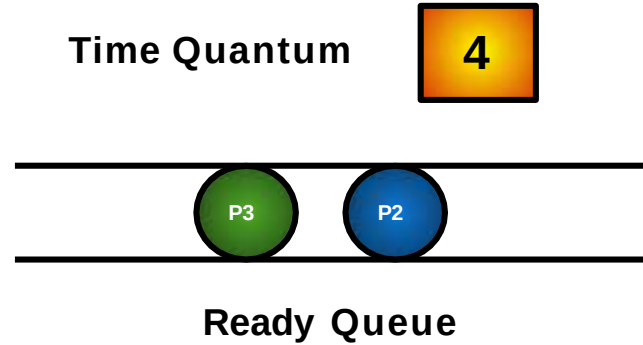
PID	Arrival Time	Burst Time
1	0	24
2	0	3
3	0	3



Let P1 gets scheduled first

Round Robin Scheduling Example

PID	Arrival Time	Burst Time
1	0	24
2	0	3
3	0	3



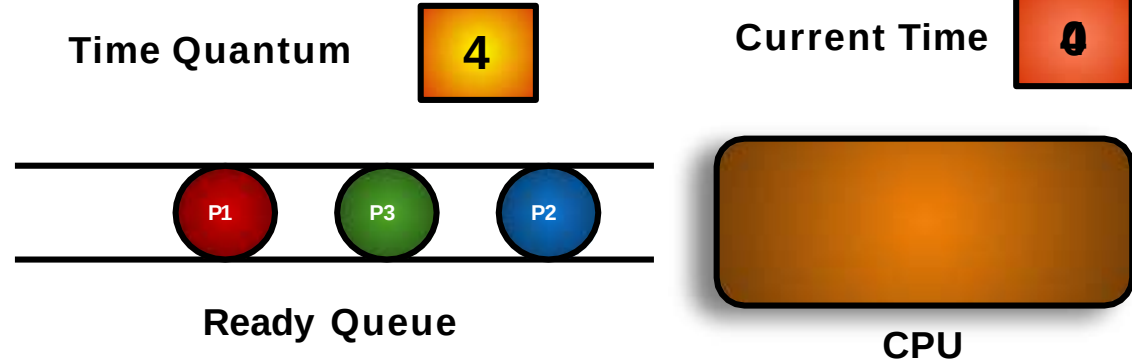
P1 executes for 4 time units as time quanta value is 4



0 4

Round Robin Scheduling Example

PID	Arrival Time	Burst Time
1	0	24
2	0	3
3	0	3



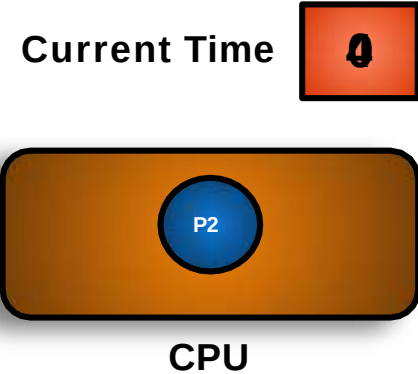
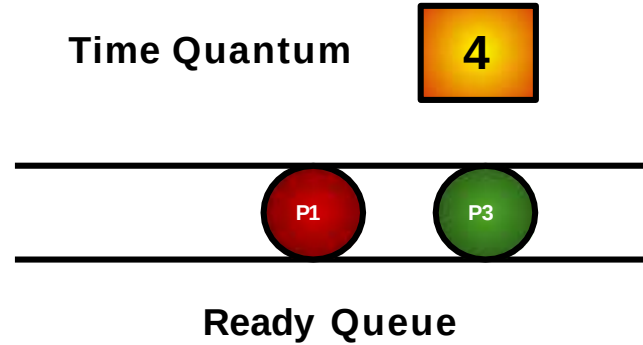
As time quantum for P1 is elapsed, P1 is moved to ready queue and P2 is scheduled next



0 4

Round Robin Scheduling Example

PID	Arrival Time	Burst Time
1	0	24
2	0	3
3	0	3



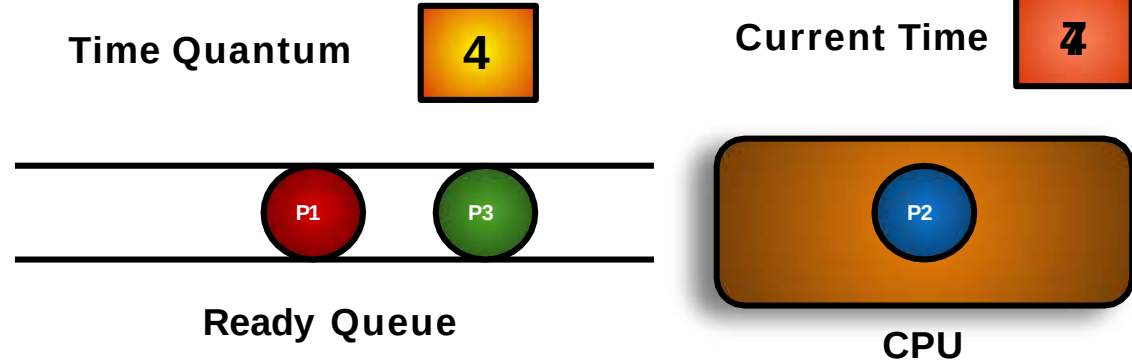
P2 gets access to CPU and starts execution



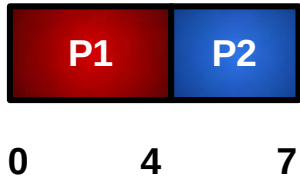
0 4

Round Robin Scheduling Example

PID	Arrival Time	Burst Time
1	0	24
2	0	3
3	0	3

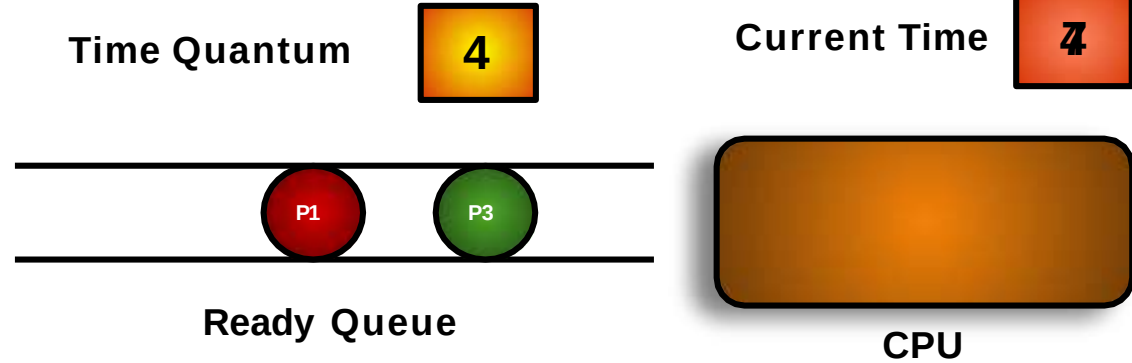


Burst time for P2 is 3 units which is less than Time quantum 4. P2 gets executed for 3 time units and terminates



Round Robin Scheduling Example

PID	Arrival Time	Burst Time
1	0	24
2	0	3
3	0	3



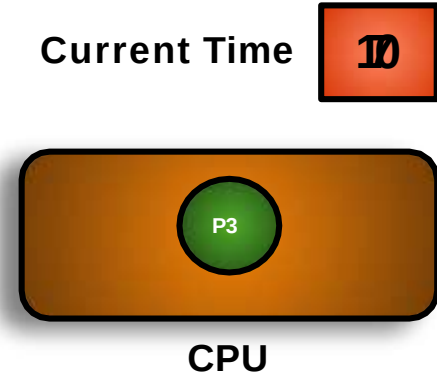
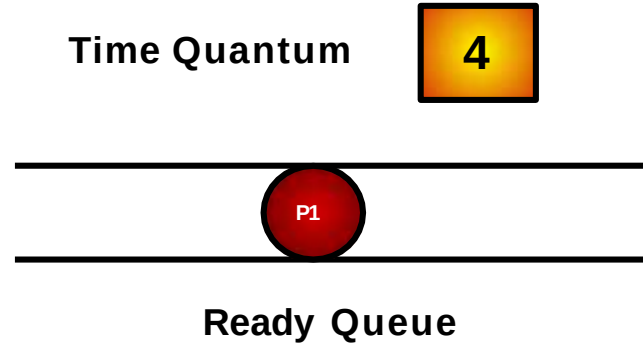
P3 is next in the ready queue and gets chance for execution



0 4 7

Round Robin Scheduling Example

PID	Arrival Time	Burst Time
1	0	24
2	0	3
3	0	3



Burst time for P3 is 3 units which is less than Time quantum 4. P3 gets executed for 3 time units and terminates



0 4 7 10

Round Robin Scheduling Example

PID	Arrival Time	Burst Time
1	0	24
2	0	3
3	0	3

Time Quantum

4

Current Time

10

Ready Queue

CPU

Now only P1 is in ready queue so it starts execution



0 4 7 10

Round Robin Scheduling Example

PID	Arrival Time	Burst Time
1	0	24
2	0	3
3	0	3

Time Quantum

4

Current Time

10

Ready Queue

CPU

P1 has already completed 4 time units, so remaining CPU burst is 20 i.e. greater than time quantum.



0 4 7 10

Round Robin Scheduling Example

PID	Arrival Time	Burst Time
1	0	24
2	0	3
3	0	3

Time Quantum

4

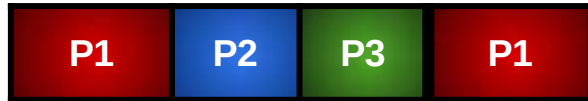
Current Time

10

Ready Queue

CPU

P1 executes for 4 time units



0 4 7 10 14

Round Robin Scheduling Example

PID	Arrival Time	Burst Time
1	0	24
2	0	3
3	0	3

Time Quantum

4

Current Time

20

Ready Queue

CPU

There is no other process in ready queue so P1 completes execution by utilizing all time quanta.



0 4 7 10 14

Round Robin Scheduling Example

PID	Arrival Time	Burst Time
1	0	24
2	0	3
3	0	3

Time Quantum

4

Current Time

20

Ready Queue

CPU

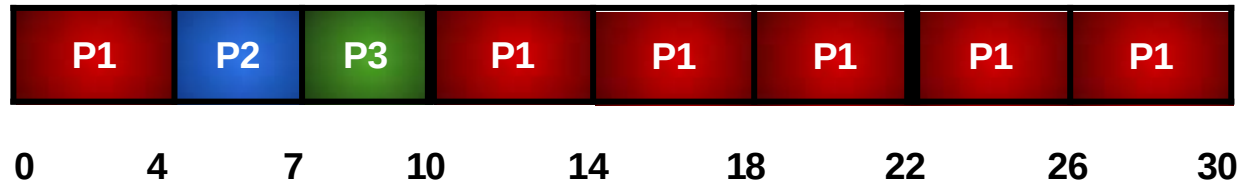
Finally P1 terminates



0 4 7 10 14

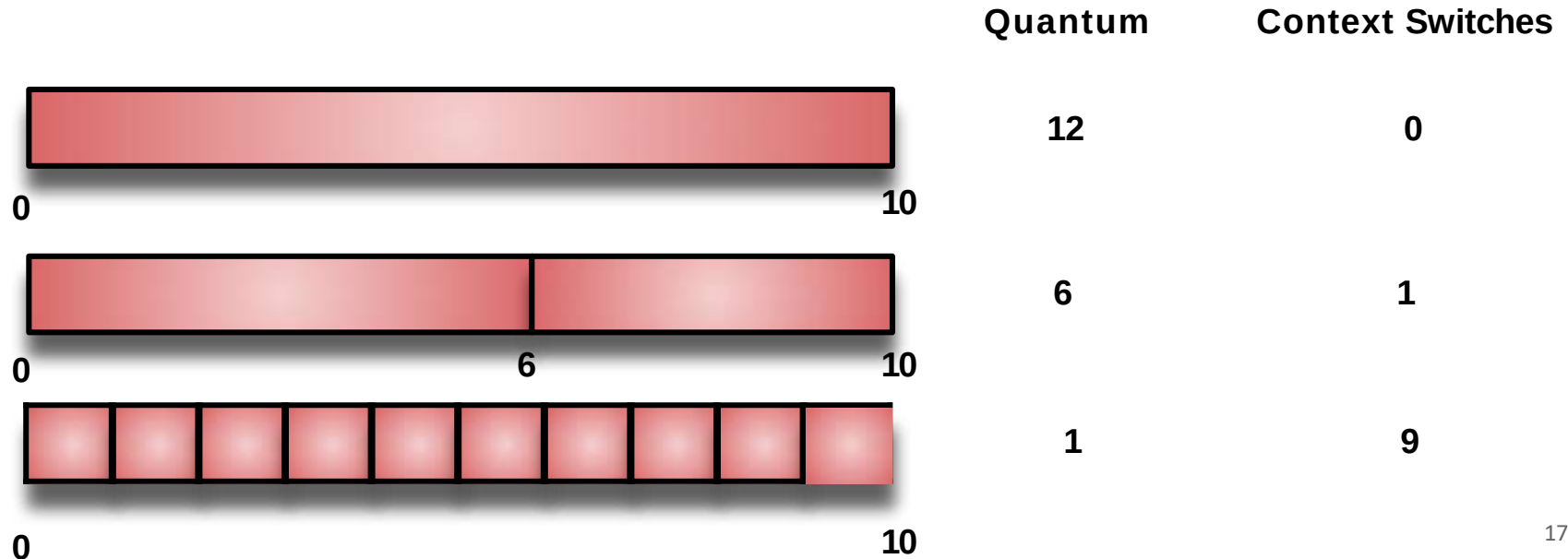
Round Robin Scheduling Example

PID	Arrival Time (AT)	Burst Time (BT)	Completion Time (CT)	Turn Around Time (TAT) = CT - AT	Waiting Time (WT) = TAT - BT
1	0	24	30	30	6
2	0	3	7	7	4
3	0	3	10	10	7



Time Quantum and Context Switch Time

- The performance of RR **depends on the size of the time quantum**.
- If the time quantum is very small RR results in a **large number of context switches**



Advantages of Round Robin Scheduling

1. Every process gets an **equal share** of the CPU.
2. RR is cyclic in nature, so there is **no starvation**.
3. No issues of starvation or convoy effect.
4. **Doesn't depend on burst time** and is easily implementable.
5. Best performance in terms of **average response time**.

Disadvantages of Round Robin Scheduling

1. Setting the quantum too short, **increases the overhead and lowers the CPU efficiency.**
2. Setting the quantum too long may cause poor response to short processes.
3. Average waiting time under the RR policy is often long.
4. Spends **more time on context switching.**
5. Performance depends on time quantum.
6. **Higher context switching overhead** due to lower time quantum.
7. **Difficult to find** a correct time quantum.

Assignment No. 4

PID	Arrival Time (AT)	Burst Time (BT)	Completion Time (CT)	Turn Around Time (TAT) = CT - AT	Waiting Time (WT) = TAT - BT
1	0	5			
2	1	6			
3	2	3			
4	3	1			
5	4	5			
6	6	4			

Apply round robin scheduling algorithm with time quantum = 2, for above processes and calculate, TAT, CT, WT and average WT.

Note:- In this example arrival time also mentioned



Priority scheduling

Priority Scheduling

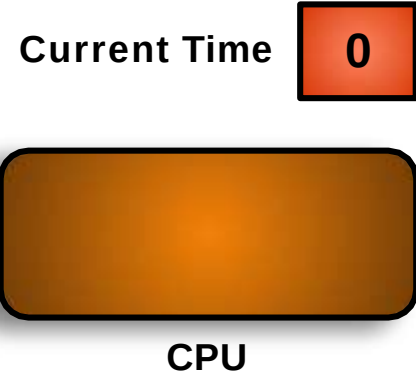
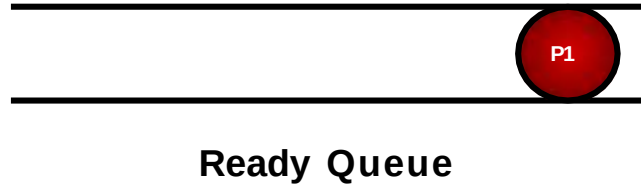
- A priority number (integer) is associated with every process
- The CPU is allocated to the process with the highest priority
- If two processes with same priority the use FCFS to break the tie or use round robin algorithm to allocate CPU
- Smallest integer number indicates the highest priority
- Priority scheduling can be preemptive or non preemptive
- SJF is priority scheduling where, priority is inverse of predicted next CPU burst

Problem:- Starvation - Low priority process may never get executed

Solution:- Aging - As time progresses increase the priority of processes waiting in ready queue

Priority Scheduling - Non preemptive Example

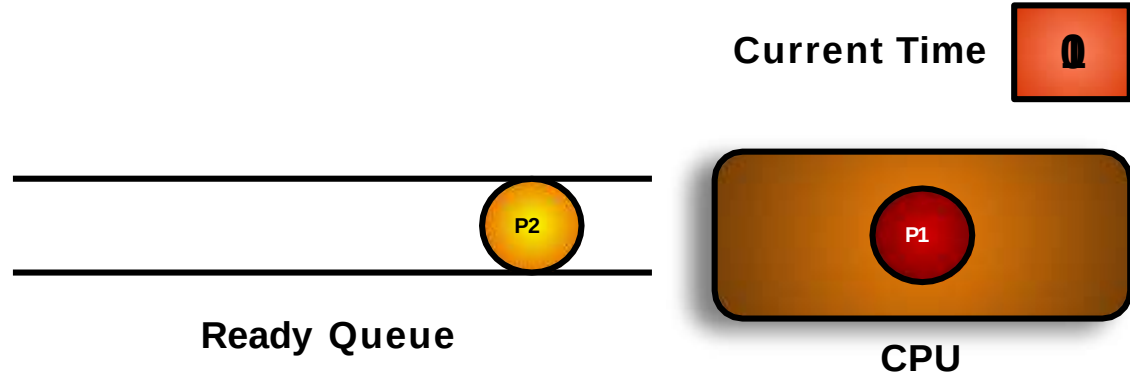
PID	Arrival Time	Burst Time	Priority
1	0	4	1
2	1	5	0
3	2	3	2
4	3	2	4
5	5	2	1



At time 0 only process P1 arrives in ready queue.

Priority Scheduling - Non preemptive Example

PID	Arrival Time	Burst Time	Priority
1	0	4	1
2	1	5	0
3	2	3	2
4	3	2	4
5	5	2	1



P1 gets scheduled on CPU

P1 executes for 1 time unit

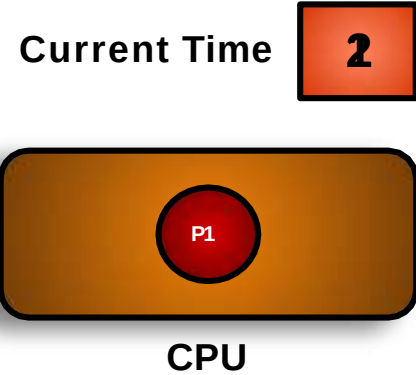
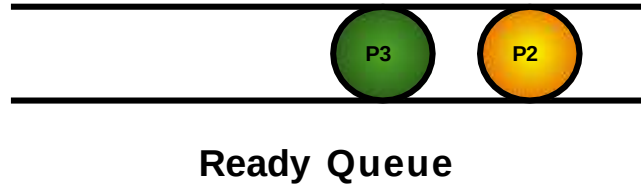
P2 arrives in ready queue at time 1



0 1

Priority Scheduling - Non preemptive Example

PID	Arrival Time	Burst Time	Priority
1	0	4	1
2	1	5	0
3	2	3	2
4	3	2	4
5	5	2	1



As this is non preemptive algorithm, P1 continues to execute

P3 arrives at time 2



0 1 2

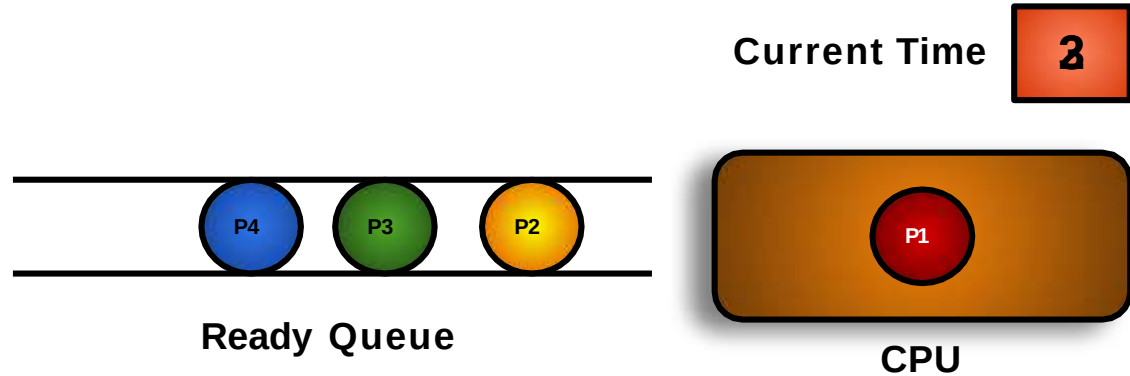
Priority Scheduling - Non preemptive Example

PID	Arrival Time	Burst Time	Priority
1	0	4	1
2	1	5	0
3	2	3	2
4	3	2	4
5	5	2	1

P1 continues to execute



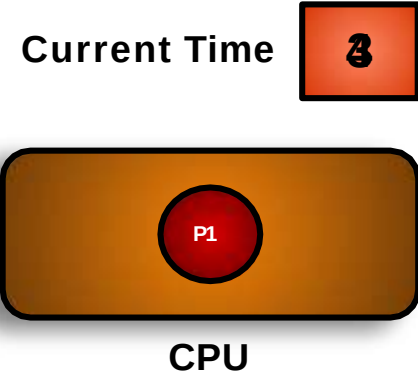
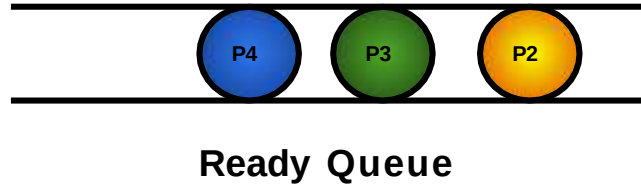
0 1 2 3



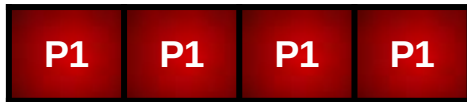
P4 arrives at time 3

Priority Scheduling - Non preemptive Example

PID	Arrival Time	Burst Time	Priority
1	0	4	1
2	1	5	0
3	2	3	2
4	3	2	4
5	5	2	1



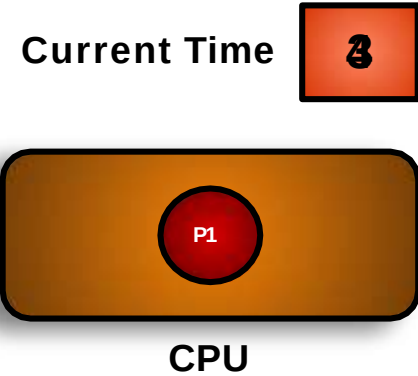
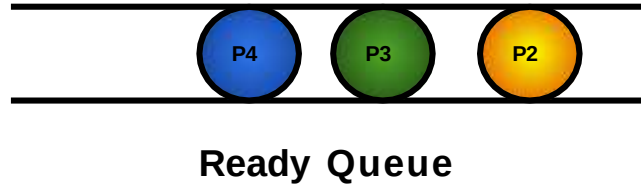
P1 continues to execute



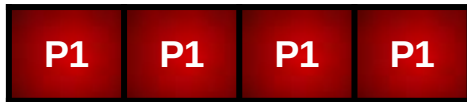
0 1 2 3 4

Priority Scheduling - Non preemptive Example

PID	Arrival Time	Burst Time	Priority
1	0	4	1
2	1	5	0
3	2	3	2
4	3	2	4
5	5	2	1



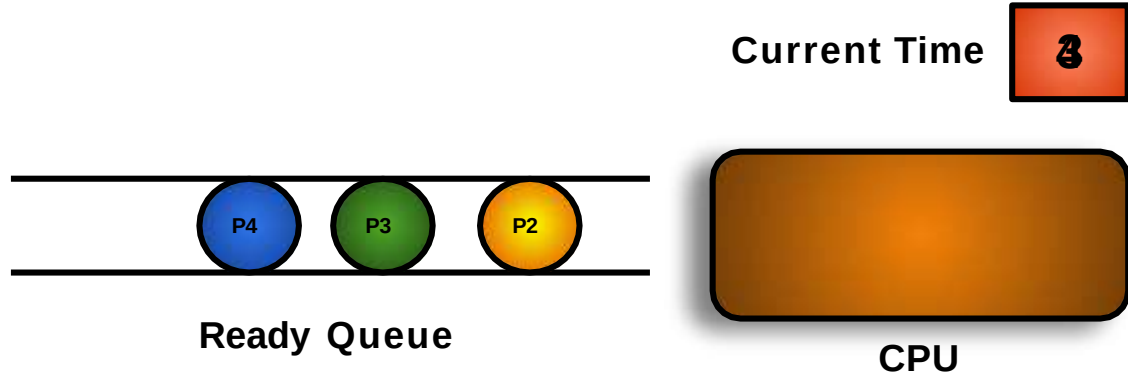
P1 Completes execution



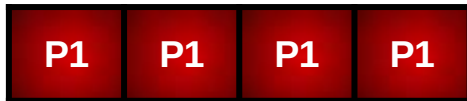
0 1 2 3 4

Priority Scheduling - Non preemptive Example

PID	Arrival Time	Burst Time	Priority
1	0	4	1
2	1	5	0
3	2	3	2
4	3	2	4
5	5	2	1



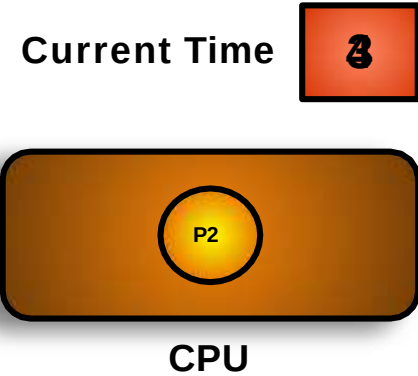
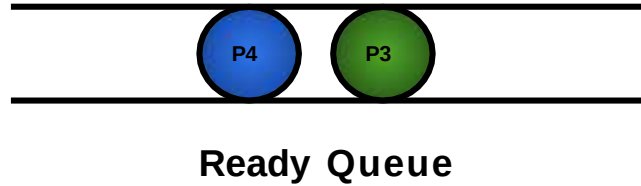
There are 3 processes in ready queue out of which P2 has highest priority so P2 gets executed



0 1 2 3 4

Priority Scheduling - Non preemptive Example

PID	Arrival Time	Burst Time	Priority
1	0	4	1
2	1	5	0
3	2	3	2
4	3	2	4
5	5	2	1

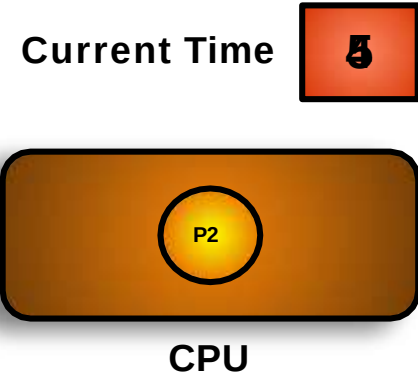
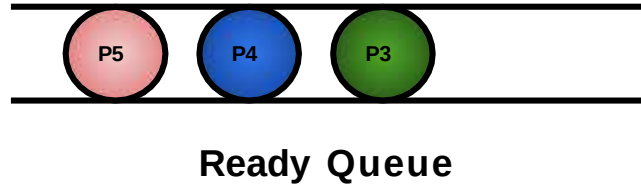


There are 3 processes in ready queue out of which P2 has highest priority so P2 gets executed



Priority Scheduling - Non preemptive Example

PID	Arrival Time	Burst Time	Priority
1	0	4	1
2	1	5	0
3	2	3	2
4	3	2	4
5	5	2	1



P2 executes for 1 time unit

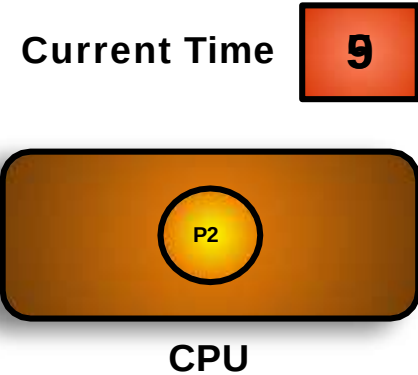
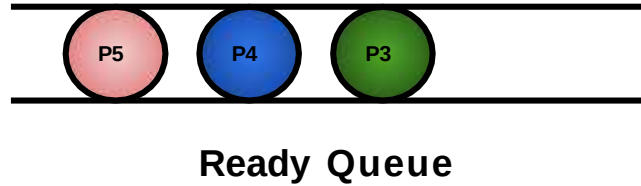
P5 arrives in ready queue



0 1 2 3 4 5

Priority Scheduling - Non preemptive Example

PID	Arrival Time	Burst Time	Priority
1	0	4	1
2	1	5	0
3	2	3	2
4	3	2	4
5	5	2	1



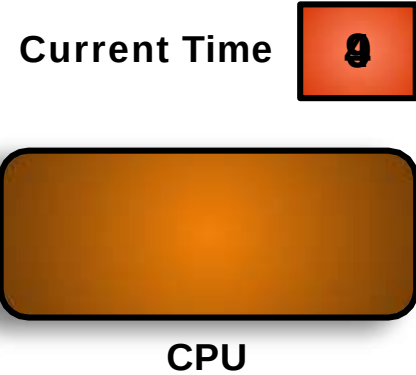
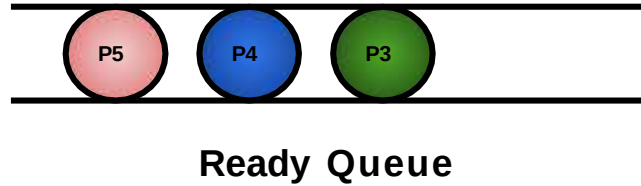
As this is non preemptive algorithm P2 gets executed till it terminates



0 1 2 3 4 5 9

Priority Scheduling - Non preemptive Example

PID	Arrival Time	Burst Time	Priority
1	0	4	1
2	1	5	0
3	2	3	2
4	3	2	4
5	5	2	1



P3, P4, P5 in ready queue, where P5 has highest priority

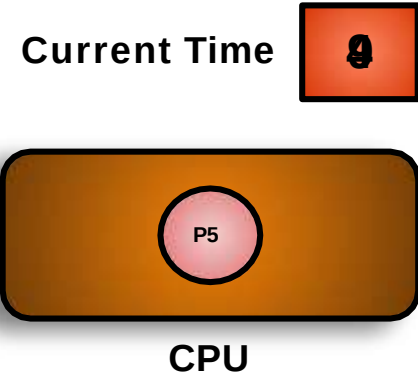
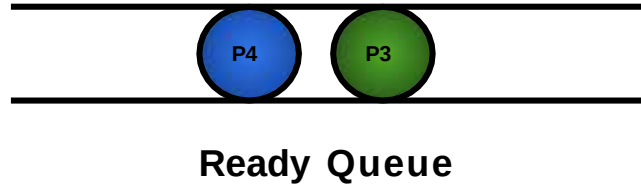
P5 scheduled



0 1 2 3 4 9

Priority Scheduling - Non preemptive Example

PID	Arrival Time	Burst Time	Priority
1	0	4	1
2	1	5	0
3	2	3	2
4	3	2	4
5	5	2	1



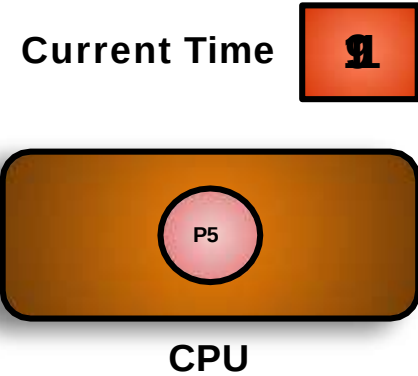
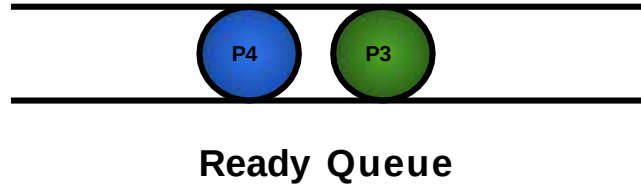
P3, P4, P5 in ready queue, where P5 has highest priority P5 scheduled



0 1 2 3 4 9

Priority Scheduling - Non preemptive Example

PID	Arrival Time	Burst Time	Priority
1	0	4	1
2	1	5	0
3	2	3	2
4	3	2	4
5	5	2	1



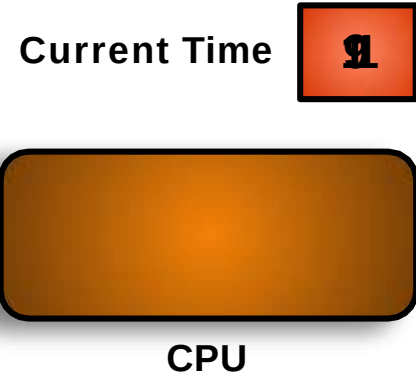
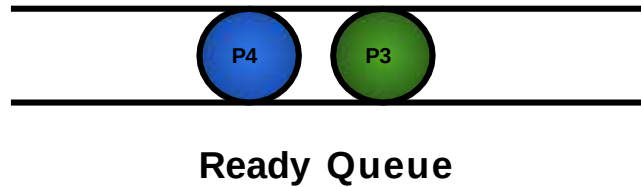
P5 completes execution

P5 Terminated



Priority Scheduling - Non preemptive Example

PID	Arrival Time	Burst Time	Priority
1	0	4	1
2	1	5	0
3	2	3	2
4	3	2	4
5	5	2	1

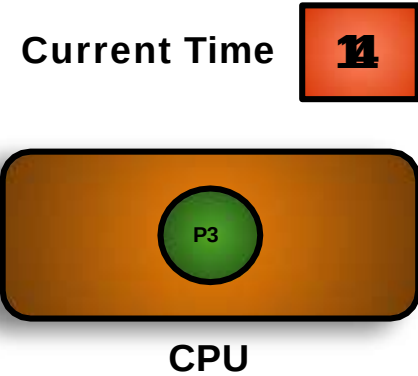
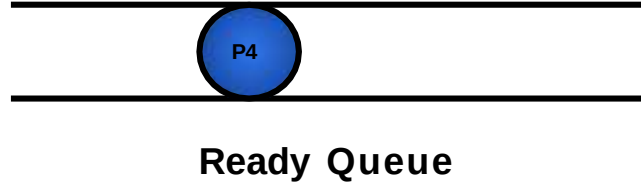


P3 and P4 in ready queue, where p3 has highest priority



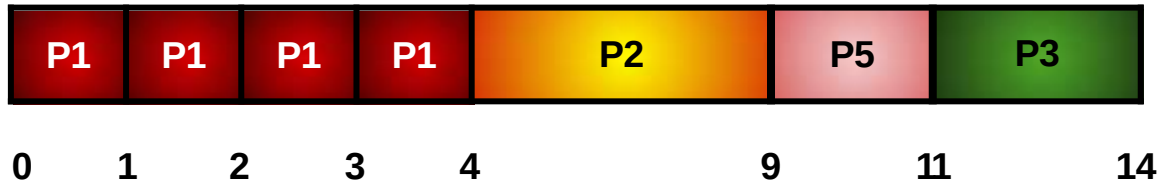
Priority Scheduling - Non preemptive Example

PID	Arrival Time	Burst Time	Priority
1	0	4	1
2	1	5	0
3	2	3	2
4	3	2	4
5	5	2	1



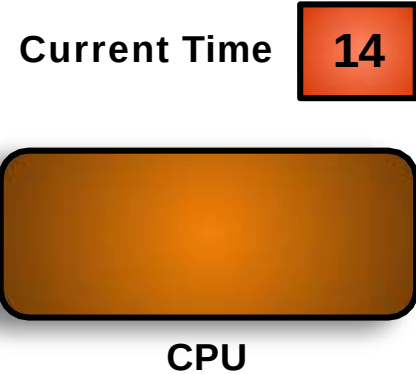
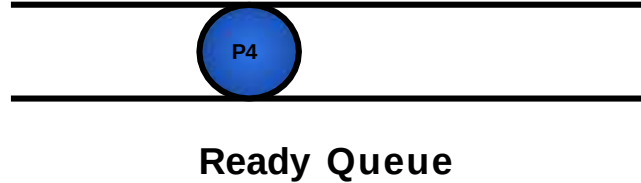
P3 scheduled for execution

P3 completes execution

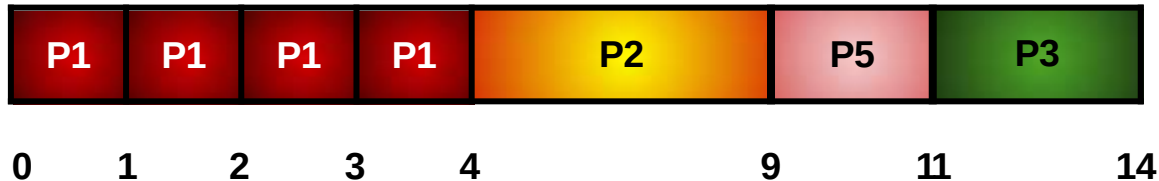


Priority Scheduling - Non preemptive Example

PID	Arrival Time	Burst Time	Priority
1	0	4	1
2	1	5	0
3	2	3	2
4	3	2	4
5	5	2	1

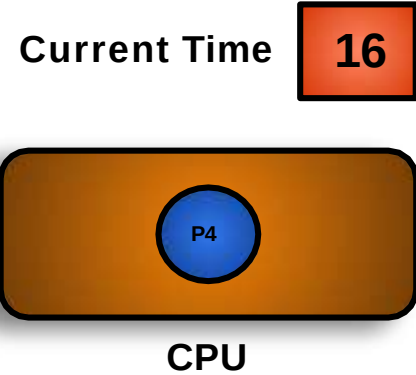
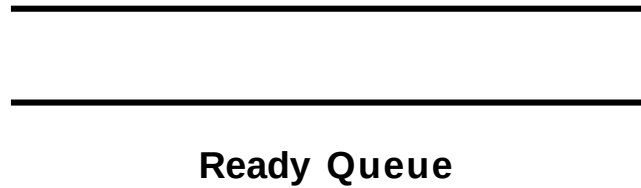


Only P4 left in ready queue.



Priority Scheduling - Non preemptive Example

PID	Arrival Time	Burst Time	Priority
1	0	4	1
2	1	5	0
3	2	3	2
4	3	2	4
5	5	2	1



P4 gets scheduled for execution.

P4 completes execution.

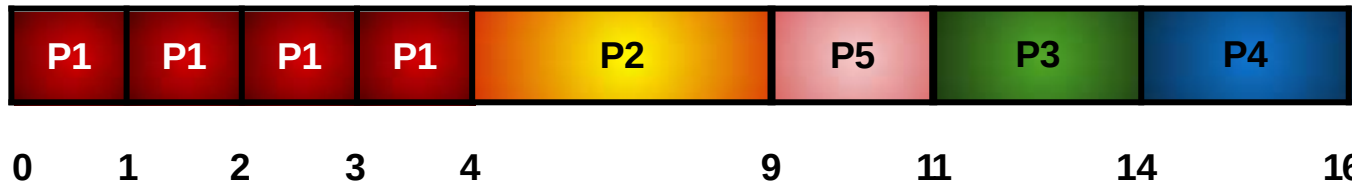
P4 terminates



Priority Scheduling - Non preemptive Example

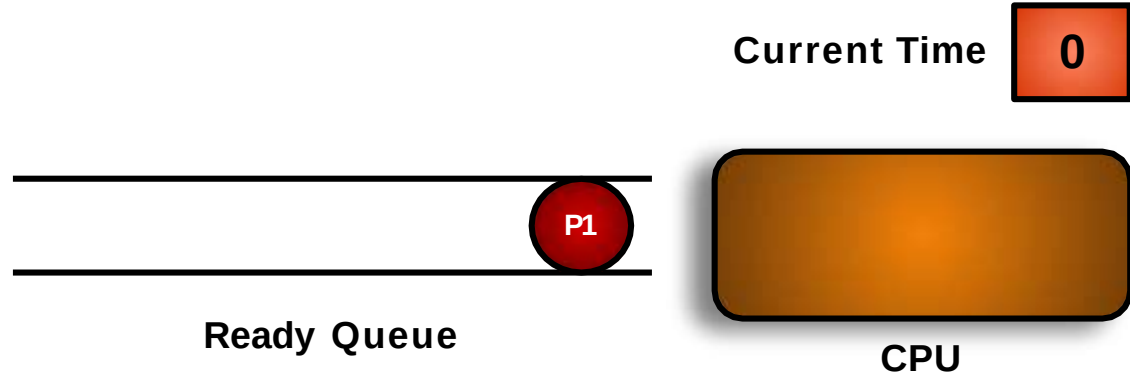
PID	Arrival Time (AT)	Burst Time (BT)	Priority	Completion Time (CT)	Turn Around Time (TAT) = CT-AT	Waiting Time = TAT - BT
1	0	4	1	4	4	0
2	1	5	0	9	8	3
3	2	3	2	14	12	9
4	3	2	4	16	13	11
5	5	2	1	11	6	4

Average waiting time = $(0 + 3 + 9 + 11 + 4)/5 = 27/5 = 5.4$



Priority Scheduling - preemptive Example

PID	Arrival Time	Burst Time	Priority
1	0	1	2(H)
2	1	7	6
3	2	3	3
4	3	6	5
5	4	5	4
6	5	15	10(L)
7	15	8	9

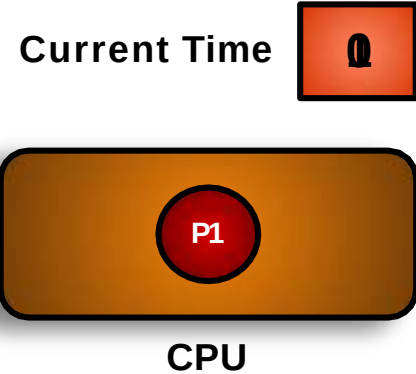
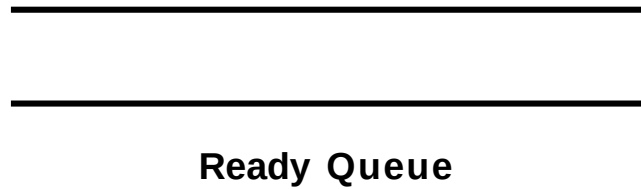


At time 0 P1 arrives in ready queue

As no other process in ready queue, P1 gets scheduled.

Priority Scheduling - preemptive Example

PID	Arrival Time	Burst Time	Priority
1	0	1	2(H)
2	1	7	6
3	2	3	3
4	3	6	5
5	4	5	4
6	5	15	10(L)
7	15	8	9



P1 completes execution for 1 time unit

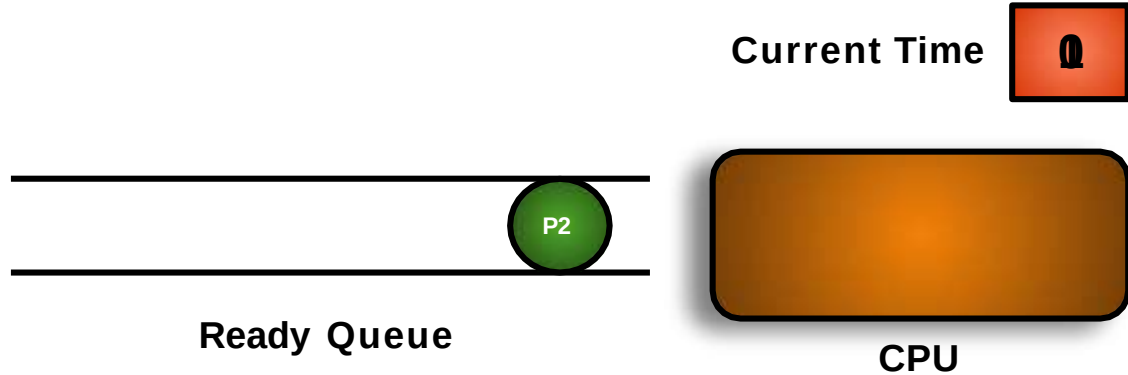
P1 completed execution and terminates



0 1

Priority Scheduling - preemptive Example

PID	Arrival Time	Burst Time	Priority
1	0	1	2(H)
2	1	7	6
3	2	3	3
4	3	6	5
5	4	5	4
6	5	15	10(L)
7	15	8	9



At time 1 P2 arrives in ready queue.

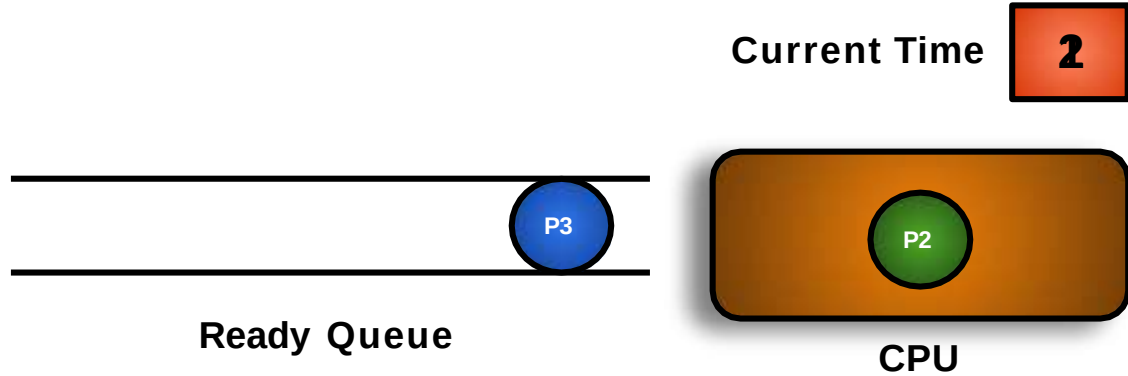
There no process executing on CPU so P2 starts execution



0 1

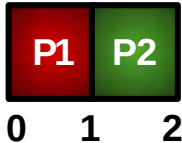
Priority Scheduling - preemptive Example

PID	Arrival Time	Burst Time	Priority
1	0	1	2(H)
2	1	7	6
3	2	3	3
4	3	6	5
5	4	5	4
6	5	15	10(L)
7	15	8	9



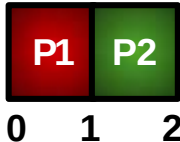
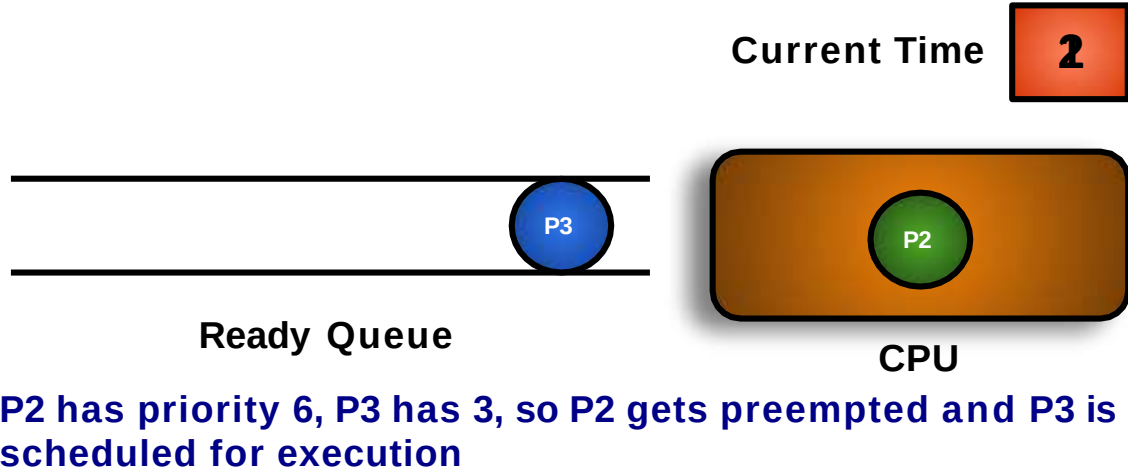
P2 continue execution for 1time unit

At time 2 P3 arrives in ready queue



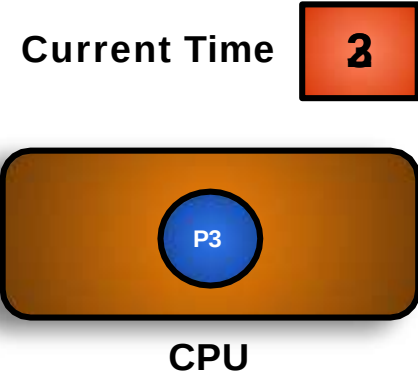
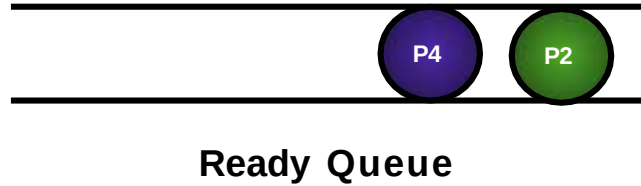
Priority Scheduling - preemptive Example

PID	Arrival Time	Burst Time	Priority
1	0	1	2(H)
2	1	7	6
3	2	3	3
4	3	6	5
5	4	5	4
6	5	15	10(L)
7	15	8	9



Priority Scheduling - preemptive Example

PID	Arrival Time	Burst Time	Priority
1	0	1	2(H)
2	1	7	6
3	2	3	3
4	3	6	5
5	4	5	4
6	5	15	10(L)
7	15	8	9



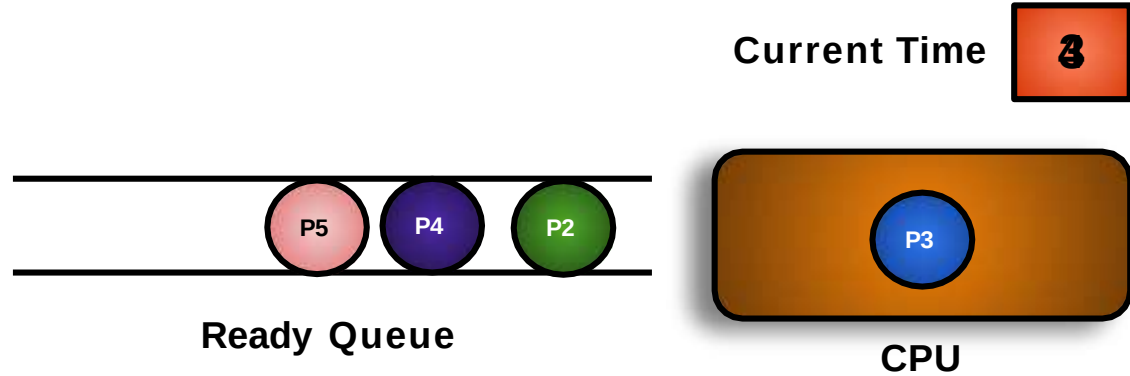
P3 completes execution for 1 unit

P4 arrives in ready queue at time 3



Priority Scheduling - preemptive Example

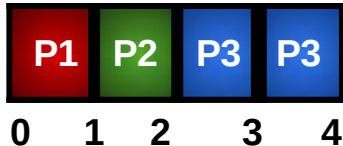
PID	Arrival Time	Burst Time	Priority
1	0	1	2(H)
2	1	7	6
3	2	3	3
4	3	6	5
5	4	5	4
6	5	15	10(L)
7	15	8	9



P3 has higher priority than P4 and P2, so P3 continues the execution

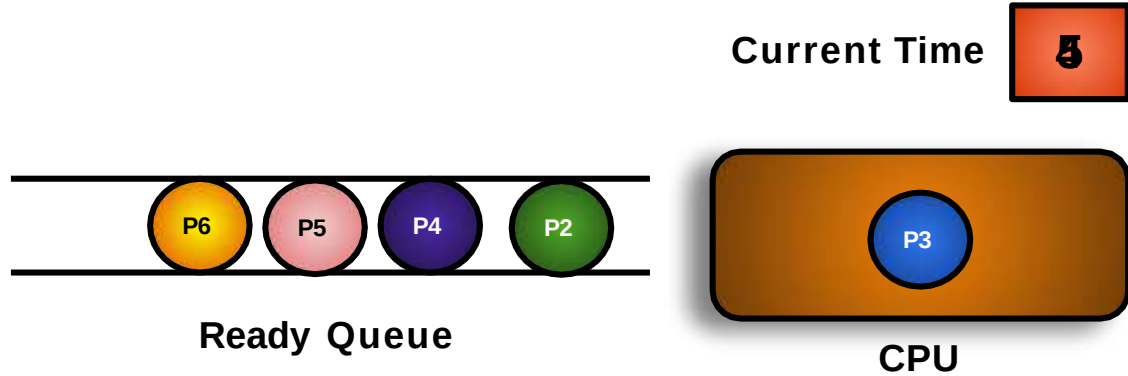
P3 executes for 1 time unit

At time 4, P5 arrives in ready queue



Priority Scheduling - preemptive Example

PID	Arrival Time	Burst Time	Priority
1	0	1	2(H)
2	1	7	6
3	2	3	3
4	3	6	5
5	4	5	4
6	5	15	10(L)
7	15	8	9



Again P3 has higher priority than all the processes in ready queue, so continues the execution

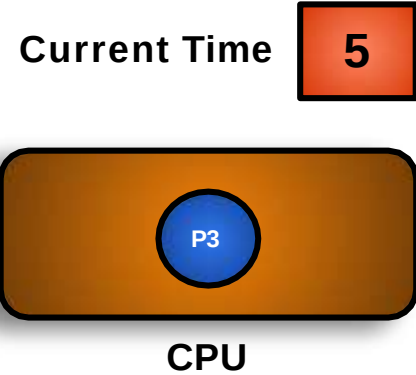
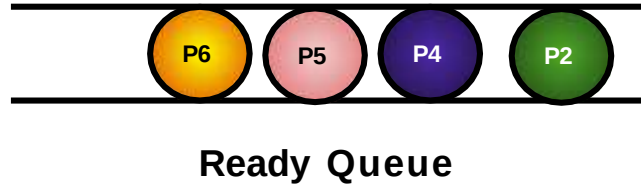
P3 continue execution for 1 time unit

At time 5 P6 arrives in ready



Priority Scheduling - preemptive Example

PID	Arrival Time	Burst Time	Priority
1	0	1	2(H)
2	1	7	6
3	2	3	3
4	3	6	5
5	4	5	4
6	5	15	10(L)
7	15	8	9

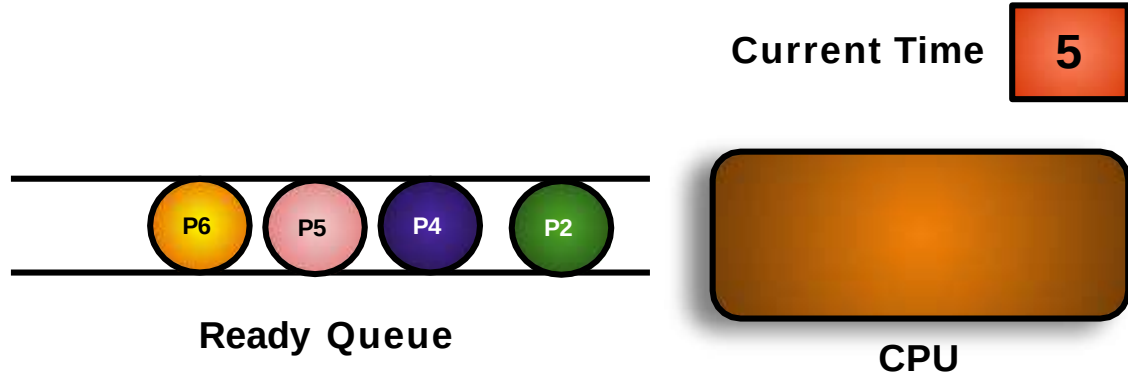


Burst time of P3 is over, so P3 terminates



Priority Scheduling - preemptive Example

PID	Arrival Time	Burst Time	Priority
1	0	1	2(H)
2	1	7	6
3	2	3	3
4	3	6	5
5	4	5	4
6	5	15	10(L)
7	15	8	9

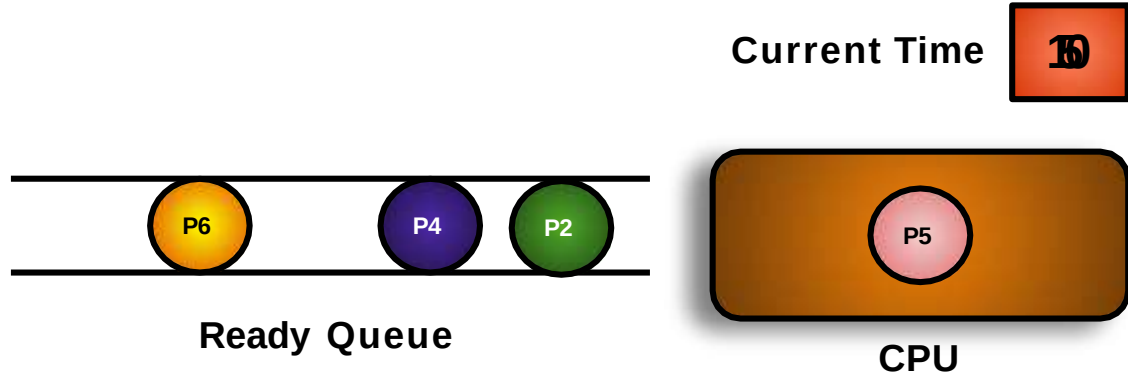


There are 4 processes in ready queue P2, P4, P5, P6 out of which P5 has highest priority so P5 gets scheduled for execution



Priority Scheduling - preemptive Example

PID	Arrival Time	Burst Time	Priority
1	0	1	2(H)
2	1	7	6
3	2	3	3
4	3	6	5
5	4	5	4
6	5	15	10(L)
7	15	8	9



As there are no new process arrives in ready queue while execution of P5, P5 continues to execute till it terminates
P5 terminates after completion of burst time



Priority Scheduling - preemptive Example

PID	Arrival Time	Burst Time	Priority
1	0	1	2(H)
2	1	7	6
3	2	3	3
4	3	6	5
5	4	5	4
6	5	15	10(L)
7	15	8	9



Ready Queue

Now, P2, P4 and P6 are present in ready queue out of which P4 has highest priority, so P4 gets scheduled to execute

Current Time



CPU



Priority Scheduling - preemptive Example

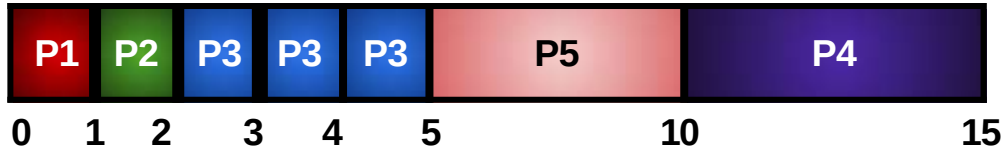
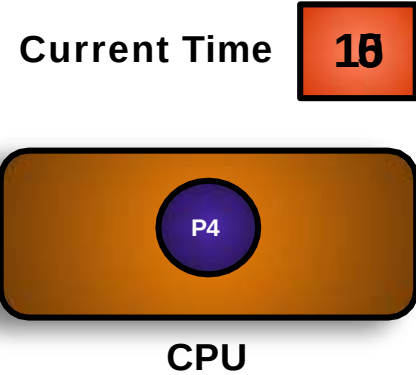
PID	Arrival Time	Burst Time	Priority
1	0	1	2(H)
2	1	7	6
3	2	3	3
4	3	6	5
5	4	5	4
6	5	15	10(L)
7	15	8	9



Ready Queue

P4 executes till time 15

At time = 15, P7 arrives in ready queue



Priority Scheduling - preemptive Example

PID	Arrival Time	Burst Time	Priority
1	0	1	2(H)
2	1	7	6
3	2	3	3
4	3	6	5
5	4	5	4
6	5	15	10(L)
7	15	8	9



Ready Queue

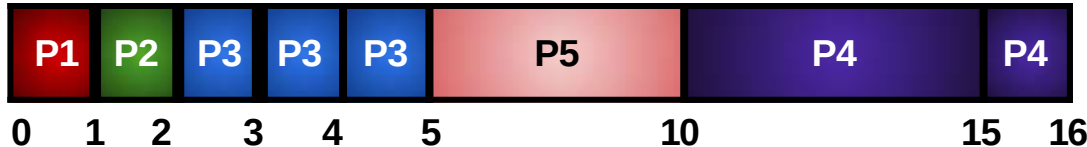
Current Time **16**



CPU

As P4 has higher priority than P2, P6 and P7 it continues to execute

Burst time of P4 is over so it terminates



Priority Scheduling - preemptive Example

PID	Arrival Time	Burst Time	Priority
1	0	1	2(H)
2	1	7	6
3	2	3	3
4	3	6	5
5	4	5	4
6	5	15	10(L)
7	15	8	9



Ready Queue

Current Time

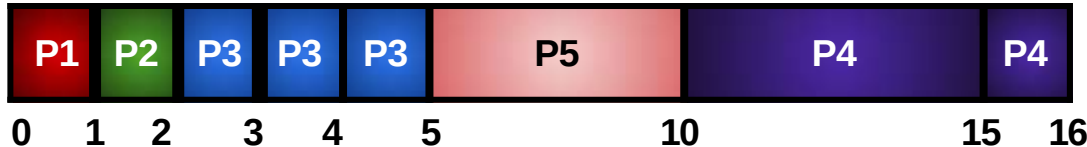
16



CPU

Three processes in ready queue, P7, P6, and P2 out of which P2 has highest priority

P2 gets scheduled for execution



Priority Scheduling - preemptive Example

PID	Arrival Time	Burst Time	Priority
1	0	1	2(H)
2	1	7	6
3	2	3	3
4	3	6	5
5	4	5	4
6	5	15	10(L)
7	15	8	9



Ready Queue

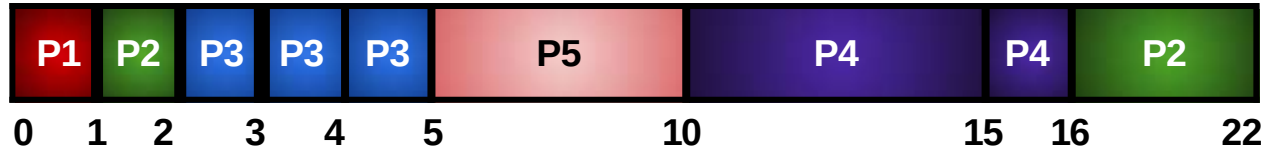
Current Time

20



CPU

As no new process arrives during execution of process P2, it executes till completion



Priority Scheduling - preemptive Example

PID	Arrival Time	Burst Time	Priority
1	0	1	2(H)
2	1	7	6
3	2	3	3
4	3	6	5
5	4	5	4
6	5	15	10(L)
7	15	8	9



Ready Queue

Current Time

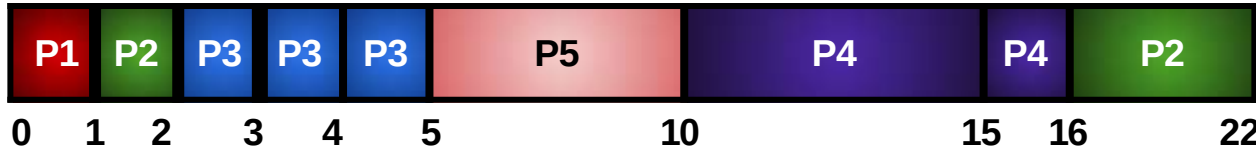
20



CPU

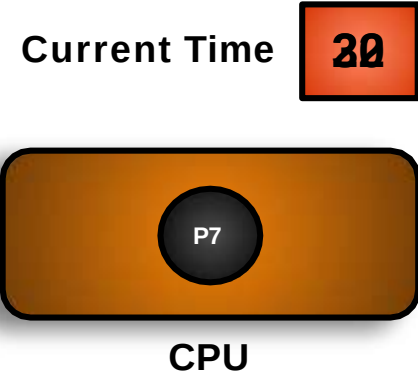
Two processes in ready queue, P6 and P7 out of which P7 has higher priority

P7 gets scheduled for execution



Priority Scheduling - preemptive Example

PID	Arrival Time	Burst Time	Priority
1	0	1	2(H)
2	1	7	6
3	2	3	3
4	3	6	5
5	4	5	4
6	5	15	10(L)
7	15	8	9



As no process arrives during execution of P7, P7 completes execution without preemption



Priority Scheduling - preemptive Example

PID	Arrival Time	Burst Time	Priority
1	0	1	2(H)
2	1	7	6
3	2	3	3
4	3	6	5
5	4	5	4
6	5	15	10(L)
7	15	8	9



Ready Queue

Only P6 is in ready queue.

P6 gets scheduled on CPU

Current Time

20

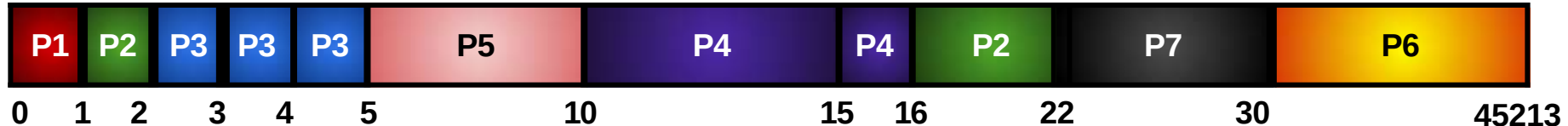
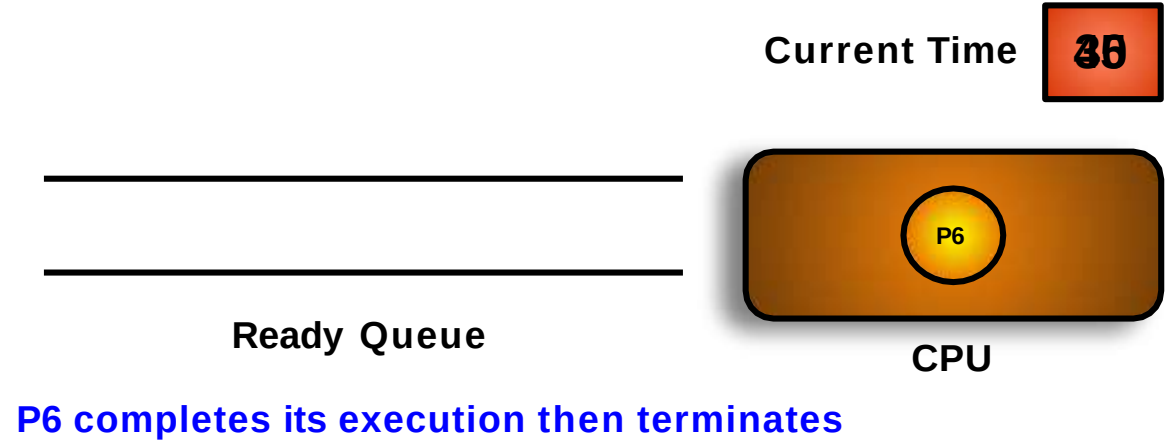


CPU



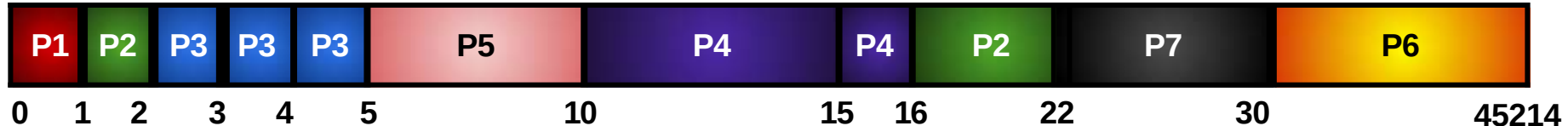
Priority Scheduling - preemptive Example

PID	Arrival Time	Burst Time	Priority
1	0	1	2(H)
2	1	7	6
3	2	3	3
4	3	6	5
5	4	5	4
6	5	15	10(L)
7	15	8	9



Priority Scheduling - preemptive Example

PID	Arrival Time	Burst Time	Priority	Completion Time (CT)	Turn Around Time (TAT) = CT-AT	Waiting Time = TAT - BT
1	0	1	2(H)	1	1	0
2	1	7	6	22	21	14
3	2	3	3	5	3	0
4	3	6	5	16	13	7
5	4	5	4	10	6	1
6	5	15	10(L)	45	40	25
7	15	8	9	30	15	7



Advantages of Priority Scheduling

- This provides a good mechanism where the relative importance of each process may be precisely defined
- It is easy to use.
- It is a user friendly algorithm.
- Simple to understand.

Disadvantages of Priority Scheduling

- If high priority processes use up a lot of CPU time, lower priority processes may starve and be postponed indefinitely (**starvation**).
- The problem of starvation can be solved by **aging**.
- Aging is a technique in which the system **gradually increases the priority of those processes** which are waiting in the system from a long time for their execution.
- Another problem is deciding which process gets which priority level assigned to it.

Assignment No. 5

PID	Arrival Time (AT)	Burst Time (BT)	Priority
1	0	4	2
2	1	3	3
3	3	1	4
4	4	5	5
5	5	2	5

Consider the set of 5 processes whose arrival time and burst time are given above. If the CPU scheduling policy is **priority preemptive**, calculate the average waiting time and average turnaround time. **(Higher number represents higher priority)**

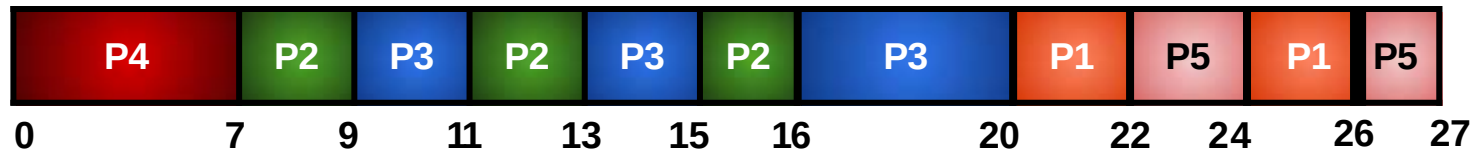
Assignment No. 6

PID	Arrival Time (AT)	Burst Time (BT)	Priority
1	0	4	2
2	1	3	3
3	3	1	4
4	4	5	5
5	5	2	5

Consider the set of 5 processes whose arrival time and burst time are given above. If the CPU scheduling policy is **priority Non-preemptive**, calculate the average waiting time and average turnaround time. **(Higher number represents higher priority)**

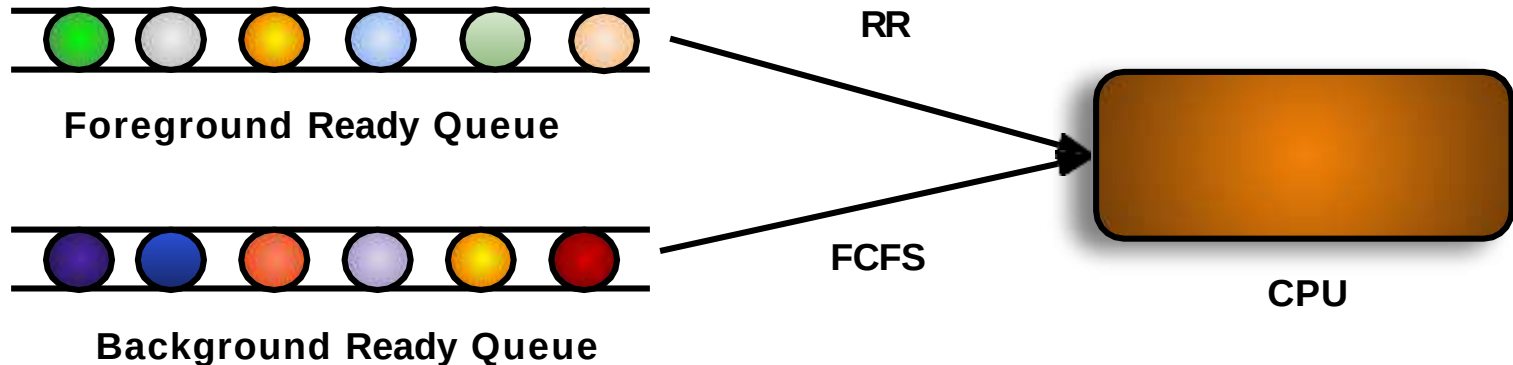
Combination of Priority and Round Robin

PID	Burst Time (BT)	Priority
1	4	3
2	5	2
3	8	2
4	7	1
5	3	3



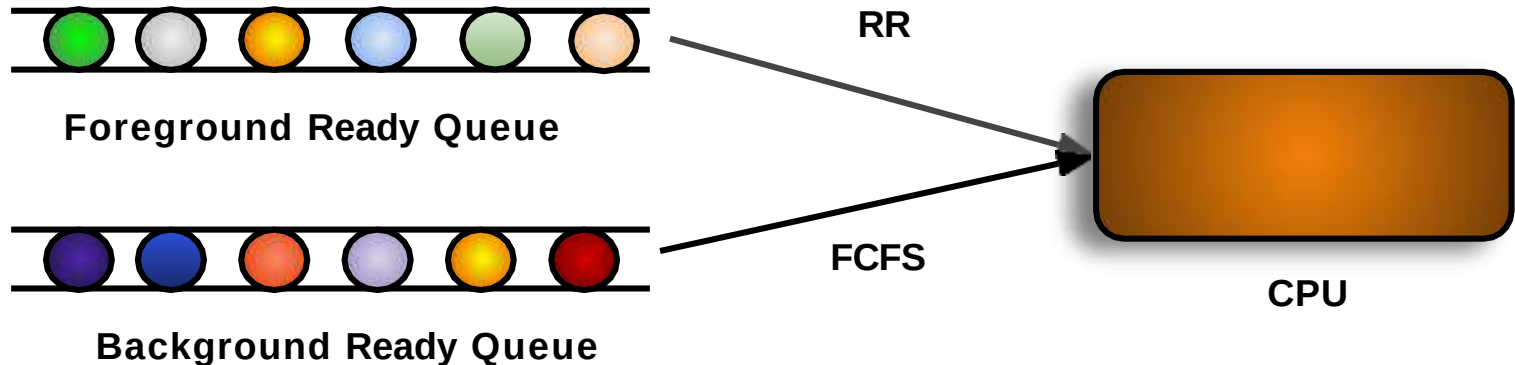
Multilevel Queue Scheduling

- Ready queue is partitioned into separate queues such as foreground (user's jobs), background (System's jobs)
- Process permanently in a given queue
- Each queue has its own scheduling algorithm



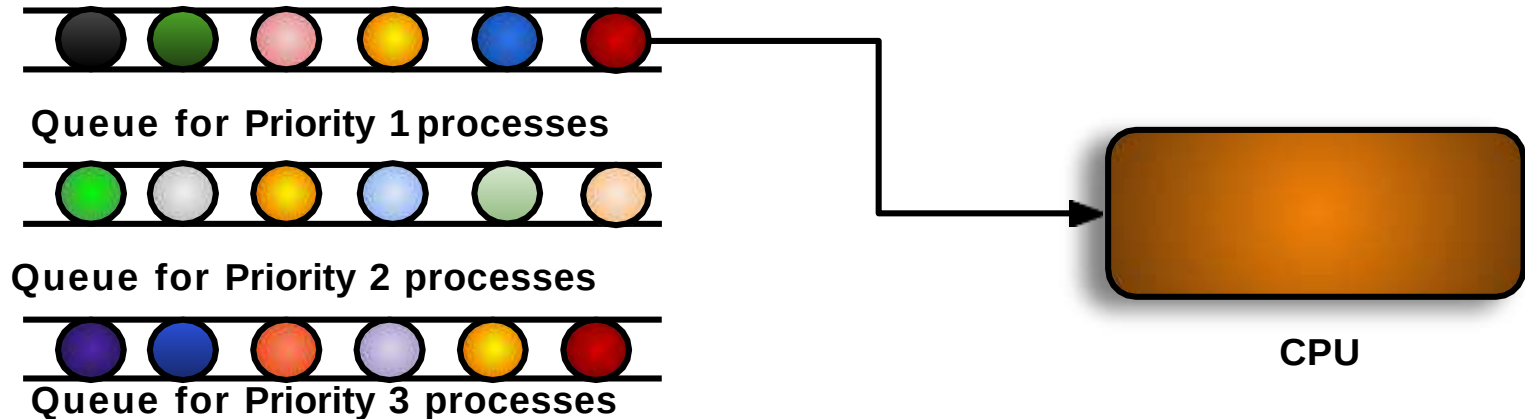
Multilevel Queue Scheduling

- Scheduling must be done between the queues
 - Fixed priority scheduling i.e. serve all from foreground then background. Possibility of starvation
 - Each queue get certain amount of CPU time in which it can schedule amongst its processes. Maybe 80% for foreground and 20% for background



Multilevel Queue Scheduling

- Another way we can consider one queue for each priority
- Once jobs in high priority queue are over then move to low priority queue
- Movement of a process from one queue to another is not possible



Multilevel Feedback Queue

- A process can move from various queues
- Scheduler is defined by following parameters
 - Number of queues
 - Scheduling algorithm for each queue
 - Method used to determine when to upgrade a process
 - Method used to determine when to demote a process
 - In which queue a process will enter at start

Multilevel Feedback Queue Example

- A new job enters in Q0
When it gains CPU, job receives 8 ms
If it does not finish in 8 sec, job is moved to Q1
- At Q1 job receives additional 16 ms
- If it still does not complete, it is preempted and moved to

Jobs in lower queue gets executed only if upper queue is empty

